

Babel

Code

Version 26.12.136372
2026/09/24

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	$\LaTeX$: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	16
4.2	Errors	24
4.3	More on selection	25
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	28
4.8	Shorthands	30
4.9	Language attributes	39
4.10	Support for saving and redefining macros	40
4.11	French spacing	42
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	49
4.15	Making glyphs available	50
4.15.1	Quotation marks	50
4.15.2	Letters	52
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	55
4.17	Load engine specific macros	55
4.18	Creating and modifying languages	55
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	68
4.21	French spacing (again)	74
4.22	Handle language system	75
4.23	Numerals	76
4.24	Casing	77
4.25	Getting info	78
4.26	BCP 47 related commands	79
5	Adjusting the Babel behavior	80
5.1	Cross referencing macros	83
5.2	Layout	85
5.3	Marks	87
5.4	Other packages	88
5.4.1	ifthen	88
5.4.2	varioref	88
5.4.3	hhline	89
5.5	Encoding and fonts	89
5.6	Basic bidi support	91
5.7	Local Language Configuration	94
5.8	Language options	95

6	The kernel of Babel	99
7	Error messages	99
8	Loading hyphenation patterns	103
9	luatex + xetex: common stuff	107
10	Hooks for XeTeX and LuaTeX	110
10.1	XeTeX	110
10.2	Support for interchar	112
10.3	Layout	114
10.4	8-bit TeX	116
10.5	LuaTeX	116
10.6	Southeast Asian scripts	123
10.7	CJK line breaking	124
10.8	Arabic justification	126
10.9	Common stuff	131
10.10	Automatic fonts and ids switching	131
10.11	Bidi	138
10.12	Layout	140
10.13	Lua: transforms	149
10.14	Lua: Auto bidi with basic and basic-r	159
11	Data for CJK	172
12	The ‘nil’ language	173
13	Calendars	174
13.1	Islamic	174
13.2	Hebrew	176
13.3	Persian	180
13.4	Coptic and Ethiopic	180
13.5	Julian	181
13.6	Buddhist	181
14	Support for Plain TeX (plain.def)	183
14.1	Not renaming hyphen.tex	183
14.2	Emulating some L ^A T _E X features	184
14.3	General tools	184
14.4	Encoding related macros	188
15	Acknowledgements	190

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `{(name=value)}`, or with a series of lines between `{(*name)}` and `{(/name)}`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 {(<version=26.12.136372>)}
2 {(<date=2026/09/24>)}
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 {(<*Basic macros>)} ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8   {\def#1{#2}}%
9   {\expandafter\def\expandafter#1\expandafter{#1#2}}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@cs#1{\csname bbl@#1\endcsname}
17 \def\bbl@c#1#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
```

```

19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \<.> for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35     \let\<\noexpand
36     \let\<\bbl@exp@en
37     \let\[\bbl@exp@ue
38     \edef\bbl@exp@aux{\endgroup#1}%
39     \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\@sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrode}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{ }%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@\expandafter{\the\toks@##1}%
120     \else
121       \toks@\expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize\undefined \else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax}%
129   \def\bbl@tempa{#1}%
130   \def\bbl@tempb{#2}%
131   \def\bbl@tempe{#3}}
132 \def\bbl@sreplace#1#2#3{%
133   \begingroup
134     \expandafter\bbl@parsedef\meaning#1\relax
135     \def\bbl@tempc{#2}%
136     \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137     \def\bbl@tempd{#3}%
138     \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139     \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140     \ifin@
141       \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142       \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143         \\makeatletter % "internal" macros with @ are assumed
144         \\scantokens{%
145           \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146           \noexpand\noexpand}%
147         \catcode64=\the\catcode64\relax}% Restore @
148     \else
149       \let\bbl@tempc\empty % Not \relax
150     \fi
151     \bbl@exp{% For the 'uplevel' assignments
152   \endgroup
153   \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bb@ifsamestring#1#2{%
156   \begingroup
157     \protected@edef\bb@tempb{#1}%
158     \edef\bb@tempb{\expandafter\strip@prefix\meaning\bb@tempb}%
159     \protected@edef\bb@tempc{#2}%
160     \edef\bb@tempc{\expandafter\strip@prefix\meaning\bb@tempc}%
161     \ifx\bb@tempb\bb@tempc
162       \aftergroup\@firstoftwo
163     \else
164       \aftergroup\@secondoftwo
165     \fi
166   \endgroup}
167 \chardef\bb@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bb@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bb@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bb@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal \let's made by \MakeUppercase and \MakeLowercase between things like \oe and \OE.

```

184 \def\bb@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187       {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bb@afterelse\expandafter\MakeUppercase
190   \else
191     \bb@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

Ordered key/value lists (aka associative arrays). The setter either replaces the current value if the key exists, or adds the new key/value at the end. The getter returns either the value if the key exists or \relax. The code is based on arguments delimited by the keys. In the getter, note the value is supposed not to start with \@empty.

```

196 \def\bb@array@set#1#2#3{%
197   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux##4{%
198     \ifx\bb@noelt##2%
199       \def#1{##1\bb@elt#2\@{##4}}%
200     \else
201       \def\bb@array@aux####1\bb@elt#2\@####2####3\bb@array@aux####4{%
202         \def#1{####1\bb@elt#2\@{####4}####3}}%
203       \expandafter\bb@array@aux#1\bb@array@aux{#3}%
204     \fi}%
205   \expandafter\bb@array@aux#1\bb@elt#2\@{\bb@noelt\bb@array@aux{#3}}
206 \def\bb@array@get#1#2#3{%
207   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux{%
208     \ifx\@empty##2\let#3\relax\else\def#3{##2}\fi}%

```

```
209 \expandafter\bbbl@array@aux#1\bbbl@elt#2\@empty\bbbl@array@aux}
```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with #'s. Used to deal with alph, Alph and frenchspacing when there are already changes (with `\babel@save`).

```
210 \def\bbbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
211 \toks@\expandafter\expandafter\expandafter{%
212 \csname extras\language\endcsname}%
213 \bbbl@exp{\in@{#1}{\the\toks@}}%
214 \ifin@else
215 \@temptokena{#2}%
216 \edef\bbbl@tempc{\the\@temptokena\the\toks@}%
217 \toks@\expandafter{\bbbl@tempc#3}%
218 \expandafter\edef\csname extras\language\endcsname{\the\toks@}%
219 \fi}
220 {/Basic macros}}
```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.

```
221 {(*Make sure ProvidesFile is defined)} ≡
222 \ifx\ProvidesFile\@undefined
223 \def\ProvidesFile#1[#2 #3 #4]{%
224 \wlog{File: #1 #4 #3 <#2>}%
225 \let\ProvidesFile\@undefined}
226 \fi
227 {/Make sure ProvidesFile is defined}}
```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```
228 {(*Define core switching macros)} ≡
229 \ifx\language\@undefined
230 \csname newcount\endcsname\language
231 \fi
232 {/Define core switching macros}}
```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```
233 {(*Define core switching macros)} ≡
234 \countdef\last@language=19
235 \def\addlanguage{\csname newlanguage\endcsname}
236 {/Define core switching macros}}
```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```
237 {*package}
238 \NeedsTeXFormat{LaTeX2e}
239 \ProvidesPackage{babel}%
240 [%@date@> v<@version@>
241 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]
```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

242 \@ifpackagewith{babel}{debug}
243 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
244 \let\bbl@debug\@firstofone
245 \ifx\directlua\@undefined\else
246 \directlua{
247     Babel = Babel or {}
248     Babel.debug = true }%
249 \input{babel-debug.tex}%
250 \fi}
251 {\providecommand\bbl@trace[1]{}%
252 \let\bbl@debug\@gobble
253 \ifx\directlua\@undefined\else
254 \directlua{
255     Babel = Babel or {}
256     Babel.debug = false }%
257 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

258 \def\bbl@error#1{% Implicit #2#3#4
259 \begingroup
260 \catcode`\=0 \catcode`\==12 \catcode`\`=12
261 \input errbabel.def
262 \endgroup
263 \bbl@error{#1}}
264 \def\bbl@warning#1{%
265 \begingroup
266 \def\{\{MessageBreak}%
267 \PackageWarning{babel}{#1}%
268 \endgroup}
269 \def\bbl@infowarn#1{%
270 \begingroup
271 \def\{\{MessageBreak}%
272 \PackageNote{babel}{#1}%
273 \endgroup}
274 \def\bbl@info#1{%
275 \begingroup
276 \def\{\{MessageBreak}%
277 \PackageInfo{babel}{#1}%
278 \endgroup}

```

Many of the following options don’t do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

279 <@Basic macros>
280 \@ifpackagewith{babel}{silent}
281 {\let\bbl@info\@gobble
282 \let\bbl@infowarn\@gobble
283 \let\bbl@warning\@gobble}
284 {}
285 %
286 \def\AfterBabelLanguage#1{%
287 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

288 \ifx\bbl@languages\@undefined\else
289 \begingroup
290 \catcode`\^^I=12
291 \@ifpackagewith{babel}{showlanguages}{%
292 \begingroup
293 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%

```

```

294      \wlog{<*languages>}%
295      \bbl@languages
296      \wlog{</languages>}%
297      \endgroup{}}
298  \endgroup
299  \def\bbl@elt#1#2#3#4{%
300    \ifnum#2=\z@
301      \gdef\bbl@nulllanguage{#1}%
302      \def\bbl@elt##1##2##3##4{%
303        \fi}%
304    \bbl@languages
305  \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```

306 \bbl@trace{Defining option 'base'}
307 \ifpackagewith{babel}{base}{%
308   \let\bbl@onlyswitch\@empty
309   \let\bbl@provide@locale\relax
310   \input babel.def
311   \let\bbl@onlyswitch\@undefined
312   \ifx\directlua\@undefined
313     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
314   \else
315     \input luababel.def
316     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
317   \fi
318   \DeclareOption{base}{}%
319   \DeclareOption{showlanguages}{}%
320   \ProcessOptions
321   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
322   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
323   \global\let\@ifl@ter@\@ifl@ter
324   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
325   \endinput}{}%

```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```

326 \bbl@trace{key=value and another general options}
327 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
328 \def\bbl@tempb#1.#2{% Removes trailing dot
329   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
330 \def\bbl@tempe#1=#2\@{
331   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
332 \def\bbl@tempd#1.#2\@nnil{%
333   \ifx\@empty#2%
334     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
335   \else
336     \in@{,provide=}{, #1}%
337     \ifin@
338       \edef\bbl@tempc{%
339         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
340     \else
341       \in@{$modifiers$}{$#1$}%

```

```

342 \ifin@
343 \bbl@tempe#2\@@
344 \else
345 \in@{=}{#1}%
346 \ifin@
347 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
348 \else
349 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
350 \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
351 \fi
352 \fi
353 \fi
354 \fi}
355 \let\bbl@tempc\@empty
356 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
357 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

358 \DeclareOption{KeepShorthandsActive}{}
359 \DeclareOption{activeacute}{}
360 \DeclareOption{activegrave}{}
361 \DeclareOption{debug}{}
362 \DeclareOption{debug-german}{} % Temporary
363 \DeclareOption{noconfigs}{}
364 \DeclareOption{showlanguages}{}
365 \DeclareOption{silent}{}
366 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
367 \chardef\bbl@iniflag\z@
368 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
369 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
370 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
371 \chardef\bbl@ldfflag\z@
372 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
373 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
374 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
375 \DeclareOption{licr=extended}{}
376 \DeclareOption{licr=unextended}{}
377 % Don't use. Experimental.
378 \newif\ifbbl@single
379 \DeclareOption{selectors=off}{\bbl@singletrue}
380 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key main, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

381 \let\bbl@opt@shorthands\@nnil
382 \let\bbl@opt@config\@nnil
383 \let\bbl@opt@main\@nnil
384 \let\bbl@opt@headfoot\@nnil
385 \let\bbl@opt@layout\@nnil
386 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

387 \def\bbl@tempa#1=#2\bbl@tempa{%
388 \bbl@csarg\ifx{opt@#1}\@nnil
389 \bbl@csarg\edef{opt@#1}{#2}%
390 \else
391 \bbl@error{bad-package-option}{#1}{#2}{}%
392 \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```
393 \let\bbl@language@opts\@empty
394 \DeclareOption*{%
395   \bbl@xin@{\string=}{\CurrentOption}%
396   \ifin@
397     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
398   \else
399     \bbl@add@list\bbl@language@opts{\CurrentOption}%
400   \fi}
```

Now we finish the first pass (and start over).

```
401 \ProcessOptions*
```

3.5. Post-process some options

```
402 \ifx\bbl@opt@provide\@nnil
403   \let\bbl@opt@provide\@empty %%% MOVE above
404 \else
405   \chardef\bbl@iniflag\@ne
406   \bbl@exp{\\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
407     \in@{,provide,}{, #1,}%
408     \ifin@
409       \def\bbl@opt@provide{#2}%
410     \fi}
411 \fi
```

If there is no `shorthands=` (*chars*), the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```
412 \bbl@trace{Conditional loading of shorthands}
413 \def\bbl@sh@string#1{%
414   \ifx#1\@empty\else
415     \ifx#1t\string~%
416     \else\ifx#1c\string,%
417     \else\string#1%
418   \fi\fi
419   \expandafter\bbl@sh@string
420 \fi}
421 \ifx\bbl@opt@shorthands\@nnil
422   \def\bbl@ifshorthand#1#2#3{#2}%
423 \else\ifx\bbl@opt@shorthands\@empty
424   \def\bbl@ifshorthand#1#2#3{#3}%
425 \else
```

The following macro tests if a shorthand is one of the allowed ones.

```
426 \def\bbl@ifshorthand#1{%
427   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
428   \ifin@
429     \expandafter\@firstoftwo
430   \else
431     \expandafter\@secondoftwo
432   \fi}
```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```
433 \edef\bbl@opt@shorthands{%
434   \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```
435 \bbl@ifshorthand{'}%
436   {\PassOptionsToPackage{activeacute}{babel}}{}
```

```

437 \bbl@ifshorthand{`}%
438 {\PassOptionsToPackage{activegrave}{babel}}{}
439 \fi\fi

```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\resetactivechars`, but seems to work.

```

440 \ifx\bbl@opt@headfoot\@nnil\else
441 \g@addto@macro\@resetactivechars{%
442 \set@typeset@protect
443 \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
444 \let\protect\noexpand}
445 \fi

```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to none.

```

446 \ifx\bbl@opt@safe\@undefined
447 \def\bbl@opt@safe{BR}
448 % \let\bbl@opt@safe\@empty % Pending of \cite
449 \fi

```

For layout an auxiliary macro is provided, available for packages and language styles. Optimization: if there is no layout, just do nothing.

```

450 \bbl@trace{Defining IfBabelLayout}
451 \ifx\bbl@opt@layout\@nnil
452 \newcommand\IfBabelLayout[3]{#3}%
453 \else
454 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
455 \in@{,layout,}{, #1,}%
456 \ifin@
457 \def\bbl@opt@layout{#2}%
458 \bbl@replace\bbl@opt@layout{ }{.}%
459 \fi}
460 \newcommand\IfBabelLayout[1]{%
461 \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
462 \ifin@
463 \expandafter\@firstoftwo
464 \else
465 \expandafter\@secondoftwo
466 \fi}
467 \fi
468 \<package>

```

3.6. Plain: `babel.def` (start)

Because of the way `docstrip` works, we need to insert some code for Plain here. However, the tools provided by the `babel` installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

469 \<core>
470 \ifx\ldf@quit\@undefined\else
471 \endinput\fi % Same line!
472 \<@Make sure ProvidesFile is defined@>
473 \ProvidesFile{babel.def}[\<@date@> v\<@version@> Babel common definitions]
474 \ifx\AtBeginDocument\@undefined
475 \<@Emulate LaTeX@>
476 \fi
477 \<@Basic macros@>
478 \</core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\text{\LaTeX}$. After it, we will resume the $\text{\LaTeX}$ -only stuff.

4. `babel.sty` and `babel.def` (common)

```

479 (*package|core)
480 \def\bbl@version{<@version>}
481 \def\bbl@date{<@date>}
482 <@Define core switching macros>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

483 \def\adddialect#1#2{%
484   \global\chardef#1#2\relax
485   \bbl@usehooks{adddialect}{#1}{#2}}%
486 \begingroup
487   \count@#1\relax
488   \def\bbl@elt##1##2##3##4{%
489     \ifnum\count@=##2\relax
490       \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
491       \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
492         set to \expandafter\string\csname l@##1\endcsname\\%
493         (\string\language\the\count@). Reported}%
494       \def\bbl@elt####1####2####3####4{%
495         \fi}%
496       \bbl@cs{languages}%
497       \endgroup

```

`\bbl@iflanguage` executes code only if the language `l@` exists. Otherwise raises an error.

The argument of `\bbl@fixname` has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when `\foreignlanguage` and the like appear in a `\MakeXXXcase`. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note `l@` is encapsulated, so that its case does not change.

```

498 \def\bbl@fixname#1{%
499   \begingroup
500   \def\bbl@tempe{l@}%
501   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
502   \bbl@tempd
503     {\lowercase\expandafter{\bbl@tempd}%
504      {\uppercase\expandafter{\bbl@tempd}%
505       \@empty
506        {\edef\bbl@tempd{\def\noexpand#1{#1}}%
507         \uppercase\expandafter{\bbl@tempd}}}%
508       {\edef\bbl@tempd{\def\noexpand#1{#1}}%
509        \lowercase\expandafter{\bbl@tempd}}}%
510     \@empty
511     \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
512   \bbl@tempd
513   \bbl@expf{\bbl@usehooks{language}{\language}{#1}}}%
514 \def\bbl@iflanguage#1{%
515   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that `sr-latn-ba` becomes `fr-Latn-BA`.

`\bbl@bcpllookup` returns `\bbl@bcp` with either the found ini tag or `\relax`. Temporary version, to be refactored with the new `\text_bcp_parse:n`, which is not currently fully expandable with invalid BCP 47 tags. The argument may be in some cases either a language name or a tag (e.g., `\foreignlanguage`. This was a huge mistake, because of the potential ambiguities (for example, the name `vai-latin` is also a syntactically valid tag).

```

516 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
517 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
518 \def\bbl@bcpllookup#1{%
519   \let\bbl@templ\@empty % language
520   \let\bbl@temps\@empty % script
521   \let\bbl@tempr\@empty % regions
522   \let\bbl@tempv\@empty % variant

```

```

523 \edef\bbbl@tempa{#1}%
524 \bbbl@replace\bbbl@tempa{-}{,}%
525 \bbbl@replace\bbbl@tempa_{}{,}%
526 \bbbl@foreach\bbbl@tempa{%
527   \ifx\bbbl@templ\@empty
528     \lowercase{\def\bbbl@templ{##1}}%
529   \else\ifnum\bbbl@cntchars{##1}=4
530     \bbbl@xin@{\@car##1\@nil}{0123456789}%
531     \ifin@
532       \def\bbbl@tempv{##1}% eg, 1901
533     \else
534       \uppercase{\edef\bbbl@tempb{\@car##1\@nil}}%
535       \lowercase{\edef\bbbl@tempb{\bbbl@tempb\@gobble##1}}%
536     \fi
537   \else\ifnum\bbbl@cntchars{##1}<4
538     \uppercase{\def\bbbl@tempv{##1}}%
539   \else\ifnum\bbbl@cntchars{##1}>4
540     \lowercase{\def\bbbl@tempv{##1}}%
541   \fi\fi\fi\fi}%
542 \bbbl@expf\\bbbl@bcpllookup@i{\bbbl@templ}{\bbbl@tempb}{\bbbl@tempv}}
543 \def\bbbl@bcpllookup@try#1{%
544   \ifx\bbbl@bcp\relax
545     \IfFileExists{babel-#1\bbbl@tempe.ini}{\edef\bbbl@bcp{#1\bbbl@tempe}}{}%
546   \fi}
547 \def\bbbl@bcpllookup@i#1#2#3#4{%
548   \let\bbbl@bcp\relax
549   \def\bbbl@tempe{#4}%
550   \ifx\bbbl@tempe\@empty\else
551     \edef\bbbl@tempe{-#4}%
552   \fi
553   \edef\bbbl@tempa{#2#3\bbbl@tempe}% en
554   \ifx\bbbl@tempa\@empty
555     \bbbl@bcpllookup@try{#1}%
556   \else
557     \edef\bbbl@tempa{#3\bbbl@tempe}% en-Latn
558     \ifx\bbbl@tempa\@empty
559       \bbbl@bcpllookup@try{#1-#2}%
560       \bbbl@bcpllookup@try{#1}%
561     \else
562       \edef\bbbl@tempa{#2\bbbl@tempe}% en-US, en-001
563       \ifx\bbbl@tempa\@empty
564         \bbbl@bcpllookup@try{#1-#3}%
565         \bbbl@bcpllookup@try{#1}%
566       \else % en-Latn-US
567         \bbbl@bcpllookup@try{#1-#2-#3}%
568         \bbbl@bcpllookup@try{#1-#2}%
569         \bbbl@bcpllookup@try{#1-#3}%
570         \bbbl@bcpllookup@try{#1}%
571       \fi
572     \fi
573   \fi
574   \ifx\bbbl@bcp\relax
575     \ifx\bbbl@tempe\@empty\else\bbbl@bcpllookup@i{#1}{#2}{#3}}\fi%
576 \fi}
577 %
578 \let\bbbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

579 \def\iflanguage#1{%

```

```

580 \bbl@iflanguage{#1}{%
581   \ifnum\csname l@#1\endcsname=\language
582     \expandafter\@firstoftwo
583   \else
584     \expandafter\@secondoftwo
585   \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

586 \let\bbl@select@type\z@
587 \edef\selectlanguage{%
588   \noexpand\protect
589   \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage_`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.

```

590 \ifx\@undefined\protect\let\protect\relax\fi

```

The following definition is preserved for backwards compatibility (e.g., *arabi*, *koma*). It is related to a trick for 2.09, now discarded.

```

591 \let\xstring\string

```

Since version 3.5 *babel* writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's *aftergroup* mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bbl@pop@language` to be executed at the end of the group. It calls `\bbl@set@language` with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bbl@language@stack` and initially empty.

```

592 \def\bbl@language@stack{}

```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```

593 \def\bbl@push@language{%
594   \ifx\language\@undefined\else
595     \ifx\currentgrouplevel\@undefined
596       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
597     \else
598       \ifnum\currentgrouplevel=\z@
599         \xdef\bbl@language@stack{\language+}%
600       \else
601         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
602       \fi
603     \fi
604   \fi}

```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro `\language`. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the ‘+’-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
605 \def\bbl@pop@lang#1+#2\@@{%
606   \edef\language{#1}%
607   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed T_EX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a ‘+’-sign (zero language names won’t occur as this macro will only be called after something has been pushed on the stack).

```
608 \let\bbl@ifrestoring\@secondoftwo
609 \def\bbl@pop@language{%
610   \expandafter\bbl@pop@lang\bbl@language@stack\@@
611   \let\bbl@ifrestoring\@firstoftwo
612   \expandafter\bbl@set@language\expandafter{\language}%
613   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to \bbl@set@language to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of \localeid. This means \l@. . . will be reserved for hyphenation patterns (so that two locales can share the same rules).

```
614 \chardef\localeid\z@
615 \gdef\bbl@id@last{0}    % No real need for a new counter
616 \def\bbl@id@assign{%
617   \bbl@ifunset\bbl@id@\language}%
618   {\count@\bbl@id@last\relax
619    \advance\count@\@ne
620    \global\bbl@csarg\chardef{id@\language}\count@
621    \xdef\bbl@id@last{\the\count@}%
622    \ifcase\bbl@engine\or
623      \directlua{
624        Babel.locale_props[\bbl@id@last] = {}
625        Babel.locale_props[\bbl@id@last].name = '\language'
626        Babel.locale_props[\bbl@id@last].vars = {}
627      }%
628    \fi}%
629   }%
630   \chardef\localeid\bbl@c{id@}}
```

The unprotected part of \selectlanguage. In case it is used as environment, declare \endselectlanguage, just for safety.

```
631 \let\bbl@select@opts\@empty
632 \expandafter\def\csname selectlanguage \endcsname{%
633   \@ifnextchar[\bbl@select@s{\bbl@select@s[]}}
634 \def\bbl@select@s[#1]#2{%
635   \def\bbl@select@opts{#1}%
636   \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\tw\fi
637   \bbl@push@language
638   \aftergroup\bbl@pop@language
639   \bbl@set@language{#2}}
640 \let\endselectlanguage\relax
```

\bbl@set@language The macro \bbl@set@language takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of \language. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in \language are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining \BabelContentsFiles, but make sure they are loaded inside a group (as aux, toc, lof, and lot do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from `hyperref`, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

641 \def\BabelContentsFiles{toc,lof,lot}
642 \def\bbl@set@language#1{% from selectlanguage, pop@
643 % The old buggy way. Preserved for compatibility, but simplified
644 \edef\language{\expandafter\string#1\@empty}%
645 \select@language{\language}%
646 \bbl@xin@{,main,},{, \bbl@select@opts,}%
647 \ifin@
648 \let\bbl@main@language\localename
649 \let\mainlocalename\localename
650 \fi
651 \let\bbl@select@opts\@empty
652 % write to aux files
653 \expandafter\ifx\csname date\language\endcsname\relax\else
654 \if@filesw
655 \bbl@xin@{,nofiles,},{, \bbl@select@opts,}%
656 \ifin@\else
657 \ifx\babel@aux\@gobbletwo\else % Set if single in the first, redundant
658 \bbl@savelastskip
659 \protected@write\@auxout{{}\string\babel@aux{\bbl@auxname}{}}%
660 \bbl@restorelastskip
661 \fi
662 \bbl@usehooks{write}{}%
663 \fi
664 \fi
665 \fi}
666 %
667 \let\bbl@restorelastskip\relax
668 \let\bbl@savelastskip\relax
669 %
670 \def\select@language#1{% from set@, babel@aux, babel@toc
671 \ifx\bbl@select@name\@empty
672 \def\bbl@select@name{select}%
673 \fi
674 % set hmap
675 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
676 % set name (when coming from babel@aux)
677 \edef\language{#1}%
678 \bbl@fixname\language
679 % define \localename when coming from set@, with a trick
680 \ifx\scantokens\undefined
681 \def\localename{??}%
682 \else
683 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
684 \fi
685 \bbl@provide@locale
686 \bbl@iflanguage\language{%
687 \let\bbl@select@type\z@
688 \expandafter\bbl@switch\expandafter{\language}}
689 \def\babel@aux#1#2{%
690 \select@language{#1}%
691 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
692 \writefile{##1}{\babel@toc{#1}{#2}\relax}}%
693 \def\babel@toc#1#2{%
694 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring `TEX` in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To

save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

695 \newif\ifbbl@usedategroup
696 \let\bbl@savextras\@empty
697 \def\bbl@switch#1{% from select@, foreign@
698   % restore
699   \originalTeX
700   \expandafter\def\expandafter\originalTeX\expandafter{%
701     \csname noextras#1\endcsname
702     \let\originalTeX\@empty
703     \babel@beginsave}%
704   \bbl@usehooks{afterreset}{}%
705   \languageshorthands{none}%
706   % set the locale id
707   \bbl@id@assign
708   % switch captions, date
709   \bbl@bsphack
710   \ifcase\bbl@select@type
711     \csname captions#1\endcsname\relax
712     \csname date#1\endcsname\relax
713   \else
714     \bbl@xin@{,captions,}{,\bbl@select@opts,}%
715     \ifin@
716       \csname captions#1\endcsname\relax
717     \fi
718     \bbl@xin@{,date,}{,\bbl@select@opts,}%
719     \ifin@ % if \foreign... within \<language>date
720       \csname date#1\endcsname\relax
721     \fi
722   \fi
723   \bbl@esphack
724   % switch extras
725   \csname bbl@preextras@#1\endcsname
726   \bbl@usehooks{beforeextras}{}%
727   \csname extras#1\endcsname\relax
728   \bbl@usehooks{afterextras}{}%
729   % > babel-ensure
730   % > babel-sh-<short>
731   % > babel-bidi
732   % > babel-fontspec
733   \let\bbl@savextras\@empty
734   % hyphenation - case mapping
735   \ifcase\bbl@opt@hyphenmap\or
736     \def\BabelLower##1##2{\lccode##1=##2\relax}%
737     \ifnum\bbl@hymapsel>4\else
738       \csname\language @bbl@hyphenmap\endcsname
739     \fi
740     \chardef\bbl@opt@hyphenmap\z@
741   \else
742     \ifnum\bbl@hymapsel>\bbl@opt@hyphenmap\else
743       \csname\language @bbl@hyphenmap\endcsname
744     \fi
745   \fi
746   \let\bbl@hymapsel\@cclv
747   % hyphenation - select rules

```

```

748 \ifnum\csname l@\language\endcsname=\l@unhyphenated
749 \edef\bbl@tempa{u}%
750 \else
751 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
752 \fi
753 % linebreaking - handle u, e, k (v in the future)
754 \bbl@xin@{/u}{/\bbl@tempa}%
755 \ifin@else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
756 \ifin@else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
757 \ifin@else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
758 \ifin@else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
759 % hyphenation - save mins
760 \babel@savevariable\lefthyphenmin
761 \babel@savevariable\righthyphenmin
762 \ifnum\bbl@engine=\@ne
763 \babel@savevariable\hyphenationmin
764 \fi
765 \ifin@
766 % unhyphenated/kashida/elongated/padding = allow stretching
767 \language\l@unhyphenated
768 \babel@savevariable\emergencystretch
769 \emergencystretch\maxdimen
770 \babel@savevariable\hbadness
771 \hbadness\@M
772 \else
773 % other = select patterns
774 \bbl@patterns{#1}%
775 \fi
776 % hyphenation - set mins
777 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
778 \set@hyphenmins\tw@\thr@@\relax
779 \@nameuse{\bbl@hyphenmins@}%
780 \else
781 \expandafter\expandafter\expandafter\set@hyphenmins
782 \csname #1hyphenmins\endcsname\relax
783 \fi
784 \@nameuse{\bbl@hyphenmins@}%
785 \@nameuse{\bbl@hyphenmins@\language}%
786 \@nameuse{\bbl@hyphenatmin@}%
787 \@nameuse{\bbl@hyphenatmin@\language}%
788 \let\bbl@selectorname\@empty

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

789 \edef\otherlanguage{%
790 \noexpand\protect
791 \expandafter\noexpand\csname otherlanguage \endcsname}
792 \expandafter\def\csname otherlanguage \endcsname{%
793 \@ifstar{\@nameuse{otherlanguage*}}{\bbl@otherlanguage}
794 \def\bbl@otherlanguage#1{%
795 \def\bbl@selectorname{other}%
796 \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\thr@@\fi
797 \csname selectlanguage \endcsname{#1}%
798 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

799 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of

`\foreign@language.`

```

800 \expandafter\def\csname otherlanguage*\endcsname{%
801   \ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
802 \def\bbl@otherlanguage@s[#1]#2{%
803   \def\bbl@selectorname{other*}%
804   \ifnum\bbl@hymapset=\@ccclv\chardef\bbl@hymapset4\relax\fi
805   \def\bbl@select@opts{#1}%
806   \foreign@language{#2}}

```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```

807 \expandafter\let\csname endotherlanguage*\endcsname\relax

```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the `\extras<language>` command doesn’t make any `\global` changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a ‘text’ command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```

808 \providecommand\bbl@beforeforeign{}
809 \edef\foreignlanguage{%
810   \noexpand\protect
811   \expandafter\noexpand\csname foreignlanguage \endcsname}
812 \expandafter\def\csname foreignlanguage \endcsname{%
813   \@ifstar\bbl@foreign@s\bbl@foreign@x}
814 \providecommand\bbl@foreign@x[3][]{%
815   \begingroup
816     \def\bbl@selectorname{foreign}%
817     \def\bbl@select@opts{#1}%
818     \let\BabelText\@firstofone
819     \bbl@beforeforeign
820     \foreign@language{#2}%
821     \bbl@usehooks{foreign}{}%
822     \BabelText{#3}% Now in horizontal mode!
823   \endgroup}
824 \def\bbl@foreign@s#1#2{%
825   \begingroup
826     {\par}%
827     \def\bbl@selectorname{foreign*}%
828     \let\bbl@select@opts\@empty
829     \let\BabelText\@firstofone
830     \foreign@language{#1}%
831     \bbl@usehooks{foreign*}{}%
832     \bbl@dirparastext
833     \BabelText{#2}% Still in vertical mode!
834   {\par}%
835   \endgroup}
836 \providecommand\BabelWrapText[1]{%

```

```

837 \def\bbl@tempa{\def\BabelText####1}%
838 \expandafter\bbl@tempa\expandafter{\BabelText{#1}}

```

\foreign@language This macro does the work for \foreignlanguage and the other language* environment. First we need to store the name of the language and check that it is a known language. Then it just calls bbl@switch.

```

839 \def\foreign@language#1{%
840 % set name
841 \edef\language#1%
842 \ifbbl@usedategroup
843 \bbl@add\bbl@select@opts{,date,}%
844 \bbl@usedategroupfalse
845 \fi
846 \bbl@fixname\language
847 \let\localename\language
848 \bbl@provide@locale
849 \bbl@iflanguage\language{%
850 \let\bbl@select@type@ne
851 \expandafter\bbl@switch\expandafter{\language}}

```

The following macro executes conditionally some code based on the selector being used.

```

852 \def\IfBabelSelectorTF#1{%
853 \bbl@xin{,\bbl@selectorname,}{,\zap@space#1 \@empty,}%
854 \ifin@
855 \expandafter\@firstoftwo
856 \else
857 \expandafter\@secondoftwo
858 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

859 \let\bbl@hyphlist\@empty
860 \let\bbl@hyphenation@\relax
861 \let\bbl@pttnlist\@empty
862 \let\bbl@patterns@\relax
863 \let\bbl@hymapsel=\@cclv
864 \def\bbl@patterns#1{%
865 \language=\expandafter\ifx\csname l@#1:f@encoding\endcsname\relax
866 \csname l@#1\endcsname
867 \edef\bbl@tempa{#1}%
868 \else
869 \csname l@#1:f@encoding\endcsname
870 \edef\bbl@tempa{#1:f@encoding}%
871 \fi
872 \@expandtwoargs\bbl@usehooks{patterns}{{#1}{\bbl@tempa}}%
873 % > luatex
874 \@ifundefined{bbl@hyphenation@}{{}% Can be \relax!
875 \begingroup
876 \bbl@xin{,\number\language,}{,\bbl@hyphlist}%
877 \ifin@\else
878 \@expandtwoargs\bbl@usehooks{hyphenation}{{#1}{\bbl@tempa}}%
879 \hyphenation%
880 \bbl@hyphenation@
881 \@ifundefined{bbl@hyphenation@#1}%
882 \@empty
883 {\space\csname bbl@hyphenation@#1\endcsname}}%

```

```

884      \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
885      \fi
886      \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change `\languagename` and when the hyphenation rules specified were not loaded it has no effect. Note however, `\lccode's` and font encodings are not set at all, so in most cases you should use `otherlanguage*`.

```

887 \def\hyphenrules#1{%
888   \edef\bbl@tempf{#1}%
889   \bbl@fixname\bbl@tempf
890   \bbl@iflanguage\bbl@tempf{%
891     \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
892     \ifx\languageshorthands\undefined\else
893       \languageshorthands{none}%
894     \fi
895     \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
896       \set@hyphenmins\tw@\thr@@\relax
897     \else
898       \expandafter\expandafter\expandafter\set@hyphenmins
899       \csname\bbl@tempf hyphenmins\endcsname\relax
900     \fi}}
901 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\(language)hyphenmins` is already defined this command has no effect.

```

902 \def\providehyphenmins#1#2{%
903   \expandafter\ifx\csname #1hyphenmins\endcsname\relax
904     \@namedef{#1hyphenmins}{#2}%
905   \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

906 \def\set@hyphenmins#1#2{%
907   \lefthyphenmin#1\relax
908   \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_\epsilon$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by `babel`.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

909 \ifx\ProvidesFile\@undefined
910   \def\ProvidesLanguage#1[#2 #3 #4]{%
911     \wlog{Language: #1 #4 #3 <#2>}%
912   }
913 \else
914   \def\ProvidesLanguage#1{%
915     \begingroup
916       \catcode`\ 10 %
917       \@makeother\/%
918       \@ifnextchar[%]
919       {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}
920   \def\@provideslanguage#1[#2]{%
921     \wlog{Language: #1 #2}%
922     \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
923   \endgroup}
924 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\TeX$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```
925 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi
```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```
926 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi
```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```
927 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}{}
928 \let\uselocale\setlocale
929 \let\locale\setlocale
930 \let\selectlocale\setlocale
931 \let\textlocale\setlocale
932 \let\textlanguage\setlocale
933 \let\languagegettext\setlocale
```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn’t been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\LaTeX 2_{\epsilon}$, so we can safely use its error handling interface. Otherwise we’ll have to ‘keep it simple’.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```
934 \edef\bbl@nulllanguage{\string\language=0}
935 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
936 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
937   \global\@namedef{#2}{\textbf{?#1?}}}%
938   \@nameuse{#2}%
939   \edef\bbl@tempa{#1}%
940   \bbl@sreplace\bbl@tempa{name}}}%
941   \bbl@warning{%
942     \@backslashchar#1 not set for '\language'. Please,\\%
943     define it after the language has been loaded\\%
944     (typically in the preamble) with:\\%
945     \string\setlocalecaption{\language}\bbl@tempa{.}\\%
946     Feel free to contribute on github.com/latex3/babel.\\%
947     Reported}}
948 \def\bbl@tentative{\protect\bbl@tentative@i}
949 \def\bbl@tentative@i#1{%
950   \bbl@warning{%
951     Some functions for '#1' are tentative.\\%
952     They might not work as expected and their behavior\\%
953     could change in the future.\\%
954     Reported}}
955 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}}{}{}
956 \def\@nopatterns#1{%
957   \bbl@warning
958     {No hyphenation patterns were preloaded for\\%
959     the language '#1' into the format.\\%
960     Please, configure your TeX system to add them and\\%
961     rebuild the format. Now I will use the patterns\\%
962     preloaded for \bbl@nulllanguage\space instead}}
963 \let\bbl@usehooks@gobbletwo
```

Here ended the now discarded switch.def.
 Here also (currently) ends the base option.
 964 \ifx\babel@onlyswitch\@empty\endinput\fi

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\babel@e<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\babel@e<language>` contains `\babel@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\babel@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

965 \babel@trace{Defining babelensure}
966 \newcommand\babelensure[2][]{%
967   \AddBabelHook{babel-ensure}{afterextras}{%
968     \ifcase\babel@select@type
969       \babel@cl{e}%
970     \fi}%
971   \begingroup
972     \let\babel@ens@include\@empty
973     \let\babel@ens@exclude\@empty
974     \def\babel@ens@fontenc{\relax}%
975     \def\babel@tempb##1{%
976       \ifx\@empty##1\else\noexpand##1\expandafter\babel@tempb\fi}%
977     \edef\babel@tempa{\babel@tempb#1\@empty}%
978     \def\babel@tempb##1=##2\@{\@namedef{\babel@ens@##1}{##2}}%
979     \babel@foreach\babel@tempa{\babel@tempb##1\@}%
980     \def\babel@tempc{\babel@ensure}%
981     \expandafter\babel@add\expandafter\babel@tempc\expandafter{%
982       \expandafter{\babel@ens@include}}%
983     \expandafter\babel@add\expandafter\babel@tempc\expandafter{%
984       \expandafter{\babel@ens@exclude}}%
985     \toks@\expandafter{\babel@tempc}%
986     \babel@exp{%
987   \endgroup
988   \def\<babel@e#2>{\the\toks@{\babel@ens@fontenc}}}%
989 \def\babel@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
990   \def\babel@tempb##1{% elt for (excluding) \babel@captionslist list
991     \ifx##1\undefined % 3.32 - Don't assume the macro exists
992       \edef##1{\noexpand\babel@nocaption
993         {\babel@stripslash##1}{\language\babel@stripslash##1}}%
994     \fi
995     \ifx##1\@empty\else
996       \in@{##1}{#2}%
997       \ifin@ \else
998         \let\babel@tempc\language\name
999         \babel@replace\babel@tempc{-}{@}%
1000         \babel@ifunset{\babel@ensure@\babel@tempc}%
1001         {\babel@exp{%
1002           \\\DeclareRobustCommand\<babel@ensure@\babel@tempc>[1]{%
1003             \\\foreignlanguage{\language\name}%
1004             {\ifx\relax#3\else
1005               \\\fontencoding{#3}\selectfont
1006             \fi
1007             #####1}}}%
1008         {}%
1009         \toks@\expandafter{##1}%
1010         \edef##1{%
1011           \babel@csarg\noexpand{ensure@\babel@tempc}%

```

```

1012         {\the\toks@}}}%
1013     \fi
1014     \expandafter\babel@tempb
1015     \fi}%
1016     \expandafter\babel@tempb\babel@captionslist\today\@empty
1017     \def\babel@tempa##1{% elt for include list
1018         \ifx##1\@empty\else
1019             \let\babel@tempc\language\language
1020             \babel@replace\babel@tempc{-}{@}%
1021             \babel@csarg\in@{ensure@\babel@tempc\expandafter}\expandafter{##1}%
1022             \ifin@\else
1023                 \babel@tempb##1\@empty
1024             \fi
1025             \expandafter\babel@tempa
1026         \fi}%
1027     \babel@tempa#1\@empty}
1028 \def\babel@captionslist{%
1029     \prefacename\refname\abstractname\bibname\chaptername\appendixname
1030     \contentsname\listfigurename\listtablename\indexname\figurename
1031     \tablename\partname\enclname\ccname\headtoname\pagename\seename
1032     \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```

1033 \babel@trace{Short tags}
1034 \newcommand\babeltags[1]{%
1035     \edef\babel@tempa{\zap@space#1 \@empty}%
1036     \def\babel@tempb##1=##2\@{
1037         \edef\babel@tempc{
1038             \noexpand\newcommand
1039             \expandafter\noexpand\csname ##1\endcsname{
1040                 \noexpand\protect
1041                 \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1042             \noexpand\newcommand
1043             \expandafter\noexpand\csname text##1\endcsname{
1044                 \noexpand\foreignlanguage{##2}}}
1045         \babel@tempc}%
1046     \babel@for\babel@tempa\babel@tempa{
1047         \expandafter\babel@tempb\babel@tempa\@{

```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```

1048 \babel@trace{Compatibility with language.def}
1049 \ifx\directlua\@undefined\else
1050     \ifx\babel@luapatterns\@undefined
1051         \input luababel.def
1052     \fi
1053 \fi
1054 \ifx\babel@languages\@undefined
1055     \ifx\directlua\@undefined
1056         \openin0 = language.def
1057         \ifeof0
1058             \closein0
1059             \message{I couldn't find the file language.def}
1060         \else
1061             \closein0
1062             \begingroup
1063                 \def\addlanguage#1#2#3#4#5{%

```

```

1064      \expandafter\ifx\csname lang@#1\endcsname\relax\else
1065      \global\expandafter\let\csname l@#1\expandafter\endcsname
1066      \csname lang@#1\endcsname
1067      \fi}%
1068      \def\uselanguage#1{%
1069      \input language.def
1070      \endgroup
1071      \fi
1072      \fi
1073      \chardef\l@english\z@
1074      \fi

```

\addto It takes two arguments, a *(control sequence)* and TeX-code to be added to the *(control sequence)*.

If the *(control sequence)* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```

1075 \def\addto#1#2{%
1076   \ifx#1\@undefined
1077     \def#1{#2}%
1078   \else
1079     \ifx#1\relax
1080       \def#1{#2}%
1081     \else
1082       {\toks@\expandafter{#1#2}%
1083        \xdef#1{\the\toks@}}%
1084     \fi
1085   \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1086 \bbl@trace{Hooks}
1087 \newcommand\AddBabelHook[3][]{%
1088   \bbl@iifunset\bbl@hk@#2}{\EnableBabelHook{#2}}}%
1089   \def\bbl@tempa##1,##3=\@empty{\def\bbl@tempb{##2}}}%
1090   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1091   \bbl@iifunset\bbl@ev@#2@#3@#1{%
1092     {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1093     {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1094   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1095 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1096 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1097 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1098 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1099   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1100   \def\bbl@elth##1{%
1101     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}}%
1102   \bbl@cs{ev@#2@}%
1103   \ifx\language\@undefined\else % Test required for Plain (?)
1104     \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1105   \def\bbl@elth##1{%
1106     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1@#3}}}%
1107   \bbl@cs{ev@#2@#1}%
1108   \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1109 \def\bbl@evargs{,% <- don't delete this comma

```

```

1110 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1111 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1112 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1113 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1114 beforestart=0,language=2,begindocument=1}
1115 \ifx\NewHook\@undefined\else % Test for Plain (?)
1116 \def\bbl@tempa#1=#2\@{\NewHook{babel/#1}\NewHook{babel/*/#1}}
1117 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@}
1118 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1119 \providecommand\PassOptionsToLocale[2]{%
1120 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit \LdfInit macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, '=', because it is sometimes used in constructions with the \let primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to \LdfInit is a control sequence. We do that by looking at the first token after passing #2 through string. When it is equal to \@backslashchar we are dealing with a control sequence which we can compare with \@undefined.

If so, we call \ldf@quit to set the main language, restore the category code of the @-sign and call \endinput

When #2 was *not* a control sequence we construct one and compare it with \relax.

Finally we check \originalTeX.

```

1121 \bbl@trace{Macros for setting language files up}
1122 \def\bbl@ldfinit{%
1123 \let\bbl@screset\@empty
1124 \let\BabelStrings\bbl@opt@string
1125 \let\BabelOptions\@empty
1126 \let\BabelLanguages\relax
1127 \ifx\originalTeX\@undefined
1128 \let\originalTeX\@empty
1129 \else
1130 \originalTeX
1131 \fi}
1132 \def\LdfInit#1#2{%
1133 \chardef\atcatcode=\catcode`\@
1134 \catcode`\@=11\relax
1135 \chardef\eqcatcode=\catcode`\=
1136 \catcode`\==12\relax
1137 \ifpackagewith{babel}{ensureinfo=off}{}%
1138 {\ifx\InputIfFileExists\@undefined\else
1139 \bbl@ifunset{\bbl@lname@#1}%
1140 {\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1141 \def\language#1}%
1142 \bbl@id@assign
1143 \bbl@load@info{#1}}}%
1144 {}%
1145 \fi}%
1146 \expandafter\if\expandafter\@backslashchar
1147 \expandafter\@car\string#2\@nil
1148 \ifx#2\@undefined\else
1149 \ldf@quit{#1}%

```

```

1150 \fi
1151 \else
1152 \expandafter\ifx\csname#2\endcsname\relax\else
1153 \ldf@quit{#1}%
1154 \fi
1155 \fi
1156 \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1157 \def\ldf@quit#1{%
1158 \expandafter\main@language\expandafter{#1}%
1159 \catcode`\@=\atcatcode \let\atcatcode\relax
1160 \catcode`\==\eqcatcode \let\eqcatcode\relax
1161 \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1162 \def\bbl@afterldf{%
1163 \bbl@afterlang
1164 \let\bbl@afterlang\relax
1165 \let\BabelModifiers\relax
1166 \let\bbl@screset\relax}%
1167 \def\ldf@finish#1{%
1168 \loadlocalcfg{#1}%
1169 \bbl@afterldf
1170 \expandafter\main@language\expandafter{#1}%
1171 \catcode`\@=\atcatcode \let\atcatcode\relax
1172 \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in $\LaTeX$.

```

1173 \@onlypreamble\LdfInit
1174 \@onlypreamble\ldf@quit
1175 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1176 \def\main@language#1{%
1177 \def\bbl@main@language{#1}%
1178 \let\language\bbl@main@language
1179 \let\localename\bbl@main@language
1180 \let\mainlocalename\bbl@main@language
1181 \bbl@id@assign
1182 \ifcase\bbl@engine\or
1183 \ifx\setattribute\@undefined\else
1184 \setattribute\bbl@attr@locale\localeid
1185 \fi
1186 \fi
1187 \bbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1188 \def\bbl@beforestart{%

```

```

1189 \def\@nolanerr##1{%
1190   \bbl@carg\chardef{l@##1}\z@
1191   \bbl@warning{Undefined language '##1' in aux.\@Reported}}%
1192 \bbl@usehooks{beforestart}{}%
1193 \global\let\bbl@beforestart\relax
1194 \AtBeginDocument{%
1195   {\@nameuse{bbl@beforestart}}% Group!
1196   \if@filesw
1197     \providecommand\babel@aux[2]{}%
1198     \immediate\write\@mainaux{\unexpanded{%
1199       \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1200     \immediate\write\@mainaux{\string\@nameuse{bbl@beforestart}}}%
1201   \fi
1202   \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1203   \ifbbl@single % must go after the line above.
1204     \renewcommand\selectlanguage[1]{}%
1205     \renewcommand\foreignlanguage[2]{#2}%
1206     \global\let\babel@aux\@gobbletwo % Also as flag
1207   \fi}

A bit of optimization. Select in heads/feet the language only if necessary.

1208 \def\select@language@x#1{%
1209   \ifcase\bbl@select@type
1210     \bbl@ifsamestring\language{#1}{\select@language{#1}}%
1211   \else
1212     \select@language{#1}%
1213   \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1214 \bbl@trace{Shorhands}
1215 \def\bbl@withactive#1#2{%
1216   \begingroup
1217   \lccode`~=#2\relax
1218   \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\LaTeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1219 \def\bbl@add@special#1% 1:a macro like "\", \?, etc.
1220 \bbl@add\dospecials{\do#1}% test @sanitize = \relax, for back. compat.
1221 \bbl@ifunset{@sanitize}{\bbl@add\@sanitize{\makeother#1}}%
1222 \ifx\nfss@catcodes\undefined\else
1223   \begingroup
1224     \catcode`#1\active
1225     \nfss@catcodes
1226     \ifnum\catcode`#1=\active
1227       \endgroup
1228       \bbl@add\nfss@catcodes{\@makeother#1}%
1229     \else
1230       \endgroup
1231     \fi
1232   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char⟨char⟩` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char⟨char⟩` by default (`⟨char⟩` being the character to be made active). Later its definition can be changed to expand to `\active@char⟨char⟩` by calling `\bbl@activate{⟨char⟩}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\⟨level⟩@group`, `⟨level⟩@active` and `⟨next-level⟩@active` (except in system).

```
1233 \def\bbl@active@def#1#2#3#4{%
1234   \namedef{#3#1}{%
1235     \expandafter\ifx\csname#2@sh@#1@\endcsname\relax
1236       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1237     \else
1238       \bbl@afterfi\csname#2@sh@#1@\endcsname
1239     \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```
1240   \long\@namedef{#3@arg#1}##1{%
1241     \expandafter\ifx\csname#2@sh@#1@\string##1@\endcsname\relax
1242       \bbl@afterelse\csname#4#1\endcsname##1%
1243     \else
1244       \bbl@afterfi\csname#2@sh@#1@\string##1@\endcsname
1245     \fi}}%

```

`\initiate@active@char` calls `\@initiate@active@char` with 3 arguments. All of them are the same character with different catcodes: active, other (`\string’ed`) and the original one. This trick simplifies the code a lot.

```
1246 \def\@initiate@active@char#1{%
1247   \bbl@ifunset{active@char\string#1}%
1248   {\bbl@withactive
1249     {\expandafter\@initiate@active@char\expandafter}#1\string#1#1}%
1250   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them `\relax` and preserving some degree of protection).

```
1251 \def\@initiate@active@char#1#2#3{%
1252   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1253   \ifx#1\@undefined
1254     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1255   \else
1256     \bbl@csarg\let{oridef@#2}#1%
1257     \bbl@csarg\edef{oridef@#2}{%
1258       \let\noexpand#1%
1259       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1260   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define `\normal@char⟨char⟩` to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example `'`) the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to `"8000 a posteriori`).

```
1261   \ifx#1#3\relax

```

```

1262 \expandafter\let\csname normal@char#2\endcsname#3%
1263 \else
1264 \bbl@info{Making #2 an active character}%
1265 \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1266 \@namedef{normal@char#2}{%
1267 \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1268 \else
1269 \@namedef{normal@char#2}{#3}%
1270 \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at `\begin{document}`. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of `\bibitem` for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1271 \bbl@restoreactive{#2}%
1272 \AtBeginDocument{%
1273 \catcode`#2\active
1274 \if@filesw
1275 \immediate\write\@mainaux{\catcode`\string#2\active}%
1276 \fi}%
1277 \expandafter\bbl@add@special\csname#2\endcsname
1278 \catcode`#2\active
1279 \fi

```

Now we have set `\normal@char<char>`, we must define `\active@char<char>`, to be executed when the character is activated. We define the first level expansion of `\active@char<char>` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active<char>` to start the search of a definition in the user, language and system levels (or eventually `normal@char<char>`).

```

1280 \let\bbl@tempa\@firstoftwo
1281 \if\string^#2%
1282 \def\bbl@tempa{\noexpand\textormath}%
1283 \else
1284 \ifx\bbl@mathnormal\@undefined\else
1285 \let\bbl@tempa\bbl@mathnormal
1286 \fi
1287 \fi
1288 \expandafter\edef\csname active@char#2\endcsname{%
1289 \bbl@tempa
1290 {\noexpand\if@safe@actives
1291 \noexpand\expandafter
1292 \expandafter\noexpand\csname normal@char#2\endcsname
1293 \noexpand\else
1294 \noexpand\expandafter
1295 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1296 \noexpand\fi}%
1297 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1298 \bbl@csarg\edef{doactive#2}{%
1299 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

$$\text{active@prefix}\langle char \rangle \text{normal@char}\langle char \rangle$$

(where `\active@char<char>` is *one* control sequence!).

```

1300 \bbl@csarg\edef{active@#2}{%
1301 \noexpand\active@prefix\noexpand#1%
1302 \expandafter\noexpand\csname active@char#2\endcsname}%
1303 \bbl@csarg\edef{normal@#2}{%
1304 \noexpand\active@prefix\noexpand#1%
1305 \expandafter\noexpand\csname normal@char#2\endcsname}%
1306 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn't exist we check for a shorthand with an argument.

```
1307 \bbl@active@def#2\user@group{user@active}{language@active}%
1308 \bbl@active@def#2\language@group{language@active}{system@active}%
1309 \bbl@active@def#2\system@group{system@active}{normal@char}%
```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ' ' ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```
1310 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1311 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1312 \expandafter\edef\csname\user@group @sh#2@\string\protect\endcsname
1313 {\expandafter\noexpand\csname user@active#2\endcsname}%
```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (') active we need to change `\pr@m@s` as well. Also, make sure that a single ' in math mode 'does the right thing'. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```
1314 \if\string'#2%
1315 \let\prim@s\bbl@prim@s
1316 \let\active@math@prime#1%
1317 \fi
1318 \bbl@usehooks{initiateactive}{{#1}{#2}{#3}}}
```

The following package options control the behavior of shorthands in math mode.

```
1319 <(*More package options)> ≡
1320 \DeclareOption{math=active}{}
1321 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1322 <(/More package options)>
```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```
1323 \ifpackagewith{babel}{KeepShorthandsActive}%
1324 {\let\bbl@restoreactive@gobble}%
1325 {\def\bbl@restoreactive#1{%
1326 \bbl@exp{%
1327 \\\AfterBabelLanguage\\CurrentOption
1328 {\catcode`#1=\the\catcode`#1\relax}%
1329 \\\AtEndOfPackage
1330 {\catcode`#1=\the\catcode`#1\relax}}}%
1331 \AtEndOfPackage{\let\bbl@restoreactive@gobble}}
```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```
1332 \def\bbl@sh@select#1#2{%
1333 \expandafter\ifx\csname#1@sh#2@sel\endcsname\relax
1334 \bbl@afterelse\bbl@scndcs
1335 \else
1336 \bbl@afterfi\csname#1@sh#2@sel\endcsname
1337 \fi}
```

\active@prefix Used in the expansion of active characters has a function similar to \OT1-cmd in that it \protects the active character whenever \protect is *not* \@typeset@protect. The \gobble is needed to remove a token such as \activechar: (when the double colon was the active character to be dealt with). There are two definitions, depending on \ifincsname is available. If there is, the expansion will be more robust.

```

1338 \begingroup
1339 \bbl@ifunset{ifincsname}
1340 {\gdef\active@prefix#1{%
1341   \ifx\protect\@typeset@protect
1342   \else
1343     \ifx\protect\@unexpandable@protect
1344       \noexpand#1%
1345     \else
1346       \protect#1%
1347     \fi
1348     \expandafter\@gobble
1349   \fi}}
1350 {\gdef\active@prefix#1{%
1351   \ifincsname
1352     \string#1%
1353     \expandafter\@gobble
1354   \else
1355     \ifx\protect\@typeset@protect
1356     \else
1357       \ifx\protect\@unexpandable@protect
1358         \noexpand#1%
1359       \else
1360         \protect#1%
1361       \fi
1362       \expandafter\expandafter\expandafter\@gobble
1363     \fi
1364   \fi}}
1365 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char<char>. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string'ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1366 \newif\if@safe@actives
1367 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1368 \def\bbl@restore@actives{\if@safe@actives\@safe@activessfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char<char> in the case of \bbl@activate, or \normal@char<char> in the case of \bbl@deactivate.

```

1369 \chardef\bbl@activated\z@
1370 \def\bbl@activate#1{%
1371   \chardef\bbl@activated\@ne
1372   \bbl@withactive{\expandafter\let\expandafter}#1%
1373   \csname bbl@active@\string#1\endcsname}
1374 \def\bbl@deactivate#1{%
1375   \chardef\bbl@activated\tw@
1376   \bbl@withactive{\expandafter\let\expandafter}#1%

```

```
1377 \csname bbl@normal@\string#1\endcsname}
```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```
1378 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1379 \def\bbl@scndcs#1#2{\csname#2\endcsname}
```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The $\TeX$ code in text mode, (2) the string for `hyperref`, (3) the $\TeX$ code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn’t discriminate the mode). This macro may be used in `ldf` files.

```
1380 \def\babel@texpdf#1#2#3#4{%
1381 \ifx\texorpdfstring\undefined
1382 \textormath{#1}{#3}%
1383 \else
1384 \texorpdfstring{\textormath{#1}{#3}}{#2}%
1385 % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1386 \fi}
1387 %
1388 \def\declare@shorthand#1#2{\@decl@short{#1}#2@nil}
1389 \def\@decl@short#1#2#3@nil#4{%
1390 \def\bbl@tempa{#3}%
1391 \ifx\bbl@tempa\empty
1392 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1393 \bbl@ifunset{#1@sh@\string#2@}{}%
1394 {\def\bbl@tempa{#4}%
1395 \expandafter\ifx\csname#1@sh@\string#2@endcsname\bbl@tempa
1396 \else
1397 \bbl@info
1398 {Redefining #1 shorthand \string#2\}%
1399 in language \CurrentOption}%
1400 \fi}%
1401 \@namedef{#1@sh@\string#2@}{#4}%
1402 \else
1403 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1404 \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1405 {\def\bbl@tempa{#4}%
1406 \expandafter\ifx\csname#1@sh@\string#2@\string#3@endcsname\bbl@tempa
1407 \else
1408 \bbl@info
1409 {Redefining #1 shorthand \string#2\string#3\}%
1410 in language \CurrentOption}%
1411 \fi}%
1412 \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1413 \fi}
```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```
1414 \def\textormath{%
1415 \ifmmode
1416 \expandafter\@secondoftwo
1417 \else
1418 \expandafter\@firstoftwo
1419 \fi}
```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```
1420 \def\user@group{user}
1421 \def\language@group{english}
1422 \def\system@group{system}
```

\useshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```
1423 \def\useshorthands{%
1424   \ifstar\bbbl@usesh@s{\bbbl@usesh@x{}}
1425 \def\bbbl@usesh@s#1{%
1426   \bbbl@usesh@x
1427   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bbbl@activate{#1}}}%
1428   {#1}}
1429 \def\bbbl@usesh@x#1#2{%
1430   \bbbl@ifshorthand{#2}%
1431   {\def\user@group{user}%
1432     \initiate@active@char{#2}%
1433     #1%
1434     \bbbl@activate{#2}}%
1435   {\bbbl@error{shorthand-is-off}{#2}}}
```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(\language) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bbbl@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```
1436 \def\user@language@group{user@\language@group}
1437 \def\bbbl@set@user@generic#1#2{%
1438   \bbbl@ifunset{user@generic@active#1}%
1439   {\bbbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1440     \bbbl@active@def#1\user@group{user@generic@active}{language@active}%
1441     \expandafter\edef\csname#2@sh@#1@\endcsname{%
1442       \expandafter\noexpand\csname normal@char#1\endcsname}%
1443     \expandafter\edef\csname#2@sh@#1@\string\protect\endcsname{%
1444       \expandafter\noexpand\csname user@active#1\endcsname}}%
1445   \@empty}
1446 \newcommand\defineshorthand[3][user]{%
1447   \edef\bbbl@tempa{\zap@space#1 \@empty}%
1448   \bbbl@for\bbbl@tempb\bbbl@tempa{%
1449     \if*\expandafter\@car\bbbl@tempb\@nil
1450       \edef\bbbl@tempb{user@\expandafter\@gobble\bbbl@tempb}%
1451       \@expandtwoargs
1452       \bbbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbbl@tempb
1453     \fi
1454     \declare@shorthand{\bbbl@tempb}{#2}{#3}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```
1455 \def\languageshorthands#1{%
1456   \bbbl@ifsamestring{none}{#1}{}%
1457   \bbbl@once{short-\localename-#1}{%
1458     \bbbl@info{'\localename' activates '#1' shorthands.\@Reported}}%
1459   \def\language@group{#1}}
```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshandands{"}{/}` is `\active@prefix / \active@char/`, so we still need to let the latter to `\active@char`".

```

1460 \def\aliasshorthand#1#2{%
1461   \bbl@ifshorthand{#2}%
1462   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1463     \ifx\document\@notprerr
1464       \@notshorthand{#2}%
1465     \else
1466       \initiate@active@char{#2}%
1467       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1468       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1469       \bbl@activate{#2}%
1470     \fi
1471   \fi}%
1472   {\bbl@error{shorthand-is-off}{#2}{}}}
```

\@notshorthand

```

1473 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}
```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```

1474 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1475 \DeclareRobustCommand*\shorthandoff{%
1476   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1477 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}
```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char`" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```

1478 \def\bbl@switch@sh#1#2{%
1479   \ifx#2\@nnil\else
1480     \bbl@ifunset{bbl@active@\string#2}%
1481     {\bbl@error{not-a-shorthand-b}{#2}{}}%
1482     {\ifcase#1 off, on, off*
1483       \catcode`#2\relax
1484     \or
1485       \catcode`#2\active
1486       \bbl@ifunset{bbl@shdef@\string#2}%
1487       {}%
1488       {\bbl@withactive{\expandafter\let\expandafter}#2%
1489         \csname bbl@shdef@\string#2\endcsname
1490         \bbl@csarg\let{shdef@\string#2}\relax}%
1491       \ifcase\bbl@activated\or
1492         \bbl@activate{#2}%
1493       \else
1494         \bbl@deactivate{#2}%
1495       \fi
1496     \or
1497       \bbl@ifunset{bbl@shdef@\string#2}%
1498       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1499       {}%
1500       \csname bbl@oricat@\string#2\endcsname
1501       \csname bbl@oridef@\string#2\endcsname
1502     \fi}%

```

```

1503 \bbl@afterfi\bbl@switch@sh#1%
1504 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1505 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1506 \def\bbl@putsh#1{%
1507 \bbl@ifunset{\bbl@active@\string#1}%
1508 {\bbl@putsh@i#1\@empty\@nnil}%
1509 {\csname bbl@active@\string#1\endcsname}}
1510 \def\bbl@putsh@i#1#2\@nnil{%
1511 \csname\language@group @sh@\string#1@%
1512 \ifx\@empty#2\else\string#2@\fi\endcsname}
1513 %
1514 \ifx\bbl@opt@shorthands\@nnil\else
1515 \let\bbl@s@initiate@active@char@initiate@active@char
1516 \def\initiate@active@char#1{%
1517 \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1518 \let\bbl@s@switch@sh\bbl@switch@sh
1519 \def\bbl@switch@sh#1#2{%
1520 \ifx#2\@nnil\else
1521 \bbl@afterfi
1522 \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1523 \fi}
1524 \let\bbl@s@activate\bbl@activate
1525 \def\bbl@activate#1{%
1526 \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1527 \let\bbl@s@deactivate\bbl@deactivate
1528 \def\bbl@deactivate#1{%
1529 \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1530 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1531 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting \prime for each right quote in mathmode is \prim@s. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1532 \def\bbl@prim@s{%
1533 \prime\futurelet\@let@token\bbl@pr@m@s}
1534 \def\bbl@if@primes#1#2{%
1535 \ifx#1\@let@token
1536 \expandafter\@firstoftwo
1537 \else\ifx#2\@let@token
1538 \bbl@afterelse\expandafter\@firstoftwo
1539 \else
1540 \bbl@afterfi\expandafter\@secondoftwo
1541 \fi\fi}
1542 \begingroup
1543 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^
1544 \catcode`\'=12 \catcode`\\"=\active \lccode`\\"='\`
1545 \lowercase{%
1546 \gdef\bbl@pr@m@s{%
1547 \bbl@if@primes" '%
1548 \pr@@@s
1549 {\bbl@if@primes*\^{\pr@@@t\egroup}}}
1550 \endgroup

```

Usually the ~ is active and expands to \penalty\@M_. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character ~ as a start character for a shorthand, it

is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```
1551 \initiate@active@char{~}
1552 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1553 \bbl@activate{~}
```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```
1554 \expandafter\def\csname OT1dqpos\endcsname{127}
1555 \expandafter\def\csname T1dqpos\endcsname{4}
```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```
1556 \ifx\f@encoding\undefined
1557   \def\f@encoding{OT1}
1558 \fi
```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro \languageattribute checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in \bbl@known@attrs. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated T_EX code.

```
1559 \bbl@trace{Language attributes}
1560 \newcommand\languageattribute[2]{%
1561   \def\bbl@tempc{#1}%
1562   \bbl@fixname\bbl@tempc
1563   \bbl@iflanguage\bbl@tempc{%
1564     \bbl@vforeach{#2}{%
1565       \ifx\bbl@known@attrs\undefined
1566         \in@false
1567       \else
1568         \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1569       \fi
1570       \ifin@
1571         \bbl@warning{%
1572           You have more than once selected the attribute '##1'\%
1573           for language #1. Reported}%
1574       \else
1575         \bbl@info{Activated '##1' attribute for\\%
1576           '\bbl@tempc'. Reported}%
1577         \bbl@exp{%
1578           \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1579         \edef\bbl@tempa{\bbl@tempc-##1}%
1580         \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1581         {\csname\bbl@tempc @attr##1\endcsname}%
1582         {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1583       \fi}}}
1584 \@onlypreamble\languageattribute
```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```
1585 \def\bbl@declare@ttribute#1#2#3{%
1586   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1587   \ifin@
1588     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1589   \fi
1590   \bbl@add@list\bbl@attributes{#1-#2}%
1591   \expandafter\def\csname#1@attr@#2\endcsname{#3}}
```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```
1592 \def\bbl@ifattributeset#1#2#3#4{%
1593   \ifx\bbl@known@attribs\@undefined
1594     \in@false
1595   \else
1596     \bbl@xin@{,#1-#2,}{,\bbl@known@attribs,}%
1597   \fi
1598   \ifin@
1599     \bbl@afterelse#3%
1600   \else
1601     \bbl@afterfi#4%
1602   \fi}
```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```
1603 \def\bbl@ifknown@ttrib#1#2{%
1604   \let\bbl@tempa\@secondoftwo
1605   \bbl@loopx\bbl@tempb{#2}{%
1606     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{,#1,}%
1607   \ifin@
1608     \let\bbl@tempa\@firstoftwo
1609   \else
1610     \fi}%
1611   \bbl@tempa}
```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```
1612 \def\bbl@clear@ttribs{%
1613   \ifx\bbl@attributes\@undefined\else
1614     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1615       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1616     \let\bbl@attributes\@undefined
1617   \fi}
1618 \def\bbl@clear@ttrib#1-#2.{%
1619   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1620 \AtBeginDocument{\bbl@clear@ttribs}
```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence

to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are `\relax`’ed.

`\babel@savecnt`

`\babel@beginsave` The initialization of a new save cycle: reset the counter to zero.

```
1621 \bbl@trace{Macros for saving definitions}
1622 \def\babel@beginsave{\babel@savecnt\z@}
```

Before it’s forgotten, allocate the counter and initialize all.

```
1623 \newcount\babel@savecnt
1624 \babel@beginsave
```

`\babel@save`

`\babel@savevariable` The macro `\babel@save{csname}` saves the current meaning of the control sequence `(csname)` to `\originalTeX` (which has to be expandable, i.e., you shouldn’t let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable{variable}` saves the value of the variable. `(variable)` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```
1625 \def\babel@save#1{%
1626   \def\bbl@tempa{,{#1,}}% Clumsy, for Plain
1627   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1628     \expandafter{\expandafter,\bbl@savextras,}}%
1629   \expandafter\in@\bbl@tempa
1630   \ifin@else
1631     \bbl@add\bbl@savextras{,{#1,}}%
1632     \bbl@carg\let\babel@number\babel@savecnt\#1\relax
1633     \toks@{\expandafter{\originalTeX\let#1=}}%
1634     \bbl@exp{%
1635       \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1636     \advance\babel@savecnt@ne
1637   \fi}
1638 \def\babel@savevariable#1{%
1639   \toks@{\expandafter{\originalTeX #1=}}%
1640   \bbl@exp{\def\\originalTeX{\the\toks@the#1\relax}}}
```

`\bbl@redefine` To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```
1641 \def\bbl@redefine#1{%
1642   \edef\bbl@tempa{\bbl@stripslash#1}%
1643   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1644   \expandafter\def\csname\bbl@tempa\endcsname}
1645 \@onlypreamble\bbl@redefine
```

`\bbl@redefine@long` This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```
1646 \def\bbl@redefine@long#1{%
1647   \edef\bbl@tempa{\bbl@stripslash#1}%
1648   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1649   \long\expandafter\def\csname\bbl@tempa\endcsname}
1650 \@onlypreamble\bbl@redefine@long
```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo`. So it is necessary to check whether `\foo` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo`.

```

1651 \def\bbl@redefineroobust#1{%
1652   \edef\bbl@tempa{\bbl@stripslash#1}%
1653   \bbl@iifunset{\bbl@tempa\space}%
1654   {\expandafter\let\csname org@\bbl@tempa\endcsname#1%
1655    \bbl@exp{\def\#1{\protect\<\bbl@tempa\space>}}}%
1656   {\bbl@exp{\let\<org@\bbl@tempa>\<\bbl@tempa\space>}}}%
1657   \@namedef{\bbl@tempa\space}}
1658 \onlypreamble\bbl@redefineroobust

```

4.11. French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don't want that. The command `\bbl@frenchspacing` switches it on when it isn't already in effect and `\bbl@nonfrenchspacing` switches it off if necessary.

```

1659 \def\bbl@frenchspacing{%
1660   \ifnum\the\sfcode`\.\=\@m
1661     \let\bbl@nonfrenchspacing\relax
1662   \else
1663     \frenchspacing
1664     \let\bbl@nonfrenchspacing\nonfrenchspacing
1665   \fi}
1666 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with `ini` files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1667 \let\bbl@elt\relax
1668 \edef\bbl@fs@chars{%
1669   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1670   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1671   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1672 \def\bbl@pre@fs{%
1673   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1674   \edef\bbl@save@sfcodes{\bbl@fs@chars}%
1675   \def\bbl@post@fs{%
1676     \bbl@save@sfcodes
1677     \edef\bbl@tempa{\bbl@cl{frspc}}%
1678     \edef\bbl@tempa{\expandafter\@car\bbl@tempa\@nil}%
1679     \if u\bbl@tempa      % do nothing
1680     \else\if n\bbl@tempa % non french
1681       \def\bbl@elt##1##2##3{%
1682         \ifnum\sfcode`##1=##2\relax
1683         \babel@savevariable{\sfcode`##1}%
1684         \sfcode`##1=##3\relax
1685       \fi}%
1686       \bbl@fs@chars
1687     \else\if y\bbl@tempa % french
1688       \def\bbl@elt##1##2##3{%
1689         \ifnum\sfcode`##1=##3\relax
1690         \babel@savevariable{\sfcode`##1}%
1691         \sfcode`##1=##2\relax
1692       \fi}%
1693       \bbl@fs@chars
1694     \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@{language}` for language ones. See

\bbl@patterns above for further details. We make sure there is a space between words when multiple commands are used.

```

1695 \bbl@trace{Hyphens}
1696 \@onlypreamble\babelhyphenation
1697 \AtEndOfPackage{%
1698   \newcommand\babelhyphenation[2][\@empty]{%
1699     \ifx\bbl@hyphenation@\relax
1700       \let\bbl@hyphenation@\@empty
1701     \fi
1702     \ifx\bbl@hyphlist@\@empty\else
1703       \bbl@warning{%
1704         You must not intermingle \string\selectlanguage\space and\\%
1705         \string\babelhyphenation\space or some exceptions will not\\%
1706         be taken into account. Reported}%
1707       \fi
1708       \ifx\@empty#1%
1709         \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1710       \else
1711         \bbl@vforeach{#1}{%
1712           \def\bbl@tempa{##1}%
1713           \bbl@fixname\bbl@tempa
1714           \bbl@iflanguage\bbl@tempa{%
1715             \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1716               \bbl@ifunset{bbl@hyphenation@\bbl@tempa}%
1717               {}%
1718               {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1719               #2}}}%
1720         \fi}}

```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1721 \ifx\NewDocumentCommand\@undefined\else
1722   \NewDocumentCommand\babelhyphenmins{sommo}{%
1723     \IfNoValueTF{#2}%
1724       {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1725       \IfValueT{#5}{%
1726         \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1727       \IfBooleanT{#1}{%
1728         \lefthyphenmin=#3\relax
1729         \righthyphenmin=#4\relax
1730         \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1731     {\edef\bbl@tempb{\zap@space#2 \@empty}%
1732      \bbl@for\bbl@tempa\bbl@tempb{%
1733        \@namedef{bbl@hyphenmins@\bbl@tempa}{\set@hyphenmins{#3}{#4}}}%
1734      \IfValueT{#5}{%
1735        \@namedef{bbl@hyphenatmin@\bbl@tempa}{\hyphenationmin=#5\relax}}}%
1736      \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1737 \fi

```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than \nobreak \hskip 0pt plus 0pt. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1738 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1739 \def\bbl@t@one{Tl}
1740 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in \babelhyphen. Instead of protecting it with \DeclareRobustCommand, which could insert a \relax, we use the same procedure as shorthands, with \active@prefix.

```

1741 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1742 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1743 \def\bbl@hyphen{%

```

```

1744 \ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i\@empty}}
1745 \def\bbl@hyphen@i#1#2{%
1746 \lowercase{\bbl@ifunset{\bbl@hy#1#2\@empty}}}%
1747 {\csname bbl@lusehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1748 {\lowercase{\csname bbl@hy#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. \nobreak is always preceded by \leavevmode, in case the shorthand starts a paragraph.

```

1749 \def\bbl@usehyphen#1{%
1750 \leavevmode
1751 \ifdim\lastskip>z@\mbox{#1}\else\nobreak#1\fi
1752 \nobreak\hskip\z@skip}
1753 \def\bbl@usehyphen#1{%
1754 \leavevmode\ifdim\lastskip>z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1755 \def\bbl@hyphenchar{%
1756 \ifnum\hyphenchar\font=\m@ne
1757 \babe\nullhyphen
1758 \else
1759 \char\hyphenchar\font
1760 \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in ldf’s. After a space, the \mbox in \bbl@hy@nobreak is redundant.

```

1761 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1762 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1763 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1764 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1765 \def\bbl@hy@nobreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1766 \def\bbl@hy@nobreak{\mbox{\bbl@hyphenchar}}
1767 \def\bbl@hy@repeat{%
1768 \bbl@usehyphen{%
1769 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1770 \def\bbl@hy@repeat{%
1771 \bbl@usehyphen{%
1772 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1773 \def\bbl@hy@empty{\hskip\z@skip}
1774 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro \bbl@disc is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1775 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by luatex and xetex. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1776 \bbl@trace{Multiencoding strings}
1777 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated \SetCase.

```

1778 <(*More package options)> ≡
1779 \DeclareOption{nocase}{}
1780 <(/More package options)>

```

The following package options control the behavior of `\SetString`.

```

1781 ({*More package options}) ≡
1782 \let\bbl@opt@strings\@nnil % accept strings=value
1783 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1784 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1785 \def\BabelStringsDefault{generic}
1786 (/More package options)

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1787 \@onlypreamble\StartBabelCommands
1788 \def\StartBabelCommands{%
1789   \begingroup
1790   \@tempcnta="7F
1791   \def\bbl@tempa{%
1792     \ifnum\@tempcnta>"FF\else
1793       \catcode\@tempcnta=11
1794       \advance\@tempcnta\@ne
1795       \expandafter\bbl@tempa
1796     \fi}%
1797   \bbl@tempa
1798   <@Macros local to BabelCommands@>
1799   \def\bbl@provstring##1##2{%
1800     \providecommand##1{##2}%
1801     \bbl@tglobal##1}%
1802   \global\let\bbl@scafter\@empty
1803   \let\StartBabelCommands\bbl@startcmds
1804   \ifx\BabelLanguages\relax
1805     \let\BabelLanguages\CurrentOption
1806   \fi
1807   \begingroup
1808   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1809   \StartBabelCommands}
1810 \def\bbl@startcmds{%
1811   \ifx\bbl@screset\@nnil\else
1812     \bbl@usehooks{stopcommands}{}%
1813   \fi
1814   \endgroup
1815   \begingroup
1816   \@ifstar
1817   {\ifx\bbl@opt@strings\@nnil
1818     \let\bbl@opt@strings\BabelStringsDefault
1819     \fi
1820     \bbl@startcmds@i}%
1821   \bbl@startcmds@i}
1822 \def\bbl@startcmds@i#1#2{%
1823   \edef\bbl@L{\zap@space#1 \@empty}%
1824   \edef\bbl@G{\zap@space#2 \@empty}%
1825   \bbl@startcmds@ii}
1826 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of `\SetString`. There are two main cases, depending of if there is an optional argument: without it and `strings=encoded`, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and `strings=encoded`, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in `strings=`) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1827 \newcommand\bbl@startcmds@ii[1][\@empty]{%

```

```

1828 \let\SetString\@gobbletwo
1829 \let\bbl@stringdef\@gobbletwo
1830 \let\AfterBabelCommands\@gobble
1831 \ifx\@empty#1%
1832   \def\bbl@sc@label{generic}%
1833   \def\bbl@encstring##1##2{%
1834     \ProvideTextCommandDefault##1{##2}%
1835     \bbl@tglobal##1%
1836     \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1837   \let\bbl@sctest\in@true
1838 \else
1839   \let\bbl@sc@charset\space % <- zapped below
1840   \let\bbl@sc@fontenc\space % <- " "
1841   \def\bbl@tempa##1=##2\@nil{%
1842     \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1843   \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1844   \def\bbl@tempa##1 ##2{% space -> comma
1845     ##1%
1846     \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1847   \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1848   \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1849   \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1850   \def\bbl@encstring##1##2{%
1851     \bbl@foreach\bbl@sc@fontenc{%
1852       \bbl@ifunset{T@###1}%
1853       {}%
1854       {\ProvideTextCommand##1{###1}{##2}%
1855         \bbl@tglobal##1%
1856         \expandafter
1857         \bbl@tglobal\csname###1\string##1\endcsname}}}%
1858   \def\bbl@sctest{%
1859     \bbl@xin@{\bbl@opt@strings,}{,\bbl@sc@label,\bbl@sc@fontenc,}%
1860 \fi
1861 \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1862 \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1863   \let\AfterBabelCommands\bbl@aftercmds
1864   \let\SetString\bbl@setstring
1865   \let\bbl@stringdef\bbl@encstring
1866 \else % i.e., strings=value
1867   \bbl@sctest
1868 \fin@
1869   \let\AfterBabelCommands\bbl@aftercmds
1870   \let\SetString\bbl@setstring
1871   \let\bbl@stringdef\bbl@provstring
1872 \fi\fi\fi
1873 \bbl@scswitch
1874 \ifx\bbl@G\@empty
1875   \def\SetString##1##2{%
1876     \bbl@error{missing-group}{##1}{}}%
1877 \fi
1878 \ifx\@empty#1%
1879   \bbl@usehooks{defaultcommands}{}%
1880 \else
1881   \@expandtwoargs
1882   \bbl@usehooks{encodedcommands}{\bbl@sc@charset}\bbl@sc@fontenc}%
1883 \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\langle group \rangle \langle language \rangle` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date\langle language \rangle` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has

been loaded).

```

1884 \def\bbl@forlang#1#2{%
1885   \bbl@for#1\bbl@L{%
1886     \bbl@xin@{,#1,},{,\BabelLanguages,}%
1887     \ifin@#2\relax\fi}}
1888 \def\bbl@scswitch{%
1889   \bbl@forlang\bbl@tempa{%
1890     \ifx\bbl@G\@empty\else
1891       \ifx\SetString@gobbletwo\else
1892         \edef\bbl@GL{\bbl@G\bbl@tempa}%
1893         \bbl@xin@{,\bbl@GL,}{,\bbl@screset,}%
1894         \ifin@\else
1895           \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1896           \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1897         \fi
1898       \fi
1899     \fi}}
1900 \AtEndOfPackage{%
1901   \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1902   \let\bbl@scswitch\relax}
1903 \@onlypreamble\EndBabelCommands
1904 \def\EndBabelCommands{%
1905   \bbl@usehooks{stopcommands}{}%
1906   \endgroup
1907   \endgroup
1908   \bbl@scafter}
1909 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1910 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1911   \bbl@forlang\bbl@tempa{%
1912     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1913     \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1914     {\bbl@exp{%
1915       \global\\bbl@add\<\bbl@G\bbl@tempa>{\\bbl@scset\\#1\<\bbl@LC>}}}%
1916     }%
1917     \def\BabelString{#2}%
1918     \bbl@usehooks{stringprocess}{}%
1919     \expandafter\bbl@stringdef
1920     \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}%

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1921 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1922 <(*Macros local to BabelCommands)> ≡
1923 \def\SetStringLoop###1#2{%
1924   \def\bbl@templ#####1{\expandafter\noexpand\csname##1\endcsname}%
1925   \count@\z@
1926   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1927     \advance\count@\@ne
1928     \toks@\expandafter{\bbl@tempa}%
1929     \bbl@exp{%
1930       \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%

```

```

1931 \count@=\the\count@\relax}}}%
1932 <</Macros local to BabelCommands>>

```

Delaying code Now the definition of \AfterBabelCommands when it is activated.

```

1933 \def\bbl@aftercmds#1{%
1934 \toks@\expandafter{\bbl@scafter#1}%
1935 \xdef\bbl@scafter{\the\toks@}}

```

Case mapping The command \SetCase is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1936 <<*Macros local to BabelCommands>> ≡
1937 \newcommand\SetCase[3][]{%
1938 \def\bbl@tempa####1####2{%
1939 \ifx####1\@empty\else
1940 \bbl@carg\bbl@add{extras\CurrentOption}{%
1941 \bbl@carg\babel@save{c__text_uppercase_\string####1_tl}%
1942 \bbl@carg\def{c__text_uppercase_\string####1_tl}{####2}%
1943 \bbl@carg\babel@save{c__text_lowercase_\string####2_tl}%
1944 \bbl@carg\def{c__text_lowercase_\string####2_tl}{####1}}%
1945 \expandafter\bbl@tempa
1946 \fi}%
1947 \bbl@tempa##1\@empty\@empty
1948 \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1949 <</Macros local to BabelCommands>>

```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1950 <<*Macros local to BabelCommands>> ≡
1951 \newcommand\SetHyphenMap[1]{%
1952 \bbl@forlang\bbl@tempa{%
1953 \expandafter\bbl@stringdef
1954 \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1955 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1956 \newcommand\BabelLower[2]{% one to one.
1957 \ifnum\lccode#1=#2\else
1958 \babel@savevariable{\lccode#1}%
1959 \lccode#1=#2\relax
1960 \fi}
1961 \newcommand\BabelLowerMM[4]{% many-to-many
1962 \@tempcnta=#1\relax
1963 \@tempcntb=#4\relax
1964 \def\bbl@tempa{%
1965 \ifnum\@tempcnta>#2\else
1966 \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1967 \advance\@tempcnta#3\relax
1968 \advance\@tempcntb#3\relax
1969 \expandafter\bbl@tempa
1970 \fi}%
1971 \bbl@tempa}
1972 \newcommand\BabelLowerM0[4]{% many-to-one
1973 \@tempcnta=#1\relax
1974 \def\bbl@tempa{%
1975 \ifnum\@tempcnta>#2\else
1976 \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1977 \advance\@tempcnta#3
1978 \expandafter\bbl@tempa
1979 \fi}%
1980 \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1981 ({*More package options})  ≡
1982 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1983 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap\@ne}
1984 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1985 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@@}
1986 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1987 (/More package options)

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1988 \AtEndOfPackage{%
1989   \ifx\bbl@opt@hyphenmap\undefined
1990     \bbl@xin@{,}{\bbl@language@opts}%
1991     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1992   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1993 \newcommand\setlocalecaption{%
1994   \ifstar\bbl@setcaption@s\bbl@setcaption@x}
1995 \def\bbl@setcaption@x#1#2#3{% language caption-name string
1996   \bbl@trim@def\bbl@tempa{#2}%
1997   \bbl@xin@{.template}{\bbl@tempa}%
1998   \ifin@
1999     \bbl@ini@captions@template{#3}{#1}%
2000   \else
2001     \edef\bbl@tempd{%
2002       \expandafter\expandafter\expandafter
2003       \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
2004     \bbl@xin@
2005       {\expandafter\string\csname #2name\endcsname}%
2006       {\bbl@tempd}%
2007     \ifin@ % Renew caption
2008       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
2009       \ifin@
2010         \bbl@exp{%
2011           \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2012           {\bbl@scset\<#2name>\<#1#2name>}%
2013           {}}%
2014         \else % Old way converts to new way
2015           \bbl@ifunset{#1#2name}%
2016             {\bbl@exp{%
2017               \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2018               \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2019               {\def\<#2name>\<#1#2name>}}%
2020               {}}}%
2021         {}%
2022       \fi
2023     \else
2024       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2025       \ifin@ % New way
2026         \bbl@exp{%
2027           \\bbl@add\<captions#1>\bbl@scset\<#2name>\<#1#2name>}%
2028           \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2029           {\bbl@scset\<#2name>\<#1#2name>}%
2030           {}}%
2031         \else % Old way, but defined in the new way
2032           \bbl@exp{%
2033             \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2034             \\bbl@ifsamestring{\bbl@tempa}{\language name}%

```

```

2035      {\def\<#2name>{\<#1#2name>}}%
2036      {}}%
2037      \fi%
2038      \fi
2039      \@namedef{#1#2name}{#3}%
2040      \toks@{\expandafter{\bbl@captionslist}%
2041      \bbl@exp{\in{\<#2name>}{\the\toks@}}}%
2042      \ifin@else
2043      \bbl@exp{\bbl@add{\bbl@captionslist{\<#2name>}}}%
2044      \bbl@tglobal\bbl@captionslist
2045      \fi
2046      \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2047 \bbl@trace{Macros related to glyphs}
2048 \def\set@low@box#1{\setbox\tw@{\hbox{,}}\setbox\z@{\hbox{#1}}%
2049   \dimen\z@{\ht\z@ \advance\dimen\z@ -\ht\tw@}%
2050   \setbox\z@{\hbox{\lower\dimen\z@ \box\z@}\ht\z@{\ht\tw@ \dp\z@\dp\tw@}}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2051 \def\save@sf@q#1{\leavevmode
2052   \begingroup
2053   \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2054   \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2055 \ProvideTextCommand{\quotedblbase}{OT1}{%
2056   \save@sf@q{\set@low@box{\textquotedblright}/}%
2057   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2058 \ProvideTextCommandDefault{\quotedblbase}{%
2059   \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2060 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2061   \save@sf@q{\set@low@box{\textquoteright}/}%
2062   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2063 \ProvideTextCommandDefault{\quotesinglbase}{%
2064   \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```

2065 \ProvideTextCommand{\guillemetleft}{OT1}{%
2066   \ifmmode
2067     \ll
2068   \else
2069     \save@sf@q{\nobreak
2070       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2071   \fi}
2072 \ProvideTextCommand{\guillemetright}{OT1}{%
2073   \ifmmode
2074     \gg
2075   \else
2076     \save@sf@q{\nobreak
2077       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2078   \fi}
2079 \ProvideTextCommand{\guillemotleft}{OT1}{%
2080   \ifmmode
2081     \ll
2082   \else
2083     \save@sf@q{\nobreak
2084       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2085   \fi}
2086 \ProvideTextCommand{\guillemotright}{OT1}{%
2087   \ifmmode
2088     \gg
2089   \else
2090     \save@sf@q{\nobreak
2091       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2092   \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2093 \ProvideTextCommandDefault{\guillemetleft}{%
2094   \UseTextSymbol{OT1}{\guillemetleft}}
2095 \ProvideTextCommandDefault{\guillemetright}{%
2096   \UseTextSymbol{OT1}{\guillemetright}}
2097 \ProvideTextCommandDefault{\guillemotleft}{%
2098   \UseTextSymbol{OT1}{\guillemotleft}}
2099 \ProvideTextCommandDefault{\guillemotright}{%
2100   \UseTextSymbol{OT1}{\guillemotright}}

```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```

2101 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2102   \ifmmode
2103     <%
2104   \else
2105     \save@sf@q{\nobreak
2106       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2107   \fi}
2108 \ProvideTextCommand{\guilsinglright}{OT1}{%
2109   \ifmmode
2110     >%
2111   \else
2112     \save@sf@q{\nobreak
2113       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2114   \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2115 \ProvideTextCommandDefault{\guilsinglleft}{%
2116   \UseTextSymbol{OT1}{\guilsinglleft}}
2117 \ProvideTextCommandDefault{\guilsinglright}{%
2118   \UseTextSymbol{OT1}{\guilsinglright}}

```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter 'ij'. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2119 \DeclareTextCommand{\ij}{OT1}{%
2120   i\kern-0.02em\bbI\allowhyphens j}
2121 \DeclareTextCommand{\IJ}{OT1}{%
2122   I\kern-0.02em\bbI\allowhyphens J}
2123 \DeclareTextCommand{\ij}{T1}{\char188}
2124 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2125 \ProvideTextCommandDefault{\ij}{%
2126   \UseTextSymbol{OT1}{\ij}}
2127 \ProvideTextCommandDefault{\IJ}{%
2128   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2129 \def\crrtic@{\hrule height0.1ex width0.3em}
2130 \def\crttic@{\hrule height0.1ex width0.33em}
2131 \def\ddj@{%
2132   \setbox0\hbox{d}\dimen@=\ht0
2133   \advance\dimen@lex
2134   \dimen@.45\dimen@
2135   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2136   \advance\dimen@ii.5ex
2137   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2138 \def\DDJ@{%
2139   \setbox0\hbox{D}\dimen@=.55\ht0
2140   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2141   \advance\dimen@ii.15ex % correction for the dash position
2142   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2143   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2144   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crttic@}}}}
2145 %
2146 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2147 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2148 \ProvideTextCommandDefault{\dj}{%
2149   \UseTextSymbol{OT1}{\dj}}
2150 \ProvideTextCommandDefault{\DJ}{%
2151   \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2152 \DeclareTextCommand{\SS}{OT1}{SS}
2153 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```
2154 \ProvideTextCommandDefault{\glq}{%
2155 \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2156 \ProvideTextCommand{\grq}{T1}{%
2157 \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}
2158 \ProvideTextCommand{\grq}{TU}{%
2159 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}
2160 \ProvideTextCommand{\grq}{OT1}{%
2161 \save@sf@q{\kern-.0125em
2162 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%
2163 \kern.07em\relax}}
2164 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}
```

\glqq

\grqq The ‘german’ double quotes.

```
2165 \ProvideTextCommandDefault{\glqq}{%
2166 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}
```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2167 \ProvideTextCommand{\grqq}{T1}{%
2168 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2169 \ProvideTextCommand{\grqq}{TU}{%
2170 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2171 \ProvideTextCommand{\grqq}{OT1}{%
2172 \save@sf@q{\kern-.07em
2173 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%
2174 \kern.07em\relax}}
2175 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}
```

\flq

\frq The ‘french’ single guillemets.

```
2176 \ProvideTextCommandDefault{\flq}{%
2177 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}
2178 \ProvideTextCommandDefault{\frq}{%
2179 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

\flqq

\frqq The ‘french’ double guillemets.

```
2180 \ProvideTextCommandDefault{\flqq}{%
2181 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}
2182 \ProvideTextCommandDefault{\frqq}{%
2183 \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command \ " needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```

2184 \def\umlauthigh{%
2185   \def\bbbl@umlauta##1{\leavevmode\bgroup%
2186     \accent\csname\fontencoding dqpos\endcsname
2187     ##1\bbbl@allowhyphens\egroup}%
2188   \let\bbbl@umlaute\bbbl@umlauta}
2189 \def\umlautlow{%
2190   \def\bbbl@umlauta{\protect\lower@umlaut}}
2191 \def\umlautelow{%
2192   \def\bbbl@umlaute{\protect\lower@umlaut}}
2193 \umlauthigh

```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra (*dimen*) register.

```

2194 \expandafter\ifx\csname U@D\endcsname\relax
2195   \csname newdimen\endcsname\U@D
2196 \fi

```

The following code fools $\TeX$'s `make\_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```

2197 \def\lower@umlaut#1{%
2198   \leavevmode\bgroup
2199   \U@D lex%
2200   {\setbox\z@\hbox{%
2201     \char\csname\fontencoding dqpos\endcsname}%
2202     \dimen@ -.45ex\advance\dimen@\ht\z@
2203     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2204   \accent\csname\fontencoding dqpos\endcsname
2205   \fontdimen5\font\U@D #1%
2206   \egroup}

```

For all vowels we declare `\` to be a composite command which uses `\bbbl@umlauta` or `\bbbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbbl@umlauta` and/or `\bbbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```

2207 \AtBeginDocument{%
2208   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbbl@umlauta{a}}%
2209   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbbl@umlaute{e}}%
2210   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbbl@umlaute{i}}%
2211   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbbl@umlaute{i}}%
2212   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbbl@umlauta{o}}%
2213   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbbl@umlauta{u}}%
2214   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbbl@umlauta{A}}%
2215   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbbl@umlaute{E}}%
2216   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbbl@umlaute{I}}%
2217   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbbl@umlauta{O}}%
2218   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```

2219 \ifx\l@english\undefined
2220   \chardef\l@english\z@
2221 \fi

```

```

2222 % The following is used to cancel rules in ini files (see Amharic).
2223 \ifx\l@unhyphenated\@undefined
2224   \newlanguage\l@unhyphenated
2225 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2226 \bbl@trace{Bidi layout}
2227 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2228 \bbl@trace{Input engine specific macros}
2229 \ifcase\bbl@engine
2230   \input txtbabel.def
2231 \or
2232   \input luababel.def
2233 \or
2234   \input xebabel.def
2235 \fi
2236 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}{}
2237 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}{}
2238 \ifx\babelposthyphenation\@undefined
2239   \let\babelposthyphenation\babelprehyphenation
2240   \let\babelpatterns\babelprehyphenation
2241   \let\babelcharproperty\babelprehyphenation
2242 \fi
2243 (/package|core)

```

4.18. Creating and modifying languages

Continue with $\LaTeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded `ldf` files.

```

2244 (*package)
2245 \bbl@trace{Creating languages and reading ini files}
2246 \let\bbl@extend@ini\@gobble
2247 \newcommand\babelprovide[2][]{%
2248   \let\bbl@savelangname\languagename
2249   \edef\bbl@savelocaleid{\the\localeid}%
2250   \global\let\bbl@afterload\@empty
2251   % Set name and locale id
2252   \edef\languagename{#2}%
2253   \bbl@id@assign
2254   % Initialize keys
2255   \bbl@vforeach{captions,date,import,main,script,language,%
2256     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2257     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2258     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2259     @import}%
2260     {\bbl@csarg\let{KVP@##1}\@nnil}%
2261   \global\let\bbl@release@transforms\@empty
2262   \global\let\bbl@release@casing\@empty
2263   \let\bbl@calendars\@empty
2264   \global\let\bbl@inidata\@empty
2265   \global\let\bbl@extend@ini\@gobble
2266   \global\let\bbl@included@inis\@empty
2267   \gdef\bbl@key@list{;}%

```

```

2268 \bbl@ifunset{bbl@passto@#2}%
2269 {\def\bbl@tempa{#1}}%
2270 {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2271 \expandafter\bbl@forkv\expandafter\bbl@tempa}%
2272 \in@{/}{##1}% With /, (re)sets a value in the ini
2273 \ifin@
2274 \bbl@renewinikey##1\@{##2}%
2275 \else
2276 \bbl@csarg\ifx{KVP@##1}\@nnil\else
2277 \bbl@error{unknown-provide-key}{##1}{}%
2278 \fi
2279 \bbl@csarg\def{KVP@##1}{##2}%
2280 \fi}%
2281 \chardef\bbl@howloaded= 0:none; 1:ldf without ini; 2:ini
2282 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2283 % == init ==
2284 \ifx\bbl@screset\undefined
2285 \bbl@ldfinit
2286 \fi
2287 % ==
2288 % If there is no import (last wins), use @import (internal, there
2289 % must be just one). To consider any order (because
2290 % \PassOptionsToLocale).
2291 \ifx\bbl@KVP@import\@nnil
2292 \let\bbl@KVP@import\bbl@KVP@@import
2293 \fi
2294 % == date (as option) ==
2295 % \ifx\bbl@KVP@date\@nnil\else
2296 % \fi
2297 % ==
2298 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2299 \ifcase\bbl@howloaded
2300 \let\bbl@lbkflag\@empty % new
2301 \else
2302 \ifx\bbl@KVP@hyphenrules\@nnil\else
2303 \let\bbl@lbkflag\@empty
2304 \fi
2305 \ifx\bbl@KVP@import\@nnil\else
2306 \let\bbl@lbkflag\@empty
2307 \fi
2308 \fi
2309 % == import, captions ==
2310 \ifx\bbl@KVP@import\@nnil\else
2311 \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2312 {\ifx\bbl@initoload\relax
2313 \begingroup
2314 \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2315 \bbl@input@texini{##2}%
2316 \endgroup
2317 \else
2318 \xdef\bbl@KVP@import{\bbl@initoload}%
2319 \fi}%
2320 {}%
2321 \let\bbl@KVP@date\@empty
2322 \fi
2323 \let\bbl@KVP@captions@@\bbl@KVP@captions
2324 \ifx\bbl@KVP@captions\@nnil
2325 \let\bbl@KVP@captions\bbl@KVP@import
2326 \fi
2327 % ==
2328 \ifx\bbl@KVP@transforms\@nnil\else
2329 \bbl@replace\bbl@KVP@transforms{ }{,}%
2330 \fi

```

```

2331 % Load ini
2332 % -----
2333 \ifcase\bbbl@howloaded
2334 \bbbl@provide@new{#2}%
2335 \else
2336 \bbbl@ifblank{#1}%
2337 {}% With \bbbl@load@basic below
2338 {\bbbl@provide@renew{#2}}%
2339 \fi
2340 % Post tasks
2341 % -----
2342 % == subsequent calls after the first provide for a locale ==
2343 \ifx\bbbl@inidata\@empty\else
2344 \bbbl@extend@ini{#2}%
2345 \fi
2346 % ==
2347 \ifx\bbbl@KVP@mapdot\@nnil\else
2348 \def\bbbl@tempa{\@empty}%
2349 \ifx\bbbl@KVP@mapdot\bbbl@tempa\else
2350 \bbbl@exp{\gdef\<bbbl@map@. .@.\language>{\bbbl@KVP@mapdot}}}%
2351 \fi
2352 \fi
2353 % == ensure captions ==
2354 \ifx\bbbl@KVP@captions\@nnil\else
2355 \bbbl@ifunset{bbbl@extracaps@#2}%
2356 {\bbbl@exp{\bbbl@babelensure[exclude=\\today]{#2}}}%
2357 {\bbbl@exp{\bbbl@babelensure[exclude=\\today,
2358 include=\bbbl@extracaps@#2]{#2}}}%
2359 \let\bbbl@tempc\language
2360 \bbbl@replace\bbbl@tempc{-}{@}%
2361 \bbbl@ifunset{bbbl@ensure@\bbbl@tempc}%
2362 {\bbbl@exp{%
2363 \\DeclareRobustCommand\<bbbl@ensure@\bbbl@tempc>[1]{%
2364 \\foreignlanguage{\language}%
2365 {###1}}}%
2366 }%
2367 \bbbl@exp{%
2368 \\bbbl@tglobal\<bbbl@ensure@\bbbl@tempc>%
2369 \\bbbl@tglobal\<bbbl@ensure@\bbbl@tempc\space>%
2370 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2371 \bbbl@load@basic{#2}%
2372 % == script, language ==
2373 % Override the values from ini or defines them
2374 \ifx\bbbl@KVP@script\@nnil\else
2375 \bbbl@csarg\edef{sname@#2}{\bbbl@KVP@script}%
2376 \fi
2377 \ifx\bbbl@KVP@language\@nnil\else
2378 \bbbl@csarg\edef{lname@#2}{\bbbl@KVP@language}%
2379 \fi
2380 \ifcase\bbbl@engine\or
2381 \bbbl@ifunset{bbbl@chrng@\language}%
2382 {\directlua{
2383 Babel.set_chranges_b('\bbbl@cl{sbc}', '\bbbl@cl{chrng}') }}%
2384 \fi
2385 % == Line breaking: intraspace, intrapenalty ==
2386 % For CJK, East Asian, Southeast Asian, if interspace in ini
2387 \ifx\bbbl@KVP@intraspace\@nnil\else % We can override the ini or set
2388 \bbbl@csarg\edef{intsp@#2}{\bbbl@KVP@intraspace}%
2389 \fi

```

```

2390 \bbl@provide@intraspace
2391 % == Line breaking: justification ==
2392 \ifx\bbl@KVP@justification\@nnil\else
2393   \let\bbl@KVP@linebreaking\bbl@KVP@justification
2394 \fi
2395 \ifx\bbl@KVP@linebreaking\@nnil\else
2396   \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2397   {,elongated,kashida,cjk,padding,unhyphenated,}%
2398   \ifin@
2399     \bbl@csarg\xdef
2400       {\lnbrk@{\language\name}}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2401   \fi
2402 \fi
2403 \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
2404 \ifin@\\else\bbl@xin@{/k}{/\bbl@cl{\lnbrk}}\fi
2405 \ifin@\bbl@arabicjust\fi
2406 \bbl@xin@{/p}{/\bbl@cl{\lnbrk}}%
2407 \ifin@\\AtBeginDocument{\@nameuse{\bbl@tibetanjust}}\fi
2408 % == Line breaking: hyphenate.other.(locale|script) ==
2409 \ifx\bbl@lbfkflag\@empty
2410   \bbl@ifunset{\bbl@hyotl@{\language\name}}{ }%
2411   {\bbl@csarg\bbl@replace{\hyotl@{\language\name}}{ }{,}%
2412   \bbl@startcommands*{\language\name}}{ }%
2413   \bbl@csarg\bbl@foreach{\hyotl@{\language\name}}{ }%
2414     \ifcase\bbl@engine
2415       \ifnum##1<257
2416         \SetHyphenMap{\BabelLower{##1}{##1}}%
2417       \fi
2418     \else
2419       \SetHyphenMap{\BabelLower{##1}{##1}}%
2420     \fi}%
2421   \bbl@endcommands}%
2422 \bbl@ifunset{\bbl@hyots@{\language\name}}{ }%
2423 {\bbl@csarg\bbl@replace{\hyots@{\language\name}}{ }{,}%
2424 \bbl@csarg\bbl@foreach{\hyots@{\language\name}}{ }%
2425 \ifcase\bbl@engine
2426   \ifnum##1<257
2427     \global\lccode##1=##1\relax
2428   \fi
2429   \else
2430     \global\lccode##1=##1\relax
2431   \fi}}%
2432 \fi
2433 % == Counters: maparabic ==
2434 % Native digits, if provided in ini (TeX level, xe and lua)
2435 \ifcase\bbl@engine\else
2436   \bbl@ifunset{\bbl@dgnat@{\language\name}}{ }%
2437   {\expandafter\ifx\csname\bbl@dgnat@{\language\name}\endcsname\@empty\else
2438     \expandafter\expandafter\expandafter
2439     \bbl@setdigits\csname\bbl@dgnat@{\language\name}\endcsname
2440     \ifx\bbl@KVP@maparabic\@nnil\else
2441       \ifx\bbl@latinarabic\@undefined
2442         \expandafter\let\expandafter\@arabic
2443         \csname\bbl@counter@{\language\name}\endcsname
2444       \else % i.e., if layout=counters, which redefines \@arabic
2445         \expandafter\let\expandafter\bbl@latinarabic
2446         \csname\bbl@counter@{\language\name}\endcsname
2447       \fi
2448     \fi
2449   \fi}%
2450 \fi
2451 % == Counters: mapdigits ==
2452 % > luababel.def

```

```

2453 % == Counters: alph, Alph ==
2454 \ifx\bbbl@KVP@alph\@nnil\else
2455   \bbbl@exp{%
2456     \\bbbl@add\<bbbl@preextras@\language\name>{%
2457       \\babel@save\\@alph
2458       \let\\@alph\<bbbl@cntr@\bbbl@KVP@alph @\language\name>}}%
2459 \fi
2460 \ifx\bbbl@KVP@Alph\@nnil\else
2461   \bbbl@exp{%
2462     \\bbbl@add\<bbbl@preextras@\language\name>{%
2463       \\babel@save\\@Alph
2464       \let\\@Alph\<bbbl@cntr@\bbbl@KVP@Alph @\language\name>}}%
2465 \fi
2466 % == Counters: mapdot ==
2467 \ifx\bbbl@KVP@mapdot\@nnil\else
2468   \bbbl@foreach\bbbl@list@the{%
2469     \bbbl@ifunset{the##1}{}%
2470     {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2471       \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2472       \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2473         \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2474         \fi}}%
2475   \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2476   \bbbl@foreach\bbbl@tempb{%
2477     \bbbl@ifunset{label##1}{}%
2478     {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2479       \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2480       \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2481         \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2482         \fi}}%
2483 \fi
2484 % == Casing ==
2485 \bbbl@release@casing
2486 \ifx\bbbl@KVP@casing\@nnil\else
2487   \bbbl@csarg\xdef{casing@\language\name}%
2488   {\@nameuse{bbbl@casing@\language\name}\bbbl@maybextx\bbbl@KVP@casing}%
2489 \fi
2490 % == Calendars ==
2491 \ifx\bbbl@KVP@calendar\@nnil
2492   \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2493 \fi
2494 \def\bbbl@tempe##1 ##2\@{ % Get first calendar
2495   \def\bbbl@tempa{##1}%
2496   \bbbl@exp{\bbbl@tempe\bbbl@KVP@calendar\space\@}%
2497 \def\bbbl@tempe##1.##2.##3.##4\@{ %
2498   \def\bbbl@tempf{##3}% First
2499   \let\bbbl@tempc\@empty \let\bbbl@tempb\@empty
2500   \bbbl@cal@getconvert\bbbl@tempc{##1}%
2501   \bbbl@cal@getconvert\bbbl@tempb{##2}%
2502   \expandafter\bbbl@tempe\bbbl@tempa...\@
2503   \bbbl@csarg\edef{calpr@\language\name}{%
2504     \ifx\bbbl@tempc\@empty\else
2505       calendar=\bbbl@tempc
2506     \fi
2507     \ifx\bbbl@tempb\@empty\else
2508       ,variant=\bbbl@tempb
2509     \fi
2510     \ifx\bbbl@tempf\@empty\else
2511       ,convert=\bbbl@tempf
2512     \fi}%
2513 % == engine specific extensions ==
2514 % Defined in XXXbabel.def
2515 \bbbl@provide@extra{##2}%

```

```

2516 % == require.babel in ini ==
2517 % To load or reload the babel-*.tex, if require.babel in ini
2518 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2519   \bbl@ifunset{bbl@rqtex@\language\name}{}%
2520     {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else
2521       \let\BabelBeforeIni@gobbletwo
2522       \chardef\atcatcode=\catcode\@
2523       \catcode\@=11\relax
2524       \def\CurrentOption{#2}%
2525       \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2526       \catcode\@=\atcatcode
2527       \let\atcatcode\relax
2528       \global\bbl@csarg\let{rqtex@\language\name}\relax
2529     \fi}%
2530 \bbl@foreach\bbl@calendars{%
2531   \bbl@ifunset{bbl@ca##1}{%
2532     \chardef\atcatcode=\catcode\@
2533     \catcode\@=11\relax
2534     \InputIfFileExists{babel-ca-##1.tex}{}{}%
2535     \catcode\@=\atcatcode
2536     \let\atcatcode\relax}%
2537   {}}%
2538 \fi
2539 % == frenchspacing ==
2540 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2541 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2542 \ifin@
2543   \bbl@extras@wrap{\bbl@pre@fs}%
2544   {\bbl@pre@fs}%
2545   {\bbl@post@fs}%
2546 \fi
2547 % == transforms ==
2548 % > luababel.def
2549 \def\CurrentOption{#2}%
2550 \@nameuse{bbl@icsave#2}%
2551 % == main ==
2552 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2553   \let\language\bbl@savelangname
2554   \chardef\localeid\bbl@savelocaleid\relax
2555 \fi
2556 % == ==
2557 \ifx\ldf@finish\@onlypreamble\else
2558   \bbl@afterload
2559 \fi
2560 % == hyphenrules (apply if current) ==
2561 \ifx\bbl@KVP@hyphenrules\@nnil\else
2562   \ifnum\bbl@savelocaleid=\localeid
2563     \language\@nameuse{l@\language\name}%
2564   \fi
2565 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2566 \def\bbl@provide@new#1{%
2567   \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2568   \@namedef{extras#1}{}%
2569   \@namedef{noextras#1}{}%
2570   \bbl@startcommands*{#1}{captions}%
2571   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2572     \def\bbl@tempb##1{% elt for \bbl@captionslist
2573       \ifx##1\@nnil\else
2574         \bbl@exp{%
2575           \\SetString\\##1{%

```

```

2576         \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2577         \expandafter\bbl@tempb
2578         \fi}%
2579         \expandafter\bbl@tempb\bbl@captionslist\@nnil
2580     \else
2581         \ifx\bbl@initoload\relax
2582             \bbl@read@ini{\bbl@KVP@captions}2%   % Here letters cat = 11
2583         \else
2584             \bbl@read@ini{\bbl@initoload}2%       % Same
2585         \fi
2586     \fi
2587 \StartBabelCommands*{#1}{date}%
2588 \ifx\bbl@KVP@date\@nnil
2589     \bbl@exp{%
2590         \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2591     \else
2592         \bbl@savetoday
2593         \bbl@savedate
2594     \fi
2595 \bbl@endcommands
2596 \bbl@load@basic{#1}%
2597 % == hyphenmins == (only if new)
2598 \bbl@exp{%
2599     \gdef<#1hyphenmins>{%
2600         {\bbl@ifunset{\bbl@lfthm@#1}{2}{\bbl@cs{lfthm@#1}}}%
2601         {\bbl@ifunset{\bbl@rgthm@#1}{3}{\bbl@cs{rgthm@#1}}}}}%
2602 % == hyphenrules (also in renew) ==
2603 \bbl@provide@hyphens{#1}%
2604 % == main ==
2605 \ifx\bbl@KVP@main\@nnil\else
2606     \expandafter\main@language\expandafter{#1}%
2607 \fi}
2608 %
2609 \def\bbl@provide@renew#1{%
2610     \ifx\bbl@KVP@captions\@nnil\else
2611         \StartBabelCommands*{#1}{captions}%
2612         \bbl@read@ini{\bbl@KVP@captions}2%   % Here all letters cat = 11
2613         \EndBabelCommands
2614     \fi
2615     \ifx\bbl@KVP@date\@nnil\else
2616         \StartBabelCommands*{#1}{date}%
2617         \bbl@savetoday
2618         \bbl@savedate
2619         \EndBabelCommands
2620     \fi
2621 % == hyphenrules (also in new) ==
2622 \ifx\bbl@lbkflag\@empty
2623     \bbl@provide@hyphens{#1}%
2624 \fi
2625 % == main ==
2626 \ifx\bbl@KVP@main\@nnil\else
2627     \expandafter\main@language\expandafter{#1}%
2628 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2629 \def\bbl@load@basic#1{%
2630     \ifcase\bbl@howloaded\or\or
2631         \ifcase\csname bbl@llevel@\language\endcsname
2632             \bbl@csarg\let{lname@\language}\relax
2633         \fi
2634     \fi

```

```

2635 \bbl@ifunset{bbl@lname@#1}%
2636 {\def\BabelBeforeIni##1##2{%
2637   \begingroup
2638     \let\bbl@ini@captions@aux\@gobbletwo
2639     \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6}%
2640     \bbl@read@ini{##1}l%
2641     \ifx\bbl@initoload\relax\endinput\fi
2642   \endgroup}%
2643   \begingroup      % boxed, to avoid extra spaces:
2644   \ifx\bbl@initoload\relax
2645     \bbl@input@texini{#1}%
2646   \else
2647     \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2648   \fi
2649   \endgroup}%
2650 {}%

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2651 \def\bbl@load@info#1{%
2652   \def\BabelBeforeIni##1##2{%
2653     \begingroup
2654       \bbl@read@ini{##1}0%
2655       \endinput      % babel- .tex may contain onlypreamble's
2656     \endgroup}%
2657     {\bbl@input@texini{#1}}%

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2658 \def\bbl@provide@hyphens#1{%
2659   \@tempcnta@m@ne % a flag
2660   \ifx\bbl@KVP@hyphenrules\@nnil\else
2661     \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2662     \bbl@foreach\bbl@KVP@hyphenrules{%
2663       \ifnum\@tempcnta=\m@ne % if not yet found
2664         \bbl@ifsamestring{##1}{+}%
2665         {\bbl@carg\addlanguage{l@#1}}%
2666         {}%
2667         \bbl@ifunset{l@#1}% After a possible +
2668         {}%
2669         {\@tempcnta\@nameuse{l@#1}}%
2670       \fi}%
2671   \ifnum\@tempcnta=\m@ne
2672     \bbl@warning{%
2673       Requested 'hyphenrules' for '\language' not found:\\%
2674       \bbl@KVP@hyphenrules.\\%
2675       Using the default value. Reported}%
2676   \fi
2677   \fi
2678   \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2679     \ifx\bbl@KVP@captions@\@nnil
2680       \bbl@ifunset{bbl@hyphr@#1}{}% use value in ini, if exists
2681       {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2682        {}%
2683        {\bbl@ifunset{l@\bbl@cl{hyphr}}}%
2684        {}%
2685        {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}}%
2686     \fi
2687     \fi
2688     \bbl@ifunset{l@#1}%
2689     {\ifnum\@tempcnta=\m@ne
2690       \bbl@carg\adddialect{l@#1}\language

```

```

2691 \else
2692 \bbl@carg\adddialect{l@#1}\@tempcnta
2693 \fi}%
2694 {\ifnum\@tempcnta=\m@ne\else
2695 \global\bbl@carg\chardef{l@#1}\@tempcnta
2696 \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2697 \def\bbl@input@texini#1{%
2698 \bbl@bsphack
2699 \bbl@exp{%
2700 \catcode`\\=\14 \catcode`\\=\0
2701 \catcode`\\=\1 \catcode`\\=\2
2702 \lowercase{\InputIfFileExists{babel-#1.tex}}{}}%
2703 \catcode`\\=\the\catcode`\relax
2704 \catcode`\\=\the\catcode`\\relax
2705 \catcode`\\=\the\catcode`\relax
2706 \catcode`\\=\the\catcode`\relax}%
2707 \bbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```

2708 \def\bbl@iniline#1\bbl@iniline{%
2709 \ifnextchar[\bbl@iniset{\ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2710 \def\bbl@iniset[#1]#2\@@{\def\bbl@section{#1}}
2711 \def\bbl@iniskip#1\@@{% if starts with ;
2712 \def\bbl@inistore#1=#2\@@{% full (default)
2713 \bbl@trim@def\bbl@tempa{#1}%
2714 \bbl@trim\toks@{#2}%
2715 \bbl@ifsamestring{\bbl@tempa}{\include}%
2716 {\bbl@read@subini{\the\toks@}}%
2717 {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2718 \ifin@ \else
2719 \bbl@xin@{,identification/include.}%
2720 {,\bbl@section/\bbl@tempa}%
2721 \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2722 \bbl@exp{%
2723 \\g@addto@macro\\bbl@inidata{%
2724 \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2725 \fi}}
2726 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2727 \bbl@trim@def\bbl@tempa{#1}%
2728 \bbl@trim\toks@{#2}%
2729 \bbl@xin@{.identification.}{.\bbl@section.}%
2730 \ifin@
2731 \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2732 \\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2733 \fi}

```

4.19. Main loop in 'provide'

Now, the 'main loop', \bbl@read@ini, which **must be executed inside a group**. At this point, \bbl@inidata may contain data declared in \babelprovide, with 'slashed' keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, 'export' some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```

2734 \def\bbl@loop@ini#1{%
2735   \loop
2736     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2737     \endlinechar\m@ne
2738     \read#1 to \bbl@line
2739     \endlinechar\^^M
2740     \ifx\bbl@line\empty\else
2741       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2742     \fi
2743   \repeat}
2744 %
2745 \def\bbl@read@subini#1{%
2746   \ifx\bbl@readsubstream\undefined
2747     \csname newread\endcsname\bbl@readsubstream
2748   \fi
2749   \openin\bbl@readsubstream=babel-#1.ini
2750   \ifeof\bbl@readsubstream
2751     \bbl@error{no-ini-file}{#1}{}{}%
2752   \else
2753     {\bbl@loop@ini\bbl@readsubstream}%
2754   \fi
2755   \closein\bbl@readsubstream}
2756 %
2757 \ifx\bbl@readstream\undefined
2758   \csname newread\endcsname\bbl@readstream
2759 \fi
2760 \def\bbl@read@ini#1#2{%
2761   \global\let\bbl@extend@ini@gobble
2762   \openin\bbl@readstream=babel-#1.ini
2763   \ifeof\bbl@readstream
2764     \bbl@error{no-ini-file}{#1}{}{}%
2765   \else
2766     % == Store ini data in \bbl@inidata ==
2767     \catcode\ =10 \catcode\ =12 \catcode\#=12
2768     \catcode\[ =12 \catcode\] =12 \catcode\&=12 \catcode\&=12
2769     \catcode\;=12 \catcode\| =12 \catcode\% =14 \catcode\-=12
2770     \ifnum#2=\m@ne % Just for the info
2771       \edef\language{tag \bbl@metalang}%
2772     \fi
2773     \ifnum#2=\m@ne
2774       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.\\%
2775         Fetching the locale name from babel-#1.ini\@gobble}%
2776     \else
2777       \bbl@info{Importing
2778         \ifcase#2font and identification \or basic \fi
2779         data for '\language'\@%
2780         from babel-#1.ini. Reported}%
2781     \fi
2782     \ifnum#2<\@ne
2783       \global\let\bbl@inidata\empty
2784       \let\bbl@inistore\bbl@inistore@min % Remember it's local
2785     \fi
2786     \def\bbl@section{identification}%
2787     \bbl@exp{%
2788       \\bbl@inistore tag.ini=#1\\@@
2789       \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\@@}%
2790     \bbl@loop@ini\bbl@readstream
2791     % == Process stored data ==
2792     \ifnum#2=\m@ne
2793       \def\bbl@tempa##1 ##2\@{##1}% Get first name
2794       \def\bbl@elt##1##2##3{%

```

```

2795 \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2796 {\edef\language\language{\bbl@tempa##3 \@@}%
2797 \let\localename\language
2798 \bbl@id@assign
2799 \def\bbl@elt###1###2###3{}}%
2800 {}}%
2801 \bbl@inidata
2802 \fi
2803 \bbl@csarg\xdef\l@ini@language{##1}%
2804 \bbl@read@ini@aux
2805 % == 'Export' data ==
2806 \bbl@ini@exports{##2}%
2807 \global\bbl@csarg\let\inidata@language\bbl@inidata
2808 \global\let\bbl@inidata\empty
2809 \bbl@xin@{,\language,},{,\bbl@ini@loaded,}%
2810 \ifin@else
2811 \bbl@exp{\\bbl@add@list\\bbl@ini@loaded{\language}}%
2812 \bbl@tglobal\bbl@ini@loaded
2813 \fi
2814 \@ifpackagewith{babel}{licr=unextended}{}%
2815 {\ifcase\bbl@engine\else % Find a better place
2816 \bbl@xin@{,\bbl@cl{sbc},},{,Cyr,}%
2817 \ifin@
2818 \bbl@once{licr-cyr}{\g@addto@macro\bbl@afterload{\input{cyr2uni.def}}}%
2819 \fi
2820 \fi}%
2821 \fi
2822 \closein\bbl@readstream}
2823 \def\bbl@read@ini@aux{%
2824 \let\bbl@savestrings\empty
2825 \let\bbl@savetoday\empty
2826 \let\bbl@savestate\empty
2827 \def\bbl@elt###1##2###3{%
2828 \def\bbl@section{##1}%
2829 \in@{=date.}{=##1}% Find a better place
2830 \ifin@
2831 \bbl@ifunset\bbl@inikv@##1{%
2832 {\bbl@ini@calendar{##1}}%
2833 {}}%
2834 \fi
2835 \bbl@ifunset\bbl@inikv@##1{}%
2836 {\csname bbl@inikv@##1\endcsname{##2}{##3}}}%
2837 \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2838 \def\bbl@extend@ini@aux#1{%
2839 \bbl@startcommands*{##1}{captions}%
2840 % Activate captions/... and modify exports
2841 \bbl@csarg\def\inikv@captions.licr{##1##2}%
2842 \setlocalecaption{##1}{##1}{##2}%
2843 \def\bbl@inikv@captions##1##2{%
2844 \bbl@ini@captions@aux{##1}{##2}}%
2845 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2846 \def\bbl@exportkey##1##2##3{%
2847 \bbl@ifunset\bbl@kv@##2{}%
2848 {\expandafter\ifx\csname bbl@kv@##2\endcsname\empty\else
2849 \bbl@exp{\global\let<bbl@##1@language>\<bbl@kv@##2>}}%
2850 \fi}}%
2851 % As with \bbl@read@ini, but with some changes
2852 \bbl@read@ini@aux
2853 \bbl@ini@exports\tw@
2854 % Update inidata@lang by pretending the ini is read.

```

```

2855 \def\bbl@elt##1##2##3{%
2856 \def\bbl@section{##1}%
2857 \bbl@iniline##2=##3\bbl@iniline}%
2858 \csname bbl@inidata@#1\endcsname
2859 \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2860 \StartBabelCommands*{#1}{date}% And from the import stuff
2861 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2862 \bbl@savetoday
2863 \bbl@savestate
2864 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2865 \def\bbl@ini@calendar#1{%
2866 \lowercase{\def\bbl@tempa{=#1=}}%
2867 \bbl@replace\bbl@tempa{=date.gregorian}{}%
2868 \bbl@replace\bbl@tempa{=date.}{}%
2869 \in@{.licr=}{#1=}%
2870 \ifin@
2871 \ifcase\bbl@engine
2872 \bbl@replace\bbl@tempa{.licr=}{}%
2873 \else
2874 \let\bbl@tempa\relax
2875 \fi
2876 \fi
2877 \ifx\bbl@tempa\relax\else
2878 \bbl@replace\bbl@tempa{=}{}%
2879 \ifx\bbl@tempa@empty\else
2880 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2881 \fi
2882 \bbl@exp{%
2883 \def\<bbl@inikv@#1>####1####2{%
2884 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2885 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2886 \def\bbl@renewinikey#1/#2\@#3{%
2887 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2888 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2889 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2890 \bbl@trim\toks@{#3}% value
2891 \bbl@exp{%
2892 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2893 \\g@addto@macro\\bbl@inidata{%
2894 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2895 \def\bbl@exportkey#1#2#3{%
2896 \bbl@ifunset{\bbl@kv@#2}%
2897 {\bbl@csarg\gdef{#1@{language}}{#3}}%
2898 {\expandafter\ifx\csname bbl@kv@#2\endcsname\@empty
2899 \bbl@csarg\gdef{#1@{language}}{#3}%
2900 \else
2901 \bbl@exp{\global\let\<bbl@#1@{language}>\<bbl@kv@#2>}}%
2902 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bbl@ini@exports is called always (via \bbl@inisec), while \bbl@after@ini must be called explicitly after \bbl@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdfTeX to select a font encoding valid (and preloaded) for a language loaded on the fly.

```
2903 \def\bbl@iniwarning#1{%
2904   \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2905   {\bbl@warning{%
2906     From babel-\bbl@cs{lini@language}.ini:\\%
2907     \bbl@cs{@kv@identification.warning#1}\\%
2908     Reported}}}%
2909 %
2910 \let\bbl@release@transforms\@empty
2911 \let\bbl@release@casing\@empty
```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```
2912 \def\bbl@ini@exports#1{%
2913   % Identification always exported
2914   \bbl@iniwarning{}%
2915   \ifcase\bbl@engine
2916     \bbl@iniwarning{.pdfLaTeX}%
2917   \or
2918     \bbl@iniwarning{.luaLaTeX}%
2919   \or
2920     \bbl@iniwarning{.XeLaTeX}%
2921   \fi%
2922   \bbl@exportkey{lllevel}{identification.load.level}{}%
2923   \bbl@exportkey{elname}{identification.name.english}{}%
2924   \bbl@expf{\bbl@exportkey{lname}{identification.name.opentype}%
2925     {\csname bbl@elname@language\endcsname}}%
2926   \bbl@exportkey{tbcpr}{identification.tag.bcp47}{}%
2927   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2928   \bbl@exportkey{lbcpr}{identification.language.tag.bcp47}{}%
2929   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2930   \bbl@exportkey{esname}{identification.script.name}{}%
2931   \bbl@expf{\bbl@exportkey{sname}{identification.script.name.opentype}%
2932     {\csname bbl@esname@language\endcsname}}%
2933   \bbl@exportkey{sbcpr}{identification.script.tag.bcp47}{}%
2934   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2935   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2936   \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2937   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2938   \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2939   \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2940   % Also maps bcp47 -> language
2941   \bbl@csarg\def{bcp@map@bbl@cl{tbcpr}}{\language}%
2942   \ifcase\bbl@engine\or
2943     \directlua{%
2944       Babel.locale_props[\the\bbl@cs{id@language}].script
2945       = '\bbl@cl{sbcpr}'}%
2946   \fi
2947   % Conditional
2948   \ifnum#1>\z@ % -1 or 0 = only info, 1 = basic, 2 = (re)new
2949     \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2950     \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2951     \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2952     \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%
```

```

2953 \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2954 \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2955 \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2956 \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2957 \bbl@exportkey{intsp}{typography.intraspace}{}%
2958 \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2959 \bbl@exportkey{chrng}{characters.ranges}{}%
2960 \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2961 \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2962 \ifnum#1=\tw@ % only (re)new
2963 \bbl@exportkey{rqtex}{identification.require.babel}{}%
2964 \bbl@tglobal\bbl@savetoday
2965 \bbl@tglobal\bbl@savedate
2966 \bbl@savestrings
2967 \fi
2968 \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2969 \def\bbl@inikv#1#2{%      key=value
2970 \toks@{#2}%              This hides #'s from ini values
2971 \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2972 \let\bbl@inikv@identification\bbl@inikv
2973 \let\bbl@inikv@date\bbl@inikv
2974 \let\bbl@inikv@typography\bbl@inikv
2975 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2976 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\language}\empty x-\fi}
2977 \def\bbl@inikv@characters#1#2{%
2978 \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2979 {\bbl@exp{%
2980 \\\g@addto@macro{\bbl@release@casing{%
2981 \\\bbl@casemapping}{\language}\unexpanded{#2}}}%
2982 {\in@{$casing.}{$#1}% e.g., casing.Uv = uV
2983 \ifin@
2984 \lowercase{\def\bbl@tempb{#1}%
2985 \bbl@replace\bbl@tempb{casing.}}}%
2986 \bbl@exp{\\\g@addto@macro{\bbl@release@casing{%
2987 \\\bbl@casemapping
2988 {\bbl@maybextx\bbl@tempb}{\language}\unexpanded{#2}}}%
2989 \else
2990 \bbl@inikv{#1}{#2}%
2991 \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2992 \def\bbl@inikv@counters#1#2{%
2993 \bbl@ifsamestring{#1}{digits}%
2994 {\bbl@error{digits-is-reserved}{}}}%
2995 {}%
2996 \def\bbl@tempc{#1}%
2997 \bbl@trim@def{\bbl@tempb*}{#2}%
2998 \in@{.1$}{#1$}%
2999 \ifin@
3000 \bbl@replace\bbl@tempc{.1}{}%
3001 \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\language}{%

```

```

3002 \noexpand\bbl@alphanumeric{\bbl@tempc}}%
3003 \fi
3004 \in@{.F.}{#1}%
3005 \ifin@else\in@{.S.}{#1}\fi
3006 \ifin@
3007 \bbl@csarg\protected@xdef{cnt@#1@language}{\bbl@tempb*}%
3008 \else
3009 \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
3010 \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
3011 \bbl@csarg\global\expandafter\let{cnt@#1@language}\bbl@tempa
3012 \fi
3013 \in@{.D$}{#1$}%
3014 \ifin@else\in@{.C$}{#1$}\fi
3015 \ifin@
3016 \bbl@replace\bbl@tempc{.D}{}%
3017 \bbl@replace\bbl@tempc{.C}{}%
3018 \bbl@exp\gdef\<bbl@cnt@bbl@tempc @language>####1{%
3019 \ \expandafter\ \bbl@zhnumeral\ \expandafter{\ \number####1}{\bbl@tempc}}}%
3020 \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3021 \ifcase\bbl@engine
3022 \bbl@csarg\def{inikv@captions.licr}#1#2{%
3023 \bbl@ini@captions@aux{#1}{#2}}
3024 \else
3025 \def\bbl@inikv@captions#1#2{%
3026 \bbl@ini@captions@aux{#1}{#2}}
3027 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3028 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3029 \bbl@replace\bbl@tempa{.template}{}%
3030 \def\bbl@toreplace{#1}{}%
3031 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3032 \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3033 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3034 \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}}%
3035 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}}%
3036 \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3037 \ifin@
3038 \@nameuse{\bbl@patch\bbl@tempa}%
3039 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3040 \fi
3041 \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3042 \ifin@
3043 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3044 \bbl@exp\gdef\<fnum@\bbl@tempa>{%
3045 \ \bbl@ifunset{\bbl@bbl@tempa fmt@\ \language}%
3046 {[fnum@\bbl@tempa]}%
3047 {\ \@nameuse{\bbl@bbl@tempa fmt@\ \language}}}%
3048 \fi}
3049 %
3050 \def\bbl@ini@captions@aux#1#2{%
3051 \bbl@trim\def\bbl@tempa{#1}%
3052 \bbl@xin@{.template}{\bbl@tempa}%
3053 \ifin@
3054 \bbl@ini@captions@template{#2}\language
3055 \else
3056 \bbl@ifblank{#2}%
3057 {\bbl@exp{%
3058 \toks@{\ \bbl@nocaption{\bbl@tempa name}{\language\bbl@tempa name}}}%
3059 {\bbl@trim\toks@{#2}}}%

```

```

3060 \bbl@exp{%
3061   \\\bbl@add\\bbl@savestrings{%
3062     \\\SetString<\bbl@tempa name>{\the\toks@}}}%
3063 \toks@ \expandafter{\bbl@captionslist}%
3064 \bbl@exp{\\\in@{\<\bbl@tempa name>}{\the\toks@}}}%
3065 \ifin@ \else
3066   \bbl@exp{%
3067     \\\bbl@add<\bbl@extracaps@ \language name>{\<\bbl@tempa name>}}%
3068     \\\bbl@tglobal \<\bbl@extracaps@ \language name>}%
3069 \fi
3070 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3071 \def\bbl@list@the{%
3072   part,chapter,section,subsection,subsubsection,paragraph,%
3073   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3074   table,page,footnote,mpfootnote,mpfn}
3075 %
3076 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3077   \bbl@ifunset{\bbl@map@#1@ \language name}%
3078     {\@nameuse{#1}}}%
3079     {\@nameuse{\bbl@map@#1@ \language name}}}
3080 %
3081 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3082   \ifin@csname#1\else
3083     \bbl@ifunset{\bbl@map@#1@ \language name}%
3084       {#1}%
3085       {\@nameuse{\bbl@map@#1@ \language name}}}
3086 \fi}
3087 %
3088 \def\bbl@inikv@labels#1#2{%
3089   \in@{.map}{#1}%
3090   \ifin@
3091     \in@{,dot.map,}{, #1,}%
3092     \ifin@
3093       \global\@namedef{\bbl@map@. @ \language name}{#2}%
3094       \fi
3095     \ifx\bbl@KVP@labels\@nnil\else
3096       \bbl@xin@{ map }{ \bbl@KVP@labels \space}%
3097       \ifin@
3098         \def\bbl@tempc{#1}%
3099         \bbl@replace\bbl@tempc{.map}{}%
3100         \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3101         \bbl@exp{%
3102           \gdef<\bbl@map@ \bbl@tempc @ \language name>%
3103             {\ifin@<#2>\else\\localecounter{#2}\fi}}%
3104         \bbl@foreach\bbl@list@the{%
3105           \bbl@ifunset{the##1}{}%
3106           {\bbl@ncarg\let\bbl@tempd{the##1}%
3107             \bbl@exp{%
3108               \\\bbl@sreplace<the##1>%
3109                 {\<\bbl@tempc>{##1}}%
3110                 {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3111               \\\bbl@sreplace<the##1>%
3112                 {\<\@empty @ \bbl@tempc>\<c@##1>%
3113                   {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3114                   \\\bbl@sreplace<the##1>%
3115                     {\\\csname @ \bbl@tempc \\endcsname \<c@##1>%
3116                       {\\\bbl@map@cnt{\bbl@tempc}{##1}}}%
3117                   \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3118                     \bbl@exp{\gdef<the##1>{\[the##1]}}%
3119                     \fi}}%
3120           \fi}

```

```

3121 \fi
3122 %
3123 \else
3124 % The following code is still under study. You can test it and make
3125 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3126 % language dependent.
3127 \in@{enumerate.}{#1}%
3128 \ifin@
3129 \def\bbl@tempa{#1}%
3130 \bbl@replace\bbl@tempa{enumerate.}{}%
3131 \def\bbl@toreplace{#2}%
3132 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}%
3133 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3134 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3135 \toks@{\expandafter\bbl@toreplace}%
3136 \bbl@exp{%
3137 \\\bbl@add<extras\language>{%
3138 \\\babel@save<labelenum\romannumeral\bbl@tempa>
3139 \def<labelenum\romannumeral\bbl@tempa>{\the\toks@}%
3140 \\\bbl@tglobal<extras\language>}%
3141 \fi
3142 \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3143 \def\bbl@chapttype{chapter}
3144 \ifx\@makechapterhead\@undefined
3145 \let\bbl@patchchapter\relax
3146 \else\ifx\thechapter\@undefined
3147 \let\bbl@patchchapter\relax
3148 \else\ifx\ps@headings\@undefined
3149 \let\bbl@patchchapter\relax
3150 \else
3151 \def\bbl@patchchapter{%
3152 \global\let\bbl@patchchapter\relax
3153 \gdef\bbl@chfmt{%
3154 \bbl@ifunset{bbl@\bbl@chapttype fmt@\language}%
3155 {\@chapapp\space\thechapter}%
3156 {\@nameuse{bbl@\bbl@chapttype fmt@\language}}}%
3157 \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3158 \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3159 \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3160 \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3161 \bbl@tglobal\appendix
3162 \bbl@tglobal\ps@headings
3163 \bbl@tglobal\chaptermark
3164 \bbl@tglobal\@makechapterhead}
3165 \let\bbl@patchappendix\bbl@patchchapter
3166 \fi\fi\fi
3167 \ifx\@part\@undefined
3168 \let\bbl@patchpart\relax
3169 \else
3170 \def\bbl@patchpart{%
3171 \global\let\bbl@patchpart\relax
3172 \gdef\bbl@partformat{%
3173 \bbl@ifunset{bbl@partfmt@\language}%
3174 {\@partname\nobreakspace\thepart}%
3175 {\@nameuse{bbl@partfmt@\language}}}%
3176 \bbl@sreplace\@part{\@partname\nobreakspace\thepart}{\bbl@partformat}%
3177 \bbl@tglobal\@part}
3178 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3179 \def\bbl@cal@getconvert#1#2{% Used in provide
3180   \in@{:}{#2}%
3181   \ifin@{\def\bbl@tempf{#2}\else\def#1{#2}\fi}
3182 \let\bbl@calendar@empty
3183 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3184 \def\bbl@localedate#1#2#3#4{%
3185   \begingroup
3186     \edef\bbl@they{#2}%
3187     \edef\bbl@them{#3}%
3188     \edef\bbl@thed{#4}%
3189     \edef\bbl@tempe{%
3190       \bbl@ifunset{\bbl@calpr\language\language}\bbl@cl{calpr}},%
3191       #1}%
3192     \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3193     \let\bbl@ld@calendar@empty
3194     \let\bbl@ld@variant@empty
3195     \let\bbl@ld@convert@relax
3196     \let\bbl@ld@@convert@relax
3197     \expandafter\bbl@forkv\expandafter{\bbl@tempe}{\bbl@csarg\edef{ld##1}{##2}}%
3198     \bbl@replace\bbl@ld@calendar{gregorian}{}%
3199     \ifx\bbl@ld@convert@relax
3200       \let\bbl@ld@convert\bbl@ld@@convert
3201     \fi
3202     \ifx\bbl@ld@convert@empty
3203       \ifx\bbl@ld@calendar@empty
3204         \let\bbl@ld@convert@relax
3205       \else
3206         \let\bbl@ld@convert\bbl@ld@calendar
3207       \fi
3208     \fi
3209     \ifx\bbl@ld@convert@relax\else
3210       \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3211       {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3212     \fi
3213     \@nameuse{\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3214     \edef\bbl@calendar{% Used in \month..., too
3215       \bbl@ld@calendar
3216       \ifx\bbl@ld@variant@empty\else
3217         .\bbl@ld@variant
3218       \fi}%
3219     \bbl@cased
3220     {\@nameuse{\bbl@date@\language @\bbl@calendar}%
3221      \bbl@they\bbl@them\bbl@thed}%
3222   \endgroup}
3223 %
3224 \def\bbl@printdate#1{%
3225   \ifnextchar[{\bbl@printdate@i{#1}}{\bbl@printdate@i{#1}[]}}
3226 \def\bbl@printdate@i#1[#2]#3#4#5{%
3227   \bbl@usedategrouptrue
3228   \def\bbl@tempc{#1}%
3229   \bbl@replace\bbl@tempc{-}{@}%
3230   \@nameuse{\bbl@ensure@\bbl@tempc}{\localedate[#2][#3][#4][#5]}
3231 %
3232 % e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3233 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3234   \bbl@trim@def\bbl@tempa{#1.#2}%
3235   \bbl@ifsamestring{\bbl@tempa}{months.wide}%      to savedate
3236   {\bbl@trim@def\bbl@tempa{#3}%
3237    \bbl@trim\toks@{#5}%
3238    \@temptokena\expandafter{\bbl@savedate}%
3239    \bbl@exp{% Reverse order - in ini last wins

```

```

3240 \def\\bbl@savestate{%
3241   \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3242   \the\@temptokena}}}%
3243 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3244 {\lowercase{\def\bbl@tempb{#6}}}%
3245 \bbl@trim@def\bbl@toreplace{#5}%
3246 \bbl@TG@@date
3247 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3248 \ifx\bbl@savestate\empty
3249 \bbl@exp{%
3250   \\AfterBabelCommands{%
3251     \gdef\<\language name date>{\\protect\<\language name date >}%
3252     \gdef\<\language name date >{\\bbl@printdate{\language name}}}%
3253   \def\\bbl@savestate{%
3254     \\SetString\\today{%
3255       \<\language name date>[@convert]%
3256       {\the\year}{\the\month}{\the\day}}}%
3257   \fi}%
3258 {}}}

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after `\bbl@replace \toks@` contains the resulting string, which is used by `\bbl@replace@finish@iii` (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3259 \let\bbl@calendar\empty
3260 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3261   \bbl@xin@{\$calendrica:}{\$#2}%
3262   \ifin@
3263     \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3264     \edef\bbl@tempa{\$#2}%
3265     \bbl@replace\bbl@tempa{\$calendrica:}{}%
3266     \bbl@replace\bbl@tempa{-}{_}%
3267     \bbl@afterelse
3268     \bbl@exp{\\bbl@calendrica#1\\@@{\bbl@tempa}}%
3269   \else
3270     \bbl@afterfi
3271     \@nameuse{\bbl@ca#2}#1\@@
3272   \fi}
3273 \def\bbl@calendrica#1-#2-#3\@@#4#5#6#7{%
3274   \directlua{
3275     local d = Babel.calendrica.new_date(
3276       {year=\number#1, month=\number#2, day=\number#3},
3277       Babel.calendrica.cal_gregorian)
3278     local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3279     token.set_macro('\bbl@stripslash#5', r.year)
3280     token.set_macro('\bbl@stripslash#6', r.month)
3281     token.set_macro('\bbl@stripslash#7', r.day) }}
3282 \newcommand\BabelDateSpace{\nobreakspace}
3283 \newcommand\BabelDateDot{.\@}
3284 \newcommand\BabelDated[1]{\number#1}
3285 \newcommand\BabelDatedd[1]{\ifnum#1<10 0\fi\number#1}
3286 \newcommand\BabelDateM[1]{\number#1}
3287 \newcommand\BabelDateMM[1]{\ifnum#1<10 0\fi\number#1}
3288 \newcommand\BabelDateMMM[1]{%
3289   \csname month\romannumeral#1\bbl@calendar name\endcsname}%
3290 \newcommand\BabelDatey[1]{\number#1}%
3291 \newcommand\BabelDateyy[1]{%
3292   \ifnum#1<10 0\number#1 %
3293   \else\ifnum#1<100 \number#1 %
3294   \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3295   \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3296   \else

```

```

3297 \bbl@error{limit-two-digits}{\}{\}%
3298 \fi\fi\fi\fi}
3299 \newcommand\BabelDateyyy[1]{\number#1}
3300 \newcommand\BabelDateU[1]{\number#1}%
3301 \def\bbl@replace@finish@iii#1{%
3302 \bbl@exp{\def\#1###1###2###3{\the\toks@}}
3303 \def\bbl@TG@date{%
3304 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace}}%
3305 \bbl@replace\bbl@toreplace{[. ]}{\BabelDateDot}}%
3306 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{###1}}%
3307 \bbl@replace\bbl@toreplace{[y|]}{\bbl@datecctr{###1|}}%
3308 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{###1}}%
3309 \bbl@replace\bbl@toreplace{[yyyy]}{\BabelDateyyy{###1}}%
3310 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{###2}}%
3311 \bbl@replace\bbl@toreplace{[M|]}{\bbl@datecctr{###2|}}%
3312 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{###2}}%
3313 \bbl@replace\bbl@toreplace{[MMM]}{\BabelDateMMM{###2}}%
3314 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{###3}}%
3315 \bbl@replace\bbl@toreplace{[d|]}{\bbl@datecctr{###3|}}%
3316 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{###3}}%
3317 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{###1}}%
3318 \bbl@replace\bbl@toreplace{[U|]}{\bbl@datecctr{###1|}}%
3319 \bbl@replace@finish@iii\bbl@toreplace}
3320 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3321 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3322 \AddToHook{begindocument/before}{%
3323 \let\bbl@normalsf\normalsfcodes
3324 \let\normalsfcodes\relax}
3325 \AtBeginDocument{%
3326 \ifx\bbl@normalsf\empty
3327 \ifnum\sfcodes\@m
3328 \let\normalsfcodes\frenchspacing
3329 \else
3330 \let\normalsfcodes\nonfrenchspacing
3331 \fi
3332 \else
3333 \let\normalsfcodes\bbl@normalsf
3334 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3335 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3336 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3337 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3338 #1[#2]{#3}{#4}{#5}}
3339 \begingroup
3340 \catcode\%=12
3341 \catcode\&=14
3342 \gdef\bbl@transforms#1#2#3{\&%
3343 \directlua{
3344 local str = [=[#2]=]
3345 str = str:gsub('%.%d+%.%d+$', '')
3346 token.set_macro('babeltempa', str)
3347 }&%

```

```

3348 \def\babeltempc{}&%
3349 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3350 \ifin@else
3351 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3352 \fi
3353 \ifin@
3354 \bbl@foreach\bbl@KVP@transforms{&%
3355 \bbl@xin@{\babeltempa,}{,##1,}&%
3356 \ifin@ &% font:font:transform syntax
3357 \directlua{
3358 local t = {}
3359 for m in string.gmatch('##1'..'':', '(.):') do
3360 table.insert(t, m)
3361 end
3362 table.remove(t)
3363 token.set_macro('babeltempc', ', fonts=' .. table.concat(t, ' '))
3364 }&%
3365 \fi}&%
3366 \in@{.0$}{#2$}&%
3367 \ifin@
3368 \directlua{&% (\attribute) syntax
3369 local str = string.match([[ \bbl@KVP@transforms]],
3370 '%([^(%[]-)%)[^%)]-\babeltempa')
3371 if str == nil then
3372 token.set_macro('babeltempb', '')
3373 else
3374 token.set_macro('babeltempb', ', attribute=' .. str)
3375 end
3376 }&%
3377 \toks@{#3}&%
3378 \bbl@exp{&%
3379 \\g@addto@macro\\bbl@release@transforms{&%
3380 \relax &% Closes previous \bbl@transforms@aux
3381 \\bbl@transforms@aux
3382 \\#1{label=\babeltempa\babeltempb\babeltempc}&%
3383 {\language\the\toks@}}&%
3384 \else
3385 \g@addto@macro\bbl@release@transforms{, {#3}}&%
3386 \fi
3387 \fi}
3388 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3389 \def\bbl@provide@lsys#1{%
3390 \bbl@ifunset\bbl@lname@#1{%
3391 {\bbl@load@info{#1}}%
3392 }%
3393 \bbl@csarg\let{lsys@#1}\@empty
3394 \bbl@ifunset\bbl@sname@#1{\bbl@csarg\gdef{sname@#1}{Default}}{%
3395 \bbl@ifunset\bbl@sotf@#1{\bbl@csarg\gdef{sotf@#1}{DFLT}}{%
3396 \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3397 \bbl@ifunset\bbl@lname@#1{%
3398 {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3399 \ifcase\bbl@engine\or\or
3400 \bbl@ifunset\bbl@prehc@#1{%
3401 {\bbl@exp{\\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3402 }%
3403 {\ifx\bbl@xenohyph\undefined

```

```

3404      \global\let\bbl@xenoxyph\bbl@xenoxyph@d
3405      \ifx\AtBeginDocument\@notprerr
3406        \expandafter\@secondoftwo % to execute right now
3407      \fi
3408      \AtBeginDocument{%
3409        \bbl@patchfont{\bbl@xenoxyph}%
3410        {\expandafter\select@language\expandafter{\language}}}%
3411    \fi}%
3412  \fi
3413  \bbl@csarg\bbl@toglobal{\sys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the `ini` file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\text{T}_{\text{E}}\text{X}$. Non-digits characters are kept. The first macro is the generic “localized” command.

[illegible]

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```

3445 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}%
3446   \ifx\#1%
3447     \bbl@exp{%
3448       \def\\bbl@tempa####1{%
3449         \<ifcase>####1\space\the\toks@\<else>\\@ctrerrr\<fi>}}%
3450   \else
3451     \toks@\expandafter{\the\toks@\or #1}%
3452     \expandafter\bbl@buildifcase
3453   \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before \@@ collect digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210.

Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```

3454 \newcommand\localenumberal[2]{%
3455   \bbl@ifunset{\bbl@cntr@#1@\language\language}%
3456   {#2}%
3457   {\bbl@cs{cntr@#1@\language}{#2}}}
3458 \def\bbl@localecntr#1#2{\localenumberal{#2}{#1}}
3459 \newcommand\localecounter[2]{%
3460   \expandafter\bbl@localecntr
3461   \expandafter{\number\csname c@#2\endcsname}{#1}}
3462 \def\bbl@alphnumer#1#2{%
3463   \expandafter\bbl@alphnumer@i\number#2 76543210\@@{#1}}
3464 \def\bbl@alphnumer@i#1#2#3#4#5#6#7#8\@@#9{%
3465   \ifcase\@car#8\@nil\or    % Currently <10000, but prepared for bigger
3466   \bbl@alphnumer@ii{#9}00000#1\or
3467   \bbl@alphnumer@ii{#9}00000#1#2\or
3468   \bbl@alphnumer@ii{#9}00000#1#2#3\or
3469   \bbl@alphnumer@ii{#9}00000#1#2#3#4\else
3470   \bbl@error{counter-too-large}{>9999}{}}%
3471   \fi}
3472 \def\bbl@alphnumer@ii#1#2#3#4#5#6#7#8{%
3473   \bbl@ifunset{\bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3474   {\bbl@cs{cntr@#1.4@\language}{#5}%
3475   \bbl@cs{cntr@#1.3@\language}{#6}%
3476   \bbl@cs{cntr@#1.2@\language}{#7}%
3477   \bbl@cs{cntr@#1.1@\language}{#8}%
3478   \ifnum#6#7#8>\z@
3479   \bbl@ifunset{\bbl@cntr@#1.S.321@\language}{}}%
3480   {\bbl@cs{cntr@#1.S.321@\language}{}}%
3481   \fi}%
3482   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
```

Some Chinese counters follow special rules.

```

3483 \def\bbl@zhnumer#1#2{%
3484   \ifnum#1<10 \bbl@zhnum000#1{#2}%
3485   \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3486   \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3487   \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3488   \else\bbl@error{counter-too-large}{#1}{}}%
3489   \fi\fi\fi\fi}
3490 \def\bbl@zhnum#1#2#3#4#5{%
3491   \ifnum\numexpr#1#2#3#4\relax=\z@
3492   \localenumberal{#5.C}\@ne
3493   \else
3494   \ifnum#1>\z@\localenumberal{#5.D}{#1}\localenumberal{#5.C}5\fi % Thousands
3495   \ifnum#2>\z@\localenumberal{#5.D}{#2}\localenumberal{#5.C}4\fi % Hundreds
3496   \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumberal{#5.C}\@ne\fi
3497   \fi\fi
3498   \ifnum#3>\z@ % Tens
3499   \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumberal{#5.D}{#3}\fi % No  for 10-19:
3500   \else\localenumberal{#5.D}{#3}%
3501   \fi
3502   \localenumberal{#5.C}\thr@@
3503   \else
3504   \ifnum#2>\z@\ifnum#4>\z@\localenumberal{#5.C}\@ne\fi\fi
3505   \fi
3506   \ifnum#4>\z@\localenumberal{#5.D}{#4}\fi % Units
3507   \fi}
```

4.24. Casing

```

3508 \newcommand\BabelUppercaseMapping[3]{%
3509   \DeclareUppercaseMapping[\@nameuse{\bbl@casing@#1}]{#2}{#3}}
```

```

3510 \newcommand\BabelTitlecaseMapping[3]{%
3511   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3512 \newcommand\BabelLowercaseMapping[3]{%
3513   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

  The parser for casing and casing. (variant).
3514 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3515   \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3516 \else
3517   \def\bbl@uftocode#1{\expandafter`\string#1}
3518 \fi
3519 \def\bbl@casemapping#1#2#3{% 1:variant
3520   \def\bbl@tempa##1 ##2{% Loop
3521     \bbl@casemapping@i{##1}%
3522     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3523   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3524   \def\bbl@tempe{0}% Mode (upper/lower...)
3525   \def\bbl@tempc{#3}% Casing list
3526   \expandafter\bbl@tempa\bbl@tempc\@empty}
3527 \def\bbl@casemapping@i#1{%
3528   \def\bbl@tempb{#1}%
3529   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3530     \@nameuse{regex_replace_all:nnN}%
3531     {[{\x{c0}-\x{ff}}][\x{80}-\x{bf}]}*}{\@empty}\bbl@tempb
3532   \else
3533     \@nameuse{regex_replace_all:nnN}{.}{\@empty}\bbl@tempb
3534   \fi
3535   \expandafter\bbl@casemapping@ii\bbl@tempb\@empty}
3536 \def\bbl@casemapping@ii#1#2#3\@empty{%
3537   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3538   \ifin@
3539     \edef\bbl@tempe{%
3540       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3541   \else
3542     \ifcase\bbl@tempe\relax
3543       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3544       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3545     \or
3546       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3547     \or
3548       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3549     \or
3550       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3551     \fi
3552   \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3553 \def\bbl@localeinfo#1#2{%
3554   \bbl@ifunset{bbl@info@#2}{#1}%
3555   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3556    {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3557 \newcommand\localeinfo[1]{%
3558   \ifx*#1\@empty
3559     \bbl@afterelse\bbl@localeinfo{}%
3560   \else
3561     \bbl@localeinfo
3562     {\bbl@error{no-ini-info}}{\{\}\{\}\}%
3563     {#1}%
3564   \fi}
3565 % \@namedef{bbl@info@name.locale}{lcname}
3566 \@namedef{bbl@info@tag.ini}{lini}

```

```

3567 \@namedef{bbl@info@name.english}{elname}
3568 \@namedef{bbl@info@name.opentype}{lname}
3569 \@namedef{bbl@info@tag.bcp47}{tbc}
3570 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3571 \@namedef{bbl@info@tag.opentype}{lotf}
3572 \@namedef{bbl@info@script.name}{esname}
3573 \@namedef{bbl@info@script.name.opentype}{sname}
3574 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3575 \@namedef{bbl@info@script.tag.opentype}{sotf}
3576 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3577 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3578 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3579 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3580 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3581 <(*More package options)> ≡
3582 \DeclareOption{ensureinfo=off}{}
3583 <(/More package options)>
3584 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3585 \newcommand\getlocaleproperty{%
3586   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3587 \def\bbl@getproperty@s#1#2#3{%
3588   \let#1\relax
3589   \def\bbl@elt##1##2##3{%
3590     \bbl@ifsamestring{##1/##2}{#3}%
3591     {\providecommand#1{##3}%
3592     \def\bbl@elt####1####2####3{}}}%
3593   {}}%
3594   \bbl@cs{inidata@#2}}%
3595 \def\bbl@getproperty@x#1#2#3{%
3596   \bbl@getproperty@s{#1}{#2}{#3}%
3597   \ifx#1\relax
3598     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3599   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3600 \let\bbl@ini@loaded\@empty
3601 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3602 \def\ShowLocaleProperties#1{%
3603   \typeout{}}%
3604   \typeout{*** Properties for language '#1' ***}%
3605   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3606   \@nameuse{bbl@inidata@#1}%
3607   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 47 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcptoname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3608 \newif\ifbbl@bcpallowed
3609 \bbl@bcpallowedfalse
3610 \def\bbl@autoload@options{@import}
3611 \def\bbl@provide@locale{%

```

```

3612 \ifx\babelprovide\undefined
3613   \bbl@error{base-on-the-fly}{\}\}%
3614 \fi
3615 \let\bbl@auxname\language
3616 \ifbbl@bcptoname
3617   \bbl@ifunset{bbl@bcp@map@\language}{\}% Move uplevel??
3618   {\edef\language{\@nameuse{bbl@bcp@map@\language}}\%
3619     \let\localname\language}%
3620 \fi
3621 \ifbbl@bcpallowed
3622   \expandafter\ifx\csname date\language\endcsname\relax
3623     \expandafter
3624     \bbl@bcplookup{\language}%
3625     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3626       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3627       \let\localname\language
3628       \expandafter\ifx\csname date\language\endcsname\relax
3629         \let\bbl@initoload\bbl@bcp
3630         \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3631         \let\bbl@initoload\relax
3632       \fi
3633       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localname}%
3634     \fi
3635   \fi
3636 \fi
3637 \expandafter\ifx\csname date\language\endcsname\relax
3638   \IfFileExists{babel-\language.tex}%
3639   {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3640   {\}%
3641 \fi}

```

$\LaTeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension `.<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to tag.bcp47.

```

3642 \providecommand\BCPdata{}
3643 \ifx\renewcommand\undefined\else
3644   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3645   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3646     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3647     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3648     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3649   \def\bbl@bcpdata@ii#1#2{%
3650     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3651     {\bbl@error{unknown-ini-field}{#1}{\}\}%
3652     {\bbl@ifunset{bbl@\csname bbl@info@#1.tag.bcp47\endcsname @#2}{\}%
3653       {\bbl@cs{\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}}
3654 \fi
3655 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3656 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3657 \newcommand\babeladjust[1]{%
3658   \bbl@forkv{#1}{%
3659     \bbl@ifunset{bbl@ADJ@##1@##2}%
3660     {\bbl@cs{ADJ@##1}{##2}}%
3661     {\bbl@cs{ADJ@##1@##2}}}%
3662 %
3663 \def\bbl@adjust@lua#1#2{%

```

```

3664 \ifvmode
3665   \ifnum\currentgrouplevel=\z@
3666     \directlua{ Babel.#2 }%
3667     \expandafter\expandafter\expandafter\@gobble
3668   \fi
3669 \fi
3670 {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3671 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3672   \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3673 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3674   \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3675 \@namedef{bbl@ADJ@bidi.text@on}{%
3676   \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3677 \@namedef{bbl@ADJ@bidi.text@off}{%
3678   \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3679 \@namedef{bbl@ADJ@bidi.math@on}{%
3680   \let\bbl@noamsmath\@empty}
3681 \@namedef{bbl@ADJ@bidi.math@off}{%
3682   \let\bbl@noamsmath\relax}
3683 %
3684 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3685   \bbl@adjust@lua{bidi}{digits_mapped=true}}
3686 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3687   \bbl@adjust@lua{bidi}{digits_mapped=false}}
3688 %
3689 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3690   \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3691 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3692   \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3693 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3694   \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3695 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3696   \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3697 \@namedef{bbl@ADJ@justify.arabic@on}{%
3698   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3699 \@namedef{bbl@ADJ@justify.arabic@off}{%
3700   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3701 %
3702 \def\bbl@adjust@layout#1{%
3703   \ifvmode
3704     #1%
3705     \expandafter\@gobble
3706   \fi
3707   {\bbl@error{layout-only-vertical}{}}}% Gobbled if everything went ok.
3708 \@namedef{bbl@ADJ@layout.tabular@on}{%
3709   \ifnum\bbl@tabular@mode=\tw@
3710     \bbl@adjust@layout{\let\@tabular\bbl@NL@@tabular}%
3711   \else
3712     \chardef\bbl@tabular@mode\@ne
3713   \fi}
3714 \@namedef{bbl@ADJ@layout.tabular@off}{%
3715   \ifnum\bbl@tabular@mode=\tw@
3716     \bbl@adjust@layout{\let\@tabular\bbl@OL@@tabular}%
3717   \else
3718     \chardef\bbl@tabular@mode\z@
3719   \fi}
3720 \@namedef{bbl@ADJ@layout.lists@on}{%
3721   \bbl@adjust@layout{\let\list\bbl@NL@list}}
3722 \@namedef{bbl@ADJ@layout.lists@off}{%
3723   \bbl@adjust@layout{\let\list\bbl@OL@list}}
3724 %
3725 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3726   \bbl@bcpallowedtrue}

```

```

3727 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3728   \bbl@bcppallowedfalse}
3729 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3730   \def\bbl@bcp@prefix{#1}}
3731 \def\bbl@bcp@prefix{bcp47-}
3732 \@namedef{bbl@ADJ@autoload.options}#1{%
3733   \def\bbl@autoload@options{#1}}
3734 \def\bbl@autoload@bcptoptions{import}
3735 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3736   \def\bbl@autoload@bcptoptions{#1}}
3737 \newif\ifbbl@bcptname
3738 %
3739 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3740   \bbl@bcptnametrue}
3741 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3742   \bbl@bcptnamefalse}
3743 %
3744 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3745   \directlua{ Babel.ignore_pre_char = function(node)
3746     return (node.lang == \the\csname \@nohyphenation\endcsname)
3747   end }}
3748 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3749   \directlua{ Babel.ignore_pre_char = function(node)
3750     return false
3751   end }}
3752 %
3753 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3754   \def\bbl@ignoreinterchar{%
3755     \ifnum\language=\@nohyphenation
3756       \expandafter\@gobble
3757     \else
3758       \expandafter\@firstofone
3759     \fi}}
3760 \@namedef{bbl@ADJ@interchar.disable@off}{%
3761   \let\bbl@ignoreinterchar\@firstofone}
3762 %
3763 \@namedef{bbl@ADJ@select.write@shift}{%
3764   \let\bbl@restorelastskip\relax
3765   \def\bbl@savelastskip{%
3766     \let\bbl@restorelastskip\relax
3767     \ifvmode
3768       \ifdim\lastskip=\z@
3769         \let\bbl@restorelastskip\nobreak
3770       \else
3771         \bbl@exp{%
3772           \def\\bbl@restorelastskip{%
3773             \skip@=\the\lastskip
3774             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3775         \fi
3776       \fi}}
3777 \@namedef{bbl@ADJ@select.write@keep}{%
3778   \let\bbl@restorelastskip\relax
3779   \let\bbl@savelastskip\relax}
3780 \@namedef{bbl@ADJ@select.write@omit}{%
3781   \AddBabelHook{babel-select}{beforestart}{%
3782     \expandafter\babel@aux\expandafter{\bbl@main@language}}}%
3783   \let\bbl@restorelastskip\relax
3784   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3785 \@namedef{bbl@ADJ@select.encoding@off}{%
3786   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The *key* argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```
3787 ({*More package options}) ≡
3788 \DeclareOption{safe=none}{\let\bbl@opt@safe\empty}
3789 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3790 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3791 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3792 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3793 ({/More package options})
```

\@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```
3794 \bbl@trace{Cross referencing macros}
3795 \ifx\bbl@opt@safe\empty\else % i.e., if 'ref' and/or 'bib'
3796   \def\@newl@bel#1#2#3{%
3797     {\@safe@activestrue
3798       \bbl@ifunset{#1@#2}%
3799       \relax
3800       {\gdef\@multiplelabels{%
3801         \@latex@warning@no@line{There were multiply-defined labels}}%
3802         \@latex@warning@no@line{Label `#2' multiply defined}}%
3803       \global\@namedef{#1@#2}{#3}}}
```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```
3804 \CheckCommand*\@testdef[3]{%
3805   \def\reserved@a{#3}%
3806   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3807   \else
3808     \@tempswatrue
3809   \fi}
```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```
3810 \def\@testdef#1#2#3{%
3811   \@safe@activestrue
3812   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3813   \def\bbl@tempb{#3}%
3814   \@safe@activesfalse
3815   \ifx\bbl@tempa\relax
3816   \else
3817     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3818   \fi
3819   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3820   \ifx\bbl@tempa\bbl@tempb
3821   \else
3822     \@tempswatrue
3823   \fi}
3824 \fi
```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```
3825 \bbl@xin@{R}\bbl@opt@safe
3826 \ifin@
3827 \edef\bbl@tempc{\expandafter\string\csname ref_code\endcsname}%
3828 \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3829 {\expandafter\strip@prefix\meaning\ref}%
3830 \ifin@
3831 \bbl@redefine\@kernel@ref#1{%
3832   \@safe@activetrue\org@@kernel@ref{#1}\@safe@activfalse}
3833 \bbl@redefine\@kernel@pageref#1{%
3834   \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activfalse}
3835 \bbl@redefine\@kernel@sref#1{%
3836   \@safe@activetrue\org@@kernel@sref{#1}\@safe@activfalse}
3837 \bbl@redefine\@kernel@spageref#1{%
3838   \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activfalse}
3839 \else
3840 \bbl@redefinero bust\ref#1{%
3841   \@safe@activetrue\org@ref{#1}\@safe@activfalse}
3842 \bbl@redefinero bust\pageref#1{%
3843   \@safe@activetrue\org@pageref{#1}\@safe@activfalse}
3844 \fi
3845 \else
3846 \let\org@ref\ref
3847 \let\org@pageref\pageref
3848 \fi
```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```
3849 \bbl@xin@{B}\bbl@opt@safe
3850 \ifin@
3851 \bbl@redefine\@citex[#1]#2{%
3852   \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activfalse
3853   \org@@citex[#1]{\bbl@tempa}}
```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```
3854 \AtBeginDocument{%
3855   \@ifpackageloaded{natbib}{%
3856     \def\@citex[#1][#2]#3{%
3857       \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activfalse
3858       \org@@citex[#1][#2]{\bbl@tempa}}%
3859   }}
```

The package `cite` has a definition of `\@citex` where the shorthands need to be turned off in both arguments.

```
3860 \AtBeginDocument{%
3861   \@ifpackageloaded{cite}{%
3862     \def\@citex[#1]#2{%
3863       \@safe@activetrue\org@@citex[#1][#2]\@safe@activfalse}%
3864   }}
```

\nocite The macro \nocite which is used to instruct BiBTeX to extract uncited references from the database.

```
3865 \bbl@redefine\nocite#1{%
3866   \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}
```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \bibcite is needed we define \bibcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \bibcite. This new definition is then activated.

```
3867 \bbl@redefine\bibcite{%
3868   \bbl@cite@choice
3869   \bibcite}
```

\bbl@bibcite The macro \bbl@bibcite holds the definition of \bibcite needed when neither natbib nor cite is loaded.

```
3870 \def\bbl@bibcite#1#2{%
3871   \org@bibcite{#1}{\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3872 \def\bbl@cite@choice{%
3873   \global\let\bibcite\bbl@bibcite
3874   \@ifpackageloaded{natbib}{\global\let\bibcite\org@bibcite}{}%
3875   \@ifpackageloaded{cite}{\global\let\bibcite\org@bibcite}{}%
3876   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3877 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal L^AT_EX macros called by \bibitem that write the citation label on the aux file.

```
3878 \bbl@redefine\@bibitem#1{%
3879   \@safe@activetrue\org@@bibitem{#1}\@safe@activesfalse}
3880 \else
3881   \let\org@nocite\nocite
3882   \let\org@@citex\@citex
3883   \let\org@bibcite\bibcite
3884   \let\org@@bibitem\@bibitem
3885 \fi
```

5.2. Layout

```
3886 \newcommand\BabelPatchSection[1]{%
3887   \@ifundefined{#1}{}{%
3888     \bbl@exp{\let\bbl@ss@#1>\<#1>}%
3889     \@namedef{#1}{%
3890       \ifstar{\bbl@presec@#1}%
3891       {\@dblarg{\bbl@presec@x{#1}}}}%
3892   \def\bbl@presec@x#1[#2]#3{%
3893     \bbl@exp{%
3894       \\select@language@x{\bbl@main@language}%
3895       \\bbl@cs{sspre@#1}%
3896       \\bbl@cs{ss@#1}%
3897       [\\foreignlanguage{\language}\unexpanded{#2}}}%
3898       {\\foreignlanguage{\language}\unexpanded{#3}}}%
3899     \\select@language@x{\language}}}
```

```

3900 \def\bbl@presec@s#1#2{%
3901   \bbl@exp{%
3902     \\select@language@x{\bbl@main@language}%
3903     \\bbl@cs{sspre@#1}%
3904     \\bbl@cs{ss@#1}*%
3905     {\foreignlanguage{\language}{\unexpanded{#2}}}%
3906     \\select@language@x{\language}}%
3907 %
3908 \IfBabelLayout{sectioning}%
3909   {\BabelPatchSection{part}%
3910    \BabelPatchSection{chapter}%
3911    \BabelPatchSection{section}%
3912    \BabelPatchSection{subsection}%
3913    \BabelPatchSection{subsubsection}%
3914    \BabelPatchSection{paragraph}%
3915    \BabelPatchSection{subparagraph}%
3916    \def\babel@toc#1{%
3917      \select@language@x{\bbl@main@language}}}%
3918 \IfBabelLayout{captions}%
3919   {\BabelPatchSection{caption}}}%

```

\BabelFootnote Footnotes.

```

3920 \bbl@trace{Footnotes}
3921 \def\bbl@footnote#1#2#3{%
3922   \ifnextchar[%
3923     {\bbl@footnote@o{#1}{#2}{#3}}%
3924     {\bbl@footnote@x{#1}{#2}{#3}}}
3925 \long\def\bbl@footnote@x#1#2#3#4{%
3926   \bgroup
3927     \select@language@x{\bbl@main@language}%
3928     \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3929   \egroup}
3930 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3931   \bgroup
3932     \select@language@x{\bbl@main@language}%
3933     \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3934   \egroup}
3935 \def\bbl@footnotetext#1#2#3{%
3936   \ifnextchar[%
3937     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3938     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3939 \long\def\bbl@footnotetext@x#1#2#3#4{%
3940   \bgroup
3941     \select@language@x{\bbl@main@language}%
3942     \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3943   \egroup}
3944 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3945   \bgroup
3946     \select@language@x{\bbl@main@language}%
3947     \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3948   \egroup}
3949 \def\BabelFootnote#1#2#3#4{%
3950   \ifx\bbl@fn@footnote\undefined
3951     \let\bbl@fn@footnote\footnote
3952   \fi
3953   \ifx\bbl@fn@footnotetext\undefined
3954     \let\bbl@fn@footnotetext\footnotetext
3955   \fi
3956   \bbl@iifblank{#2}%
3957     {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3958      \namedef{\bbl@stripslash#1text}%
3959      {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3960     {\def#1{\bbl@exp{\\bbl@footnote{\\foreignlanguage{#2}}}{#3}{#4}}}%

```

```

3961 \namedef{\bbl@stripslash#1text}%
3962 {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}{#3}{#4}}}%
3963 \IfBabelLayout{footnotes}%
3964 {\let\bbl@OL@footnote\footnote
3965 \BabelFootnote\footnote\languagesname{}}%
3966 \BabelFootnote\localfootnote\languagesname{}}%
3967 \BabelFootnote\mainfootnote{}}%
3968 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3969 \bbl@trace{Marks}
3970 \IfBabelLayout{sectioning}
3971 {\ifx\bbl@opt@headfoot\@nnil
3972 \g@addto@macro\resetactivechars{%
3973 \set@typeset@protect
3974 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3975 \let\protect\noexpand
3976 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3977 \edef\thepage{%
3978 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3979 \fi}%
3980 \fi}
3981 {\ifbbl@single\else
3982 \bbl@ifunset{markright } \bbl@redefine\bbl@redefineroobust
3983 \markright#1{%
3984 \bbl@ifblank{#1}%
3985 {\org@markright{}}%
3986 {\toks@{#1}%
3987 \bbl@exp{%
3988 \org@markright{\protect\foreignlanguage{\languagesname}%
3989 \protect\bbl@restore@actives\the\toks@}}}%

```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, ~~La~~^{TeX} stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3990 \ifx\@mkboth\markboth
3991 \def\bbl@tempc{\let\@mkboth\markboth}%
3992 \else
3993 \def\bbl@tempc{%
3994 \fi
3995 \bbl@ifunset{markboth } \bbl@redefine\bbl@redefineroobust
3996 \markboth#1#2{%
3997 \protected@edef\bbl@tempb##1{%
3998 \protect\foreignlanguage
3999 {\languagesname}{\protect\bbl@restore@actives##1}}%
4000 \bbl@ifblank{#1}%
4001 {\toks@{}}%
4002 {\toks@\expandafter{\bbl@tempb{#1}}}%
4003 \bbl@ifblank{#2}%
4004 {\@temptokena{}}%
4005 {\@temptokena\expandafter{\bbl@tempb{#2}}}%

```

```

4006      \bbl@exp{\org@markboth{\the\toks@}{\the\temptokena}}}%
4007      \bbl@tempc
4008      \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}{
%      {code for odd pages}
%      {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

4009 \bbl@trace{Preventing clashes with other packages}
4010 \ifx\org@ref\undefined\else
4011   \bbl@xin@{R}\bbl@opt@safe
4012   \ifin@
4013     \AtBeginDocument{%
4014       \@ifpackageloaded{ifthen}{%
4015         \bbl@redefine@long\ifthenelse#1#2#3{%
4016           \let\bbl@temp@pref\pageref
4017           \let\pageref\org@pageref
4018           \let\bbl@temp@ref\ref
4019           \let\ref\org@ref
4020           \@safe@activestrue
4021           \org@ifthenelse{#1}%
4022             {\let\pageref\bbl@temp@pref
4023              \let\ref\bbl@temp@ref
4024              \@safe@activesfalse
4025              #2}%
4026             {\let\pageref\bbl@temp@pref
4027              \let\ref\bbl@temp@ref
4028              \@safe@activesfalse
4029              #3}%
4030           }%
4031         }{}%
4032       }
4033 \fi

```

5.4.2. varioref

\@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

4034 \AtBeginDocument{%
4035   \@ifpackageloaded{varioref}{%
4036     \bbl@redefine\@vpageref#1[#2]#3{%
4037       \@safe@activestrue
4038       \org@@@vpageref{#1}[#2]{#3}%
4039       \@safe@activesfalse}%

```

```

4040 \bbl@redefine\vrefpagemum#1#2{%
4041 \@safe@activestrue
4042 \org\vrefpagemum{#1}{#2}%
4043 \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

4044 \expandafter\def\csname Ref \endcsname#1{%
4045 \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4046 }{}%
4047 }
4048 \fi

```

5.4.3. `hhline`

`\hhline` Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the “`:`” character which is made active by the french support in `babel`. Therefore we need to *reload* the package when the “`:`” is an active character. Note that this happens *after* the category code of the `@`-sign has been changed to other, so we need to temporarily change it to letter again.

```

4049 \AtEndOfPackage{%
4050 \AtBeginDocument{%
4051 \@ifpackageloaded{hhline}%
4052 {\expandafter\ifx\csname normal@char\string\endcsname\relax
4053 \else
4054 \makeatletter
4055 \def\@currname{hhline}\input{hhline.sty}\makeatother
4056 \fi}%
4057 {}}}

```

`\substitutefontfamily` *Deprecated.* It creates an `fd` file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by $\TeX$ (`\DeclareFontFamilySubstitution`).

```

4058 \def\substitutefontfamily#1#2#3{%
4059 \lowercase{\immediate\openout15=#1#2.fd\relax}%
4060 \immediate\write15{%
4061 \string\ProvidesFile{#1#2.fd}%
4062 [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4063 \space generated font description file]^J
4064 \string\DeclareFontFamily{#1}{#2}{}}^J
4065 \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
4066 \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
4067 \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
4068 \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
4069 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
4070 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
4071 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
4072 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
4073 }%
4074 \closeout15
4075 }
4076 \@onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of `\TeX` and `\LaTeX` for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```
4077 \bbl@trace{Encoding and fonts}
4078 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4079 \newcommand\BabelNonText{TS1,T3,TS3}
4080 \let\org@TeX\TeX
4081 \let\org@LaTeX\LaTeX
4082 \let\ensureascii\@firstofone
4083 \let\asciienencoding\@empty
4084 \AtBeginDocument{%
4085   \def\@elt#1{,#1,}%
4086   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4087   \let\@elt\relax
4088   \let\bbl@tempb\@empty
4089   \def\bbl@tempc{OT1}%
4090   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4091     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4092   \bbl@foreach\bbl@tempa{%
4093     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4094     \ifin@
4095       \def\bbl@tempb{#1}% Store last non-ascii
4096     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4097       \ifin@else
4098         \def\bbl@tempc{#1}% Store last ascii
4099       \fi
4100     \fi}%
4101   \ifx\bbl@tempb\@empty\else
4102     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4103     \ifin@else
4104       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4105     \fi
4106     \let\asciienencoding\bbl@tempc
4107     \renewcommand\ensureascii[1]{%
4108       {\fontencoding{\asciienencoding}\selectfont#1}}%
4109     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4110     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4111   \fi}
```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than 'latin' (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```
4112 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}
```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\@ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```
4113 \AtBeginDocument{%
4114   \@ifpackageloaded{fontspec}%
4115   {\xdef\latinencoding{%
4116     \ifx\UTFencname\undefined
4117       EU\ifcase\bbl@engine\or2\or1\fi
4118     \else
4119       \UTFencname
4120     \fi}}%
4121   {\gdef\latinencoding{OT1}%
4122     \ifx\cf@encoding\bbl@t@one
4123       \xdef\latinencoding{\bbl@t@one}%
4124     \else
4125       \def\@elt#1{,#1,}%
4126       \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%

```

```

4127      \let\@elt\relax
4128      \bbl@xin@{,Tl,}\bbl@tempa
4129      \ifin@
4130      \xdef\latinencoding{\bbl@t@one}%
4131      \fi
4132      \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4133 \DeclareRobustCommand{\latintext}{%
4134   \fontencoding{\latinencoding}\selectfont
4135   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4136 \ifx\@undefined\DeclareTextFontCommand
4137   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4138 \else
4139   \DeclareTextFontCommand{\textlatin}{\latintext}
4140 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\text{\LaTeX}$ 2021-06-01, there is a hook for this purpose.

```

4141 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at ARABI (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdfTeX` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\text{\TeX}$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\text{\TeX}$ -ja` shows, vertical typesetting is possible, too.

```

4142 \bbl@trace{Loading basic (internal) bidi support}
4143 \ifodd\bbl@engine
4144 \else % Any xe+lua bidi
4145   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4146     \bbl@error{bidi-only-lua}{\}\}\}\}%
4147     \let\bbl@beforeforeign\leavevmode
4148     \AtEndOfPackage{%
4149       \EnableBabelHook{babel-bidi}%
4150       \bbl@xebidipar}
4151   \fi\fi
4152   \def\bbl@loadxebidi#1{%
4153     \ifx\RTLfootnotetext\@undefined
4154       \AtEndOfPackage{%
4155         \EnableBabelHook{babel-bidi}%

```

```

4156 \ifx\fontspec\undefined
4157 \usepackage{fontspec}% bidi needs fontspec
4158 \fi
4159 \usepackage#1{bidi}%
4160 \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4161 \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4162 \ifnum\@nameuse{bbl@wdir\language}\tw@ % 'AL' bidi
4163 \bbl@digitsdotdash % So ignore in 'R' bidi
4164 \fi}%
4165 \fi}
4166 \ifnum\bbl@bidimode>200 % Any xe bidi=
4167 \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4168 \bbl@tentative{bidi=bidi}
4169 \bbl@loadxebidi{}
4170 \or
4171 \bbl@loadxebidi{[rldocument]}
4172 \or
4173 \bbl@loadxebidi{}
4174 \fi
4175 \fi
4176 \fi
4177 \ifnum\bbl@bidimode=\@ne % bidi=default
4178 \let\bbl@beforeforeign\leavevmode
4179 \ifodd\bbl@engine % lua
4180 \newattribute\bbl@attr@dir
4181 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4182 \bbl@exp{\output{\bodydir\pagedir\the\output}}
4183 \fi
4184 \AtEndOfPackage{%
4185 \EnableBabelHook{babel-bidi}% pdf/lua/xe
4186 \ifodd\bbl@engine\else % pdf/xe
4187 \bbl@xebidipar
4188 \fi}
4189 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4190 \bbl@trace{Macros to switch the text direction}
4191 \def\bbl@alscripts{%
4192 ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4193 \def\bbl@rscripts{%
4194 Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4195 Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4196 Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4197 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4198 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4199 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4200 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4201 Meroitic,N'Ko,Orkhon,Todhri}
4202 %
4203 \def\bbl@provide@dirs#1{%
4204 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4205 \ifin@
4206 \global\bbl@csarg\chardef{wdir@#1}\@ne
4207 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4208 \ifin@
4209 \global\bbl@csarg\chardef{wdir@#1}\tw@
4210 \fi
4211 \else
4212 \global\bbl@csarg\chardef{wdir@#1}\z@
4213 \fi
4214 \ifodd\bbl@engine

```

```

4215 \bbl@csarg\ifcase{wdir@#1}%
4216 \directlua{ Babel.locale_props[\the\localeid].textdir = 'l' }%
4217 \or
4218 \directlua{ Babel.locale_props[\the\localeid].textdir = 'r' }%
4219 \or
4220 \directlua{ Babel.locale_props[\the\localeid].textdir = 'al' }%
4221 \fi
4222 \fi}
4223 %
4224 \def\bbl@switchdir{%
4225 \bbl@ifunset{\bbl@sys{\languagename}}{\bbl@provide@sys{\languagename}}{}%
4226 \bbl@ifunset{\bbl@wdir{\languagename}}{\bbl@provide@dirs{\languagename}}{}%
4227 \bbl@exp{\bbl@setdirs\bbl@cl{wdir}}}%
4228 \def\bbl@setdirs#1{%
4229 \ifcase\bbl@select@type
4230 \bbl@bodydir{#1}%
4231 \bbl@pardir{#1}% <- Must precede \bbl@textdir
4232 \fi
4233 \bbl@textdir{#1}}
4234 \ifnum\bbl@bidimode>\z@
4235 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4236 \DisableBabelHook{babel-bidi}
4237 \fi

```

Now the engine-dependent macros.

```

4238 \ifodd\bbl@engine % luatex=1
4239 \else % pdftex=0, xetex=2
4240 \newcount\bbl@dirlevel
4241 \chardef\bbl@thetexdir\z@
4242 \chardef\bbl@thepardir\z@
4243 \def\bbl@textdir#1{%
4244 \ifcase#1\relax
4245 \chardef\bbl@thetexdir\z@
4246 \@nameuse{setlatin}%
4247 \bbl@textdir@i\beginL\endL
4248 \else
4249 \chardef\bbl@thetexdir@ne
4250 \@nameuse{setnonlatin}%
4251 \bbl@textdir@i\beginR\endR
4252 \fi}
4253 \def\bbl@textdir@i#1#2{%
4254 \ifhmode
4255 \ifnum\currentgrouplevel>\z@
4256 \ifnum\currentgrouplevel=\bbl@dirlevel
4257 \bbl@error{multiple-bidi}{}{}%
4258 \bgroup\aftergroup#2\aftergroup\egroup
4259 \else
4260 \ifcase\currentgroup\or % 0 bottom
4261 \aftergroup#2% 1 simple {}
4262 \or
4263 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4264 \or
4265 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4266 \or\or\or % vbox vtop align
4267 \or
4268 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4269 \or\or\or\or\or\or % output math disc insert vcent mathchoice
4270 \or
4271 \aftergroup#2% 14 \begingroup
4272 \else
4273 \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4274 \fi
4275 \fi

```

```

4276      \bbl@dirlevel\currentgrouplevel
4277      \fi
4278      #1%
4279      \fi}
4280      \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4281      \let\bbl@bodydir\@gobble
4282      \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4283      \def\bbl@xebidipar{%
4284      \let\bbl@xebidipar\relax
4285      \TeXeTstate\@ne
4286      \def\bbl@xeeverypar{%
4287      \ifcase\bbl@thepardir
4288      \ifcase\bbl@thetextdir\else\beginR\fi
4289      \else
4290      {\setbox\z@\lastbox\beginR\box\z@}%
4291      \fi}%
4292      \AddToHook{para/begin}{\bbl@xeeverypar}}
4293      \ifnum\bbl@bidimode>200 % Any xe bidi=
4294      \let\bbl@textdir\i\@gobbletwo
4295      \let\bbl@xebidipar\@empty
4296      \AddBabelHook{bidi}{foreign}{%
4297      \ifcase\bbl@thetextdir
4298      \BabelWrapText{\LR{##1}}}%
4299      \else
4300      \BabelWrapText{\RL{##1}}}%
4301      \fi}
4302      \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4303      \fi
4304      \fi

```

A tool for weak L (mainly digits). We also disable warnings with `hyperref`. There is a variant for `bidi=unibidi`, but using formatting characters is discouraged by Unicode UAX9, so it must be revised.

```

4305      \ifnum\bbl@bidimode>300
4306      \DeclareRobustCommand\babelsublr[1]{\leavevmode{\^^^200f#1}}
4307      \else
4308      \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4309      \fi
4310      \AtBeginDocument{%
4311      \ifx\pdfstringdefDisableCommands\@undefined\else
4312      \ifx\pdfstringdefDisableCommands\relax\else
4313      \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4314      \fi
4315      \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4316      \bbl@trace{Local Language Configuration}
4317      \ifx\loadlocalcfg\@undefined
4318      \ifpackagewith{babel}{noconfigs}%
4319      {\let\loadlocalcfg\@gobble}%
4320      {\def\loadlocalcfg#1{%
4321      \InputIfFileExists{#1.cfg}%

```

```

4322      {\typeout{*****^J%
4323              * Local config file #1.cfg used^^J%
4324              *}}%
4325      \@empty}}
4326 \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```

4327 \bbl@trace{Language options}
4328 \def\BabelDefinitionFile#1#2#3{
4329 \let\bbl@afterlang\relax
4330 \let\BabelModifiers\relax
4331 \let\bbl@loaded\@empty
4332 \def\bbl@load@language#1{%
4333   \InputIfFileExists{#1.ldf}%
4334   {\edef\bbl@loaded{\CurrentOption
4335     \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4336     \expandafter\let\expandafter\bbl@afterlang
4337     \csname\CurrentOption.ldf-h@k\endcsname
4338     \expandafter\let\expandafter\BabelModifiers
4339     \csname bbl@mod@\CurrentOption\endcsname
4340     \bbl@exp{\AtBeginDocument{%
4341       \bbl@usehooks@lang{\CurrentOption}{\begindocument}{\CurrentOption}}}%
4342     {\bbl@error{unknown-package-option}}{}}}%

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language as been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetada we also force it with `\foreignlanguage` (this is also done in bidi texts).

```

4343 \ifx\GetDocumentProperties\undefined\else
4344   \let\bbl@beforeforeign\leavevmode
4345   \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4346   \let\bbl@collect@other\@empty
4347   \bbl@foreach\bbl@tempa{%
4348     \edef\bbl@metalang{#1}%
4349     \ifx\bbl@metalang\@empty\else
4350       \begingroup
4351         \expandafter
4352         \bbl@bcplookup{\bbl@metalang}%
4353         \ifx\bbl@bcp\relax
4354           \ifx\bbl@opt@main\@nnil
4355             \bbl@error{no-locale-for-meta}{\bbl@metalang}{}%
4356           \fi
4357         \else
4358           \bbl@read@ini{\bbl@bcp}\m@ne
4359           \xdef\bbl@collect@other{\bbl@collect@other,\language\name}%
4360           \GenericInfo{(babel) \spaces\@spaces\@spaces
4361             Passing '\language\name' to babel\@gobble}%
4362         \fi
4363       \endgroup
4364     \fi}
4365   \ifx\bbl@collect@other\@empty\else
4366     \xdef\bbl@language@opts{\bbl@collect@other,\bbl@language@opts}%
4367   \fi

```

```

4368 \edef\bbl@metalang{\GetDocumentProperties{document/language}}%
4369 \ifx\bbl@metalang\empty\else
4370   \begingroup
4371     \expandafter
4372     \bbl@bcplookup{\bbl@metalang}%
4373     \ifx\bbl@bcp\relax
4374       \ifx\bbl@opt@main\@nnil
4375         \bbl@error{no-locale-for-meta}{\bbl@metalang}{}{}%
4376       \fi
4377     \else
4378       \bbl@read@ini{\bbl@bcp}\m@ne
4379       \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4380       \ifx\bbl@opt@main\@nnil
4381         \global\let\bbl@opt@main\language
4382       \fi
4383       \GenericInfo{}{(babel) \@spaces\@spaces\@spaces
4384         Passing '\language' to babel\@gobble}%
4385     \fi
4386   \endgroup
4387 \fi
4388 \fi
4389 \ifx\bbl@opt@config\@nnil
4390   \@ifpackagewith{babel}{noconfigs}{}%
4391   {\InputIfFileExists{bblopts.cfg}%
4392     {\bbl@info{Configuration files are deprecated, as\\%
4393       they can break document portability.\\%
4394       Reported}%
4395     \typeout{*****^J%
4396       * Local config file bblopts.cfg used^^J%
4397       *}%
4398     }{}%
4399 \else
4400   \InputIfFileExists{\bbl@opt@config.cfg}%
4401   {\bbl@info{Configuration files are deprecated, as\\%
4402     they can break document portability.\\%
4403     Reported}%
4404   \typeout{*****^J%
4405     * Local config file \bbl@opt@config.cfg used^^J%
4406     *}%
4407   {\bbl@error{config-not-found}{}{}{}%
4408 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may content a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `@import`).

```

4409 \def\BabelBeforeIni#1#2{%
4410   \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4411   \let\bbl@tempb\empty
4412   #2%
4413   \edef\bbl@tload{%
4414     \ifx\bbl@tload\empty\else\bbl@tload,\fi
4415     \bbl@tload@last}%
4416   \edef\bbl@tload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}

```

```

4417 \def\BabelDefinitionFile#1#2#3{%
4418   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4419   \namedef\bbl@preldf@{\CurrentOption}{#3}%
4420   \endinput}%

```

For efficiency, first preprocess the class options to remove those with =, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\text{\LaTeX}$ 3 layer.

```

4421 \def\bbl@tempf{,}
4422 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4423   \in@{=}{#1}%
4424   \ifin@ \else
4425     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4426   \fi}

```

Store the class/package options in a list. If there is an explicit main, it's placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4427 \let\bbl@toload\@empty
4428 \let\bbl@toload@last\@empty
4429 \let\bbl@unkopt\@gobble %% <- Ugly
4430 \edef\bbl@tempc{%
4431   \bbl@tempf,@,\bbl@language@opts
4432   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4433 \let\BabelLocalesTentative\bbl@tempc
4434 %
4435 \bbl@foreach\bbl@tempc{%
4436   \in@{@@}{#1}% <- Ugly
4437   \ifin@
4438     \def\bbl@unkopt##1{%
4439       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}{}}}%
4440   \else
4441     \def\CurrentOption{#1}%
4442     \bbl@xin@{//#1//}{\bbl@toload@last}% Collapse consecutive
4443     \ifin@ \else
4444       \lowercase{\InputIfFileExists{babel-#1.tex}}{ }{%
4445         \IfFileExists{#1.ldf}%
4446         {\edef\bbl@toload{%
4447           \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4448           \bbl@toload@last}%
4449           \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4450         {\bbl@unkopt{#1}}}%
4451     \fi
4452   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4453 \ifx\bbl@opt@main\@nnil
4454   \ifx\bbl@toload@last\@empty
4455     \def\bbl@toload@last{0/0//nil//und-x-nil-nil}
4456     \bbl@info{%
4457       You haven't specified a language as a class or package\\%
4458       option. I'll load 'nil'. Reported}
4459   \fi
4460 \else
4461   \let\bbl@tempc\@empty
4462   \bbl@foreach\bbl@toload{%
4463     \bbl@xin@{//#1//\bbl@opt@main//}{#1}%
4464     \ifin@ \else
4465       \bbl@add@list\bbl@tempc{#1}%
4466     \fi}

```

```

4467 \let\bbl@toload\bbl@tempc
4468 \fi
4469 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4470 \def\AfterBabelLanguage#1{%
4471 \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang}{}}
4472 \NewHook{babel/presets}
4473 \UseHook{babel/presets}
4474 %
4475 \let\bbl@tempb\@empty
4476 \def\bbl@tempc#1/#2//#3//#4/#5\@@{%
4477 \count@\z@
4478 \ifnum#2=\@m % if no \BabelDefinitionFile
4479 \ifnum#1=\z@ % not main. -- % if provide+=!, provide*=!
4480 \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@@
4481 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4482 \fi
4483 \else % 10 = main -- % if provide+=!, provide*=!
4484 \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@@
4485 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4486 \fi
4487 \fi
4488 \else
4489 \ifnum#1=\z@ % not main
4490 \ifnum\bbl@iniflag>\@ne\else % if ø, provide
4491 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4492 \fi
4493 \else % 10 = main
4494 \ifodd\bbl@iniflag\else % if provide+, provide*
4495 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4496 \fi
4497 \fi
4498 \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4499 \fi}

```

Based on the value of \count@, do the actual loading. If ‘ldf’, we load the basic info from the ‘ini’ file before.

```

4500 \def\bbl@tempd#1#2#3#4#5{%
4501 \DeclareOption{#3}{}%
4502 \ifcase\count@
4503 \bbl@exp{\bbl@add\bbl@tempb{%
4504 \global\let\bbl@afterload\@empty
4505 \\\@nameuse{bbl@preini@#3}%
4506 \\\bbl@ldfinit
4507 \def\\CurrentOption{#3}%
4508 \\\babelprovide[import=#4,\ifnum#1=\z@\else\bbl@opt@provide,main\fi]{#3}%
4509 \\\bbl@afterldf
4510 \\\bbl@afterload}}%
4511 \else
4512 \bbl@add\bbl@tempb{%
4513 \global\let\bbl@afterload\@empty
4514 \def\CurrentOption{#3}%
4515 \let\localename\CurrentOption
4516 \let\languagename\localename
4517 \def\BabelIniTag{#4}%
4518 \@nameuse{bbl@preldf@#3}%
4519 \begingroup
4520 \bbl@id@assign
4521 \bbl@read@ini{\BabelIniTag}0%
4522 \endgroup

```

```

4523 \bbl@load@language{#5}%
4524 \bbl@afterload}%
4525 \fi}
4526 %
4527 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4528 \bbl@tempb
4529 \DeclareOption*{}
4530 \ProcessOptions
4531 %
4532 \bbl@exp{%
4533 \\\AtBeginDocument{\\\bbl@usehooks@lang{/}{\begindocument}{}}}%
4534 \def\AfterBabelLanguage{\bbl@error{late-after-babel}}{}{}
4535 \AddToHook{begindocument/end}{%
4536 \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4537 (/package)

```

6. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4538 (*kernel)
4539 \let\bbl@onlyswitch\@empty
4540 \input babel.def
4541 \let\bbl@onlyswitch\@undefined
4542 (/kernel)

```

7. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^`, `M`, `%` and `=` are reset before loading the file.

```

4543 (*errors)
4544 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4545 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4546 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4547 \catcode`\@=11 \catcode`\^=7
4548 %
4549 \ifx\MessageBreak\@undefined
4550 \gdef\bbl@error@i#1#2{%
4551 \begingroup
4552 \newlinechar=^^J
4553 \def\{^^J(babel) }%
4554 \errhelp{#2}\errmessage{\{#1}%
4555 \endgroup}
4556 \else
4557 \gdef\bbl@error@i#1#2{%
4558 \begingroup
4559 \def\{\MessageBreak}%
4560 \PackageError{babel}{#1}{#2}%
4561 \endgroup}
4562 \fi

```

```

4563 \def\bbl@errmessage#1#2#3{%
4564   \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4565     \bbl@error@i{#2}{#3}}}%
4566 % Implicit #2#3#4:
4567 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4568 %
4569 \bbl@errmessage{not-yet-available}
4570   {Not yet available}%
4571   {Find an armchair, sit down and wait}
4572 \bbl@errmessage{bad-package-option}%
4573   {Bad option '#1=#2'. Either you have misspelled the\\%
4574     key or there is a previous setting of '#1'. Valid\\%
4575     keys are, among others, 'shorthands', 'main', 'bidi',\\%
4576     'strings', 'config', 'headfoot', 'safe', 'math'.}%
4577   {See the manual for further details.}
4578 \bbl@errmessage{base-on-the-fly}
4579   {For a language to be defined on the fly 'base'\\%
4580     is not enough, and the whole package must be\\%
4581     loaded. Either delete the 'base' option or\\%
4582     request the languages explicitly}%
4583   {See the manual for further details.}
4584 \bbl@errmessage{undefined-language}
4585   {You haven't defined the language '#1' yet.\\%
4586     Perhaps you misspelled it or your installation\\%
4587     is not complete}%
4588   {Your command will be ignored, type <return> to proceed}
4589 \bbl@errmessage{invalid-ini-name}
4590   {'#1' not valid with the 'ini' mechanism.\\%
4591     I think you want '#2' instead. You may continue,\\%
4592     but you should fix the name. See the babel manual\\%
4593     for the available locales with 'provide'}%
4594   {See the manual for further details.}
4595 \bbl@errmessage{shorthand-is-off}
4596   {I can't declare a shorthand turned off (\string#2)}
4597   {Sorry, but you can't use shorthands which have been\\%
4598     turned off in the package options}
4599 \bbl@errmessage{not-a-shorthand}
4600   {The character '\string #1' should be made a shorthand character;\\%
4601     add the command \string\usesshorthands\string{#1\string} to
4602     the preamble.\\%
4603     I will ignore your instruction}%
4604   {You may proceed, but expect unexpected results}
4605 \bbl@errmessage{not-a-shorthand-b}
4606   {I can't switch '\string#2' on or off--not a shorthand\\%
4607     This character is not a shorthand. Maybe you made\\%
4608     a typing mistake?}%
4609   {I will ignore your instruction.}
4610 \bbl@errmessage{unknown-attribute}
4611   {The attribute #2 is unknown for language #1.}%
4612   {Your command will be ignored, type <return> to proceed}
4613 \bbl@errmessage{missing-group}
4614   {Missing group for string \string#1}%
4615   {You must assign strings to some category, typically\\%
4616     captions or extras, but you set none}
4617 \bbl@errmessage{only-lua-xe}
4618   {This macro is available only in LuaLaTeX and XeLaTeX.}%
4619   {Consider switching to these engines.}
4620 \bbl@errmessage{only-lua}
4621   {This macro is available only in LuaLaTeX}%
4622   {Consider switching to that engine.}
4623 \bbl@errmessage{unknown-provide-key}
4624   {Unknown key '#1' in \string\babelprovide}%
4625   {See the manual for valid keys}%

```

```

4626 \bbl@errmessage{unknown-mapfont}
4627 {Option '\bbl@KVP@mapfont' unknown for\\%
4628   mapfont. Use 'direction'}%
4629 {See the manual for details.}
4630 \bbl@errmessage{no-ini-file}
4631 {There is no ini file for the requested language\\%
4632   (#1: \language). Perhaps you misspelled it or your\\%
4633   installation is not complete}%
4634 {Fix the name or reinstall babel.}
4635 \bbl@errmessage{digits-is-reserved}
4636 {The counter name 'digits' is reserved for mapping\\%
4637   decimal digits}%
4638 {Use another name.}
4639 \bbl@errmessage{limit-two-digits}
4640 {Currently two-digit years are restricted to the\\%
4641   range 0-9999}%
4642 {There is little you can do. Sorry.}
4643 \bbl@errmessage{counter-too-large}
4644 {Counter too large (#1)}%
4645 {Currently this is the limit.}
4646 \bbl@errmessage{no-ini-info}
4647 {I've found no info for the current locale.\\%
4648   The corresponding ini file has not been loaded\\%
4649   Perhaps it doesn't exist}%
4650 {See the manual for details.}
4651 \bbl@errmessage{unknown-ini-field}
4652 {Unknown field '#1' in \string\BCPdata.\\%
4653   Perhaps you misspelled it}%
4654 {See the manual for details.}
4655 \bbl@errmessage{unknown-locale-key}
4656 {Unknown key for locale '#2':\\%
4657   #3\\%
4658   \string#1 will be set to \string\relax}%
4659 {Perhaps you misspelled it.}%
4660 \bbl@errmessage{adjust-only-vertical}
4661 {Currently, #1 related features can be adjusted only\\%
4662   in the main vertical list}%
4663 {Maybe things change in the future, but this is what it is.}
4664 \bbl@errmessage{layout-only-vertical}
4665 {Currently, layout related features can be adjusted only\\%
4666   in vertical mode}%
4667 {Maybe things change in the future, but this is what it is.}
4668 \bbl@errmessage{bidi-only-lua}
4669 {The bidi method 'basic' is available only in\\%
4670   luatex. I'll continue with 'bidi=default', so\\%
4671   expect wrong results.\\%
4672   Suggested actions:\\%
4673   * If possible, switch to luatex, as xetex is not\\%
4674     recommend anymore.\\
4675   * If you can't, try 'bidi=bidi' with xetex.\\%
4676   * With pdftex, only 'bidi=default' is available.}%
4677 {See the manual for further details.}
4678 \bbl@errmessage{multiple-bidi}
4679 {Multiple bidi settings inside a group\\%
4680   I'll insert a new group, but expect wrong results.\\%
4681   Suggested action:\\%
4682   * Add a new group where appropriate.}
4683 {See the manual for further details.}
4684 \bbl@errmessage{unknown-package-option}
4685 {Unknown option '\CurrentOption'.\\%
4686   Suggested actions:\\%
4687   * Make sure you haven't misspelled it\\%
4688   * Check in the babel manual that it's supported\\%

```

```

4689 * If supported and it's a language, you may\\%
4690 \space\space need in some distributions a separate\\%
4691 \space\space installation\\%
4692 * If installed, check there isn't an old\\%
4693 \space\space version of the required files in your system\\%
4694 * If it's an unsupported language, create it with\\%
4695 \string\babelprovide (see the manual)}
4696 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4697 activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4698 headfoot=, strings=, config=, hyphenmap=, or a language name.}
4699 \bbl@errmessage{config-not-found}
4700 {Local config file '\bbl@opt@config.cfg' not found.\\%
4701 Suggested actions:\\%
4702 * Make sure you haven't misspelled it in config=\\%
4703 * Check it exists and it's in the correct path}%
4704 {Perhaps you misspelled it.}
4705 \bbl@errmessage{late-after-babel}
4706 {Too late for \string\AfterBabelLanguage}%
4707 {Languages have been loaded, so I can do nothing}
4708 \bbl@errmessage{double-hyphens-class}
4709 {Double hyphens aren't allowed in \string\babelcharclass\\%
4710 because it's potentially ambiguous}%
4711 {See the manual for further info}
4712 \bbl@errmessage{unknown-interchar}
4713 {'#1' for '\language' cannot be enabled.\\%
4714 Maybe there is a typo}%
4715 {See the manual for further details.}
4716 \bbl@errmessage{unknown-interchar-b}
4717 {'#1' for '\language' cannot be disabled.\\%
4718 Maybe there is a typo}%
4719 {See the manual for further details.}
4720 \bbl@errmessage{charproperty-only-vertical}
4721 {\string\babelcharproperty\space can be used only in\\%
4722 vertical mode (preamble or between paragraphs)}%
4723 {See the manual for further info}
4724 \bbl@errmessage{unknown-char-property}
4725 {No property named '#2'. Allowed values are\\%
4726 direction (bc), mirror (bmg), and linebreak (lb)}%
4727 {See the manual for further info}
4728 \bbl@errmessage{bad-transform-option}
4729 {Bad option '#1' in a transform.\\%
4730 I'll ignore it but expect more errors}%
4731 {See the manual for further info.}
4732 \bbl@errmessage{font-conflict-transforms}
4733 {Transforms cannot be re-assigned to different\\%
4734 fonts. The conflict is in '\bbl@kv@label'.\\%
4735 Apply the same fonts or use a different label}%
4736 {See the manual for further details.}
4737 \bbl@errmessage{transform-not-available}
4738 {'#1' for '\language' cannot be enabled.\\%
4739 Maybe there is a typo or it's a font-dependent transform}%
4740 {See the manual for further details.}
4741 \bbl@errmessage{transform-not-available-b}
4742 {'#1' for '\language' cannot be disabled.\\%
4743 Maybe there is a typo or it's a font-dependent transform}%
4744 {See the manual for further details.}
4745 \bbl@errmessage{year-out-range}
4746 {Year out of range.\\%
4747 The allowed range is #1}%
4748 {See the manual for further details.}
4749 \bbl@errmessage{only-pdftex-lang}
4750 {The '#1' ldf style doesn't work with #2,\\%
4751 but you can use the ini locale instead.\\%

```

```

4752 Try adding 'provide=*' to the option list. You may\\%
4753 also want to set 'bidi=' to some value}%
4754 {See the manual for further details.}
4755 \bbl@errmessage{hyphenmins-args}
4756 {\string\babelhyphenmins\ accepts either the optional\\%
4757 argument or the star, but not both at the same time}%
4758 {See the manual for further details.}
4759 \bbl@errmessage{no-locale-for-meta}
4760 {There isn't currently a locale for the 'lang' requested\\%
4761 in the PDF metadata ('#1'). To fix it, you can\\%
4762 set explicitly a similar language (using the same\\%
4763 script) with the key main= when loading babel. If you\\%
4764 continue, I'll fallback to the 'nil' language, with\\%
4765 tag 'und' and script 'Latn', but expect a bad font\\%
4766 rendering with other scripts. You may also need set\\%
4767 explicitly captions and date, too}%
4768 {See the manual for further details.}
4769 (/errors)
4770 (*patterns)

```

8. Loading hyphenation patterns

The following code is meant to be read by `iniTEX` because it should instruct `TEX` to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4771 <@Make sure ProvidesFile is defined@>
4772 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4773 \xdef\bbl@format{\jobname}
4774 \def\bbl@version{<@version@>}
4775 \def\bbl@date{<@date@>}
4776 \ifx\AtBeginDocument\@undefined
4777 \def\@empty{}
4778 \fi
4779 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```

4780 \def\process@line#1#2 #3 #4 {%
4781 \ifx=#1%
4782 \process@synonym{#2}%
4783 \else
4784 \process@language{#1#2}{#3}{#4}%
4785 \fi
4786 \ignorespaces}

```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```

4787 \toks@{}
4788 \def\bbl@languages{}

```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmin` parameters for the synonym.

```

4789 \def\process@synonym#1{%
4790 \ifnum\last@language=\m@ne
4791 \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4792 \else
4793 \expandafter\chardef\curname l@#1\endcsname\last@language

```

```

4794 \wlog{\string\l@#1=\string\language\the\last@language}%
4795 \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4796 \csname\language\hyphenmins\endcsname
4797 \let\bbl@elt\relax
4798 \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}{}}}%
4799 \fi}

```

\process@language The macro `\process@language` is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call `\addlanguage` to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file `language.dat` by adding for instance ‘:T1’ to the name of the language. The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to `\lefthyphenmin` and `\righthyphenmin`. $\TeX$ does not keep track of these assignments. Therefore we try to detect such assignments and store them in the `\(language)hyphenmins` macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the `\lccode` or `\uccode` arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the `\patterns` command acts globally so its effect will be remembered.

Then we globally store the settings of `\lefthyphenmin` and `\righthyphenmin` and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

`\bbl@languages` saves a snapshot of the loaded languages in the form `\bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}`. Note the last 2 arguments are in ‘dialects’ defined in `language.dat` with `=`. Note also the language name can have encoding info.

Finally, if the counter `\language` is equal to zero we execute the synonyms stored.

```

4800 \def\process@language#1#2#3{%
4801 \expandafter\addlanguage\csname l@#1\endcsname
4802 \expandafter\language\csname l@#1\endcsname
4803 \edef\language{#1}%
4804 \bbl@hook@everylanguage{#1}%
4805 % > luatex
4806 \bbl@get@enc#1:.\@@@
4807 \begingroup
4808 \lefthyphenmin\m@ne
4809 \bbl@hook@loadpatterns{#2}%
4810 % > luatex
4811 \ifnum\lefthyphenmin=\m@ne
4812 \else
4813 \expandafter\xdef\csname #1hyphenmins\endcsname{%
4814 \the\lefthyphenmin\the\righthyphenmin}%
4815 \fi
4816 \endgroup
4817 \def\bbl@tempa{#3}%
4818 \ifx\bbl@tempa\@empty\else
4819 \bbl@hook@loadexceptions{#3}%
4820 % > luatex
4821 \fi
4822 \let\bbl@elt\relax
4823 \edef\bbl@languages{%
4824 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4825 \ifnum\the\language=\z@
4826 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4827 \set@hyphenmins\tw@\thr@@\relax

```

```

4828 \else
4829 \expandafter\expandafter\expandafter\set@hyphenmins
4830 \csname #lhyphenmins\endcsname
4831 \fi
4832 \the\toks@
4833 \toks@{}%
4834 \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```

4835 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4836 \def\bbl@hook@everylanguage#1{}
4837 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4838 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4839 \def\bbl@hook@loadkernel#1{%
4840 \def\addlanguage{\csname newlanguage\endcsname}%
4841 \def\adddialect##1##2{%
4842 \global\chardef##1##2\relax
4843 \wlog{\string##1 = a dialect from \string\language##2}}%
4844 \def\iflanguage##1{%
4845 \expandafter\ifx\csname l@##1\endcsname\relax
4846 \nolanner{##1}%
4847 \else
4848 \ifnum\csname l@##1\endcsname=\language
4849 \expandafter\expandafter\expandafter\@firstoftwo
4850 \else
4851 \expandafter\expandafter\expandafter\@secondoftwo
4852 \fi
4853 \fi}%
4854 \def\providehyphenmins##1##2{%
4855 \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4856 \@namedef{##1hyphenmins}{##2}%
4857 \fi}%
4858 \def\set@hyphenmins##1##2{%
4859 \lefthyphenmin##1\relax
4860 \righthyphenmin##2\relax}%
4861 \def\selectlanguage{%
4862 \errhelp{Selecting a language requires a package supporting it}%
4863 \errmessage{No multilingual package has been loaded}}%
4864 \let\foreignlanguage\selectlanguage
4865 \let\otherlanguage\selectlanguage
4866 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4867 \def\bbl@usehooks##1##2{%
4868 \def\setlocale{%
4869 \errhelp{Find an armchair, sit down and wait}%
4870 \errmessage{(babel) Not yet available}}%
4871 \let\uselocale\setlocale
4872 \let\locale\setlocale
4873 \let\selectlocale\setlocale
4874 \let\localename\setlocale
4875 \let\textlocale\setlocale
4876 \let\textlanguage\setlocale
4877 \let\languagetext\setlocale}
4878 \begingroup
4879 \def\AddBabelHook#1##2{%
4880 \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4881 \def\next{\toks1}%
4882 \else

```

```

4883     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname####1}%
4884     \fi
4885     \next}
4886     \ifx\directlua@undefined
4887     \ifx\XeTeXinputencoding\@undefined\else
4888         \input xebabel.def
4889     \fi
4890 \else
4891     \input luababel.def
4892 \fi
4893 \openin1 = babel-\bbl@format.cfg
4894 \ifeof1
4895 \else
4896     \input babel-\bbl@format.cfg\relax
4897 \fi
4898 \closein1
4899 \endgroup
4900 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4901 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4902 \def\language{english}%
4903 \ifeof1
4904 \message{I couldn't find the file language.dat,\space
4905         I will try the file hyphen.tex}
4906 \input hyphen.tex\relax
4907 \chardef\l@english\z@
4908 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```

4909 \last@language@m@ne

```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```

4910 \loop
4911 \endlinechar@m@ne
4912 \read1 to \bbl@line
4913 \endlinechar`\^M

```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```

4914 \if T\ifeof1F\fi T\relax
4915 \ifx\bbl@line\@empty\else
4916     \edef\bbl@line{\bbl@line\space\space\space}%
4917     \expandafter\process@line\bbl@line\relax
4918 \fi
4919 \repeat

```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```

4920 \begingroup
4921 \def\bbl@elt#1#2#3#4{%
4922     \global\language=#2\relax
4923     \gdef\language{#1}%
4924     \def\bbl@elt##1##2##3##4{}}%

```

```

4925 \bbl@languages
4926 \endgroup
4927 \fi
4928 \closein1

```

We add a message about the fact that babel is loaded in the format and with which language patterns to the `\everyjob` register.

```

4929 \if/\the\toks@/\else
4930 \errhelp{language.dat loads no language, only synonyms}
4931 \errmessage{Orphan language synonym}
4932 \fi

```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```

4933 \let\bbl@line\@undefined
4934 \let\process@line\@undefined
4935 \let\process@synonym\@undefined
4936 \let\process@language\@undefined
4937 \let\bbl@get@enc\@undefined
4938 \let\bbl@hyph@enc\@undefined
4939 \let\bbl@tempa\@undefined
4940 \let\bbl@hook@loadkernel\@undefined
4941 \let\bbl@hook@everylanguage\@undefined
4942 \let\bbl@hook@loadpatterns\@undefined
4943 \let\bbl@hook@loadexceptions\@undefined
4944 (/patterns)

```

Here the code for `iniTeX` ends.

9. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdftex`).

```

4945 ((*More package options)) ≡
4946 \chardef\bbl@bidimode\z@
4947 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4948 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4949 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4950 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4951 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4952 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4953 \DeclareOption{bidi=unibidi}{\chardef\bbl@bidimode=301 }
4954 (/More package options))

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `\bbl@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```

4955 ((*Font selection)) ≡
4956 \bbl@trace{Font handling with fontspec}
4957 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4958 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@ckeckstdfonts}
4959 \DisableBabelHook{babel-fontspec}
4960 @onlypreamble\babelfont
4961 \ifx\NewDocumentCommand\@undefined\else % Not plain
4962 \NewDocumentCommand\babelfont{0}{m0}{m0}}{%
4963 \bbl@bblfont@o[#1]{#2}{#3,#5}{#4}}
4964 \fi
4965 \newcommand\bbl@bblfont@o[2][]{% 1=langs/scripts 2=fam
4966 \ifx\fontspec\@undefined
4967 \usepackage{fontspec}%
4968 \fi

```

```

4969 \EnableBabelHook{babel-fontspec}%
4970 \edef\bbl@tempa{#1}%
4971 \def\bbl@tempb{#2}% Used by \bbl@bblfont
4972 \bbl@bblfont}
4973 \newcommand\bbl@bblfont[2][]{% 1=features 2=fontname, @font=rm|sf|tt
4974 \bbl@ifunset{\bbl@tempb family}%
4975 {\bbl@providfam{\bbl@tempb}}%
4976 {}%
4977 % For the default font, just in case:
4978 \bbl@ifunset{\bbl@sys@\language}\bbl@provide@sys{\language}}{%
4979 \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
4980 {\bbl@csarg\edef{\bbl@tempb dflt@}{<#1>{#2}}% save bbl@rmdflt@
4981 \bbl@exp{%
4982 \let<\bbl@tempb dflt@\language><\bbl@tempb dflt@>%
4983 \\\bbl@font@set<\bbl@tempb dflt@\language>%
4984 \<\bbl@tempb default>\<\bbl@tempb family>}}%
4985 {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
4986 \bbl@csarg\def{\bbl@tempb dflt@##1}{<#1>{#2}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4987 \def\bbl@providfam#1{%
4988 \bbl@exp{%
4989 \\\newcommand<#1default>{}% Just define it
4990 \\\bbl@add@list\\bbl@font@fams{#1}%
4991 \\\NewHook{#1family}%
4992 \\\DeclareRobustCommand<#1family>{%
4993 \\\not@math@alphabet<#1family>\relax
4994 % \\\prepare@family@series@update{#1}<#1default>% TODO. Fails
4995 \\\fontfamily<#1default>%
4996 \\\UseHook{#1family}%
4997 \\\selectfont}%
4998 \\\DeclareTextFontCommand{\<text#1>}{<#1family>}}}

```

The following macro is activated when the hook babel-fontspec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4999 \def\bbl@nostdfont#1{%
5000 \bbl@once{nostdfam-f@family}%
5001 {\bbl@infowarn{The current font is not a babel standard family:\\%
5002 #1%
5003 \fontname\font\\%
5004 There is nothing intrinsically wrong, and you can\\%,
5005 ignore this message altogether if you do not need\\%
5006 this font. If they are used in the document, be aware\\%
5007 'babel' will not set Script and Language for it, so\\%
5008 you may consider defining a new family with \string\babelfont.\\%
5009 See the manual for further details about \string\babelfont.
5010 Reported}}%
5011 {}}%
5012 \gdef\bbl@switchfont{%
5013 \bbl@ifunset{\bbl@sys@\language}\bbl@provide@sys{\language}}{%
5014 \bbl@exp{% e.g., Arabic -> arabic
5015 \lowercase{\edef\\bbl@tempa{\bbl@cl{sname}}}}%
5016 \bbl@foreach\bbl@font@fams{%
5017 \bbl@ifunset{\bbl@##1dflt@\language}% (1) language?
5018 {\bbl@ifunset{\bbl@##1dflt*~\bbl@tempa}% (2) from script?
5019 {\bbl@ifunset{\bbl@##1dflt@}% 2=F - (3) from generic?
5020 {}% 123=F - nothing!
5021 {\bbl@exp{% 3=T - from generic
5022 \global\let<\bbl@##1dflt@\language>%
5023 \<\bbl@##1dflt@>}}}%
5024 {\bbl@exp{% 2=T - from script
5025 \global\let<\bbl@##1dflt@\language>%
5026 \<\bbl@##1dflt*~\bbl@tempa>}}}%
5027 {}}% 1=T - language, already defined

```

```

5028 \def\bbl@tempa{\bbl@nostdfont{}}%
5029 \bbl@foreach\bbl@font@fams{%      don't gather with prev for
5030   \bbl@ifunset{\bbl@##1dflt@\language}%
5031   {\bbl@cs{famrst@##1}%
5032    \global\bbl@csarg\let{famrst@##1}\relax}%
5033   {\bbl@exp{% order is relevant.
5034    \\bbl@add\\originalTeX{%
5035     \\bbl@font@rst{\bbl@ccl{##1dflt}}%
5036     \<##1default>\<##1family>{##1}}%
5037    \\bbl@font@set\<bbl@##1dflt@\language>% the main part!
5038    \<##1default>\<##1family>}}}%
5039 \bbl@ifrestoring{ }\bbl@tempa}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with `\babelfont`.

```

5040 \ifx\fontfamily\undefined\else      % if latex
5041 \ifcase\bbl@engine                    % if pdftex
5042   \let\bbl@cckstdfont\relax
5043 \else
5044   \def\bbl@cckstdfont{%
5045     \begingroup
5046     \global\let\bbl@cckstdfont\relax
5047     \let\bbl@tempa\empty
5048     \bbl@foreach\bbl@font@fams{%
5049       \bbl@ifunset{\bbl@##1dflt@}%
5050       {\@nameuse{##1family}%
5051        \bbl@csarg\gdef{WFF@fontfamily}{ }% Flag
5052        \bbl@exp{\\bbl@add\\bbl@tempa{* \<##1family>= \fontfamily\\}%
5053         \space\space\fontname\font\\}%
5054        \bbl@csarg\xdef{##1dflt@}{fontfamily}%
5055        \expandafter\xdef\csname ##1default\endcsname{fontfamily}%
5056        { }}%
5057       \ifx\bbl@tempa\empty\else
5058         \bbl@info{The following font families will use the default\\%
5059          settings for all or some languages:\\%
5060          \bbl@tempa
5061          There is nothing intrinsically wrong with it, but\\%
5062          'babel' will no set Script and Language, which could\\%
5063          be relevant in some languages. If your document uses\\%
5064          these families, consider redefining them with \string\babelfont.\\%
5065          Reported}%
5066       \fi
5067     \endgroup}
5068 \fi
5069 \fi

```

Now the macros defining the font with `fontspec`.

When there are repeated keys in `fontspec`, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes `bx/sc` is the correct font, but sometimes points to `b/n`, even if `b/sc` exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5070 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
5071   \bbl@xin@{<>}{#1}%
5072   \ifin@
5073     \bbl@exp{\\bbl@fontspec@set\\#1\expandafter\@gobbletwo#1\\#3}%
5074   \fi
5075   \bbl@exp{%
5076     \def\\#2{#1}%      e.g., \rmdefault{\bbl@rmdflt@lang}
5077     \\bbl@ifsamestring{#2}{fontfamily}%

```

```

5078      {\#3%
5079      \\\bbl@ifsamestring{\f@series}{\bfdefault}{\bfseries}}}%
5080      \let\\bbl@tempa\relax}%
5081      {}}}

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5082 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5083 \let\bbl@tempe\bbl@mapselect
5084 \edef\bbl@tempb{\bbl@stripslash#4/% Catcodes hack (better pass it).
5085 \bbl@exp{\bbl@replace\\bbl@tempb{\bbl@stripslash\family/}}}%
5086 \let\bbl@mapselect\relax
5087 \let\bbl@tempfam#4% e.g., '\rmfamily', to be restored below
5088 \let#4\empty % Make sure \renewfontfamily is valid
5089 \bbl@set@renderer
5090 \bbl@exp{%
5091 \let\\bbl@temp@pfam\<\bbl@stripslash#4\space> e.g., '\rmfamily '
5092 \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}}%
5093 {\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5094 \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}}%
5095 {\bbl@cl{lname}}{\bbl@cl{lotf}}}%
5096 \\renewfontfamily\\#4%
5097 [\bbl@cl{lsys},% xetex removes unknown features :-(
5098 \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5099 #2]}{#3}% i.e., \bbl@exp{..}{#3}
5100 \bbl@unset@renderer
5101 \begingroup
5102 #4%
5103 \xdef#1{\f@family}% e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5104 \endgroup
5105 \bbl@xin@{\detokenize{>ssub*}}}%
5106 {\expandafter\meaning\curname TU/#1/bx/sc\endcsname}%
5107 \ifin@
5108 \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5109 \fi
5110 \bbl@xin@{\detokenize{>ssub*}}}%
5111 {\expandafter\meaning\curname TU/#1/bx/scit\endcsname}%
5112 \ifin@
5113 \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5114 \fi
5115 \let#4\bbl@tempfam
5116 \bbl@exp{\let\<\bbl@stripslash#4\space>}\bbl@temp@pfam
5117 \let\bbl@mapselect\bbl@tempe}%

```

`font@rst` and `famrst` are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5118 \def\bbl@font@rst#1#2#3#4{%
5119 \bbl@ccarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with `\babelfont`.

```

5120 \def\bbl@font@fams{rm,sf,tt}
5121 (</Font selection>)

```

10. Hooks for XeTeX and LuaTeX

10.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to `utf8`, which seems a sensible default.

Now, the code.

```
5122 (*xetex)
5123 \def\BabelStringsDefault{unicode}
5124 \let\xebbl@stop\relax
5125 \AddBabelHook{xetex}{encodedcommands}{%
5126   \def\bbl@tempa{#1}%
5127   \ifx\bbl@tempa\@empty
5128     \XeTeXinputencoding"bytes"%
5129   \else
5130     \XeTeXinputencoding"#1"%
5131   \fi
5132   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5133 \AddBabelHook{xetex}{stopcommands}{%
5134   \xebbl@stop
5135   \let\xebbl@stop\relax}
5136 \def\bbl@input@classes{% Used in CJK intraspaces
5137   \input{load-unicode-xetex-classes.tex}%
5138   \let\bbl@input@classes\relax}
5139 \def\bbl@intraspace#1 #2 #3\@{%
5140   \bbl@csarg\gdef{xeisp@\language}%
5141   {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5142 \def\bbl@intrapenalty#1\@{%
5143   \bbl@csarg\gdef{xeipn@\language}%
5144   {\XeTeXlinebreakpenalty #1\relax}}
5145 \def\bbl@provide@intraspace{%
5146   \bbl@xin@{/s}{\bbl@cl{lnbrk}}}%
5147   \ifin@else\bbl@xin@{/c}{\bbl@cl{lnbrk}}}\fi
5148   \ifin@
5149     \bbl@ifunset{bbl@intsp@\language}{}%
5150     {\expandafter\ifx\csname bbl@intsp@\language\endcsname\@empty\else
5151       \ifx\bbl@KVP@intraspace\@nnil
5152         \bbl@exp{%
5153           \\bbl@intraspace\bbl@cl{intsp}\\\@}%
5154         \fi
5155         \ifx\bbl@KVP@intrapenalty\@nnil
5156           \bbl@intrapenalty0\@
5157         \fi
5158         \fi
5159         \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5160           \expandafter\bbl@intraspace\bbl@KVP@intraspace\@
5161         \fi
5162         \ifx\bbl@KVP@intrapenalty\@nnil\else
5163           \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@
5164         \fi
5165         \bbl@exp{%
5166           \\bbl@add\<extras\language>{%
5167             \XeTeXlinebreaklocale "\bbl@cl{tbcpr}"%
5168             \<bbl@xeisp@\language>%
5169             \<bbl@xeipn@\language>%
5170             \\bbl@tglobal\<extras\language>%
5171             \\bbl@add\<noextras\language>{%
5172               \XeTeXlinebreaklocale ""}%
5173             \\bbl@tglobal\<noextras\language>%
5174             \ifx\bbl@ispacesize\undefined
5175               \gdef\bbl@ispacesize{\bbl@cl{xeisp}}%
5176             \ifx\AtBeginDocument\@notprerr
5177               \expandafter\@secondoftwo % to execute right now
5178             \fi
5179             \AtBeginDocument{\bbl@patchfont{\bbl@ispacesize}}%
5180           \fi}%
5181   \fi}
5182 \ifx\DisableBabelHook\undefined\endinput\fi
5183 \let\bbl@set@renderer\relax
```

```

5184 \let\bbl@unset@renderer\relax
5185 <@Font selection>
5186 \def\bbl@provide@extra#1{}

```

Hack for unhyphenated line breaking. See \bbl@provide@lsys in the common code.

```

5187 \def\bbl@xeno-hyph@d{%
5188   \bbl@ifset{\bbl@prehc@{\language name}}%
5189     {\ifnum\hyphenchar\font=\defaultthyphenchar
5190       \iffontchar\font\bbl@cl{prehc}\relax
5191       \hyphenchar\font\bbl@cl{prehc}\relax
5192       \else\iffontchar\font"200B
5193         \hyphenchar\font"200B
5194       \else
5195         \bbl@warning
5196           {Neither 0 nor ZERO WIDTH SPACE are available\\%
5197            in the current font, and therefore the hyphen\\%
5198            will be printed. Try changing the fontspec's\\%
5199            'HyphenChar' to another value, but be aware\\%
5200            this setting is not safe (see the manual).\\%
5201            Reported}%
5202         \hyphenchar\font\defaultthyphenchar
5203       \fi\fi
5204     \fi}%
5205   {\hyphenchar\font\defaultthyphenchar}}

```

10.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5206 \ifnum\xe@alloc@intercharclass<\thr@@
5207   \xe@alloc@intercharclass\thr@@
5208 \fi
5209 \chardef\bbl@xe@class@default@=\z@
5210 \chardef\bbl@xe@class@cjkideogram@=\@ne
5211 \chardef\bbl@xe@class@cjkleftpunctuation@=\tw@
5212 \chardef\bbl@xe@class@cjkrightpunctuation@=\thr@@
5213 \chardef\bbl@xe@class@boundary@=4095
5214 \chardef\bbl@xe@class@ignore@=4096

```

The machinery is activated with a hook (enabled only if actually used). Here \bbl@tempc is pre-set with \bbl@usingxe@class, defined below. The standard mechanism based on \originalTeX to save, set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```

5215 \AddBabelHook{babel-interchar}{beforeextras}{%
5216   \@nameuse{\bbl@xechars@{\language name}}
5217 \DisableBabelHook{babel-interchar}
5218 \protected\def\bbl@charclass#1{%
5219   \ifnum\count@<\z@
5220     \count@-\count@
5221   \loop
5222     \bbl@exp{%
5223       \\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5224       \XeTeXcharclass\count@ \bbl@tempc
5225       \ifnum\count@<`#1\relax
5226       \advance\count@\@ne
5227     \repeat
5228   \else
5229     \babel@savevariable{\XeTeXcharclass`#1}%
5230     \XeTeXcharclass`#1 \bbl@tempc
5231   \fi
5232   \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above

has internally the form `\bbl@usingxeclass\bbl@xeclass@punct@english\bbl@charclass{.}`
`\bbl@charclass{,}` (etc.), where `\bbl@usingxeclass` stores the class to be applied to the
subsequent characters. The `\ifcat` part deals with the alternative way to enter characters as macros
(e.g., `\`). As a special case, hyphens are stored as `\bbl@upto`, to deal with ranges.

```

5233 \newcommand\bbl@ifinterchar[1]{%
5234   \let\bbl@tempa\@gobble           % Assume to ignore
5235   \edef\bbl@tempb{\zap@space#1 \@empty}%
5236   \ifx\bbl@KVP@interchar\@nnil\else
5237     \bbl@replace\bbl@KVP@interchar{ }{,}%
5238     \bbl@foreach\bbl@tempb{%
5239       \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5240       \ifin@
5241         \let\bbl@tempa\@firstofone
5242       \fi}%
5243   \fi
5244   \bbl@tempa}
5245 \newcommand\IfBabelIntercharT[2]{%
5246   \bbl@carg\bbl@add{\bbl@icsave@\CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5247 \newcommand\babelcharclass[3]{%
5248   \EnableBabelHook{babel-interchar}%
5249   \bbl@csarg\newXeTeXintercharclass{xeclass@#2@#1}%
5250   \def\bbl@tempb##1{%
5251     \ifx##1\@empty\else
5252       \ifx##1-%
5253         \bbl@upto
5254       \else
5255         \bbl@charclass{%
5256           \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5257       \fi
5258       \expandafter\bbl@tempb
5259     \fi}%
5260   \bbl@ifunset{\bbl@xechars@#1}%
5261   {\toks@{%
5262     \babel@savevariable\XeTeXinterchartokenstate
5263     \XeTeXinterchartokenstate\@ne
5264   }}%
5265   {\toks@\expandafter\expandafter\expandafter{%
5266     \csname bbl@xechars@#1\endcsname}}%
5267   \bbl@csarg\edef{xechars@#1}{%
5268     \the\toks@
5269     \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5270     \bbl@tempb#3\@empty}}
5271 \protected\def\bbl@usingxeclass#1{\count@\zap@ \let\bbl@tempc#1}
5272 \protected\def\bbl@upto{%
5273   \ifnum\count@>\zap@
5274     \advance\count@\@ne
5275     \count@-\count@
5276   \else\ifnum\count@=\zap@
5277     \bbl@charclass{-}%
5278   \else
5279     \bbl@error{double-hyphens-class}{\count@}{\count@}%
5280   \fi\fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is a intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5281 \def\bbl@ignoreinterchar{%
5282   \ifnum\language=\l@nohyphenation
5283     \expandafter\@gobble
5284   \else
5285     \expandafter\@firstofone
5286   \fi}
5287 \newcommand\babelinterchar[5][[]]{%

```

```

5288 \let\bbl@kv@label\@empty
5289 \bbl@forkv{#1}{\bbl@csarg\edef{kv@##1}{##2}}%
5290 \namedef{\zap@space bbl@xeinter@\bbl@kv@label @#3@#4@#2 \@empty}%
5291 {\bbl@ignoreinterchar{#5}}%
5292 \bbl@csarg\let{ic@\bbl@kv@label @#2}\@firstofone
5293 \bbl@exp{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%
5294 \bbl@exp{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%
5295 \XeTeXinterchartoks
5296 \@nameuse{bbl@xeclasse@\bbl@tempa @#2}
5297 \bbl@ifunset{bbl@xeclasse@\bbl@tempa @#2}{#2} %
5298 \@nameuse{bbl@xeclasse@\bbl@tempb @#2}
5299 \bbl@ifunset{bbl@xeclasse@\bbl@tempb @#2}{#2} %
5300 = \expandafter{%
5301 \csname bbl@ic@\bbl@kv@label @#2\expandafter\endcsname
5302 \csname\zap@space bbl@xeinter@\bbl@kv@label
5303 @#3@#4@#2 \@empty\endcsname}}}}
5304 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5305 \bbl@ifunset{bbl@ic@#1@language}%
5306 {\bbl@error{unknown-interchar}{#1}{}}%
5307 {\bbl@csarg\let{ic@#1@language}\@firstofone}}
5308 \DeclareRobustCommand\disablelocaleinterchar[1]{%
5309 \bbl@ifunset{bbl@ic@#1@language}%
5310 {\bbl@error{unknown-interchar-b}{#1}{}}%
5311 {\bbl@csarg\let{ic@#1@language}\@gobble}}
5312 </xetex>

```

10.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titleps, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdftex and xetex.

```

5313 <*xetex|texet>
5314 \providecommand\bbl@provide@intraspace{}
5315 \bbl@trace{Redefinitions for bidi layout}

Finish here if there in no layout.

5316 \ifx\bbl@opt@layout\@nnil\else % if layout=..
5317 \ifx\bbl@opt@layout\@undefined\else % Plain
5318 \IfBabelLayout{nopars}
5319 {}
5320 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5321 \fi
5322 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5323 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5324 \ifnum\bbl@bidimode>\z@
5325 \IfBabelLayout{pars}
5326 {\def\@hangfrom#1{%
5327 \setbox\@tempboxa\hbox{#1}}%
5328 \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5329 \noindent\box\@tempboxa}
5330 \def\raggedright{%
5331 \let\\\@centercr
5332 \bbl@startskip\z@skip
5333 \@rightskip\@flushglue
5334 \bbl@endskip\@rightskip
5335 \parindent\z@
5336 \parfillskip\bbl@startskip}
5337 \def\raggedleft{%
5338 \let\\\@centercr

```

```

5339 \bbl@startskip\@flushglue
5340 \bbl@endskip\z@skip
5341 \parindent\z@
5342 \parfillskip\bbl@endskip}}
5343 {}
5344 \fi
5345 \IfBabelLayout{lists}
5346 {\bbl@sreplace\list
5347 {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5348 \def\bbl@listleftmargin{%
5349 \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5350 \ifcase\bbl@engine
5351 \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5352 \def\p@enumiii{\p@enumii}\theenumii{}\fi
5353 \fi
5354 \bbl@sreplace\@verbatim
5355 {\leftskip\@totalleftmargin}%
5356 {\bbl@startskip\textwidth
5357 \advance\bbl@startskip-\linewidth}%
5358 \bbl@sreplace\@verbatim
5359 {\rightskip\z@skip}%
5360 {\bbl@endskip\z@skip}}%
5361 {}
5362 \IfBabelLayout{contents}
5363 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5364 \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5365 {}
5366 \IfBabelLayout{columns}
5367 {\bbl@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bbl@outputbox}%
5368 \def\bbl@outputbox#1{%
5369 \hb@xt@\textwidth{%
5370 \hskip\columnwidth
5371 \hfil
5372 {\normalcolor\vrule \@width\columnseprule}%
5373 \hfil
5374 \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5375 \hskip-\textwidth
5376 \hb@xt@\columnwidth{\box\@outputbox \hss}%
5377 \hskip\columnsep
5378 \hskip\columnwidth}}}%
5379 {}

```

Implicitly reverses sectioning labels in bidi=basic, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5380 \IfBabelLayout{counters*}%
5381 {\bbl@add\bbl@opt@layout{.counters.}%
5382 \AddToHook{shipout/before}{%
5383 \let\bbl@tempa\babelsublr
5384 \let\babelsublr\@firstofone
5385 \let\bbl@save@thepage\thepage
5386 \protected@edef\thepage{\thepage}%
5387 \let\babelsublr\bbl@tempa}%
5388 \AddToHook{shipout/after}{%
5389 \let\thepage\bbl@save@thepage}}{}
5390 \IfBabelLayout{counters}%
5391 {\let\bbl@latinarabic=\@arabic
5392 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5393 \let\bbl@asciroman=\@roman
5394 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5395 \let\bbl@asciiRoman=\@Roman
5396 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5397 \fi % end if layout
5398 (/xetex|texet)

```

10.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```
5399 {*texxet}
5400 \def\bbl@provide@extra#1{%
5401   % == auto-select encoding ==
5402   \ifx\bbl@encoding@select@off\@empty\else
5403     \bbl@ifunset\bbl@encoding@#1{%
5404       {\def\elt##1{,##1,}%
5405        \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5406        \count@z@
5407        \bbl@foreach\bbl@tempe{%
5408          \def\bbl@tempd{##1}% Save last declared
5409          \advance\count@\@ne}%
5410        \ifnum\count@>\@ne % (1)
5411          \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5412          \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5413          \bbl@replace\bbl@tempa{ }{,}%
5414          \global\bbl@csarg\let{encoding@#1}\@empty
5415          \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5416          \ifin\@else % if main encoding included in ini, do nothing
5417            \let\bbl@tempb\relax
5418            \bbl@foreach\bbl@tempa{%
5419              \ifx\bbl@tempb\relax
5420                \bbl@xin@{,##1,}{,\bbl@tempe,}%
5421                \ifin\def\bbl@tempb{##1}\fi
5422              \fi}%
5423            \ifx\bbl@tempb\relax\else
5424              \bbl@exp{%
5425                \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>%
5426                \gdef\<bbl@encoding@#1>{%
5427                  \\babel@save\\f@encoding
5428                  \\bbl@add\\originalTeX{\\selectfont}%
5429                  \\fontencoding{\bbl@tempb}%
5430                  \\selectfont}}%
5431              \fi
5432            \fi
5433          \fi}%
5434        }%
5435      \fi}
5436 {/texxet}
```

10.5. LuaTeX

The loader for luatex is based solely on language.dat, which is read on the fly. The code shouldn't be executed when the format is build, so we check if \AddBabelHook is defined. Then comes a modified version of the loader in hyphen.cfg (without the hyphenmins stuff, which is under the direct control of babel).

The names \l@<language> are defined and take some value from the beginning because all ldf files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the ldf finishes). If a language has been loaded, \bbl@hyphendata@<num> exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in language.dat have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, lua(e)tex is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on babel, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format language.dat is used (under the principle of a single source), instead of language.def.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by babel) provide a command to allocate them (although there are packages like ctablestack). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, etex.sty changes the way languages are allocated.

This files is read at three places: (1) when plain.def, babel.sty starts, to read the list of available languages from language.dat (for the base option); (2) at hyphen.cfg, to modify some macros; (3) in the middle of plain.def and babel.sty, by babel.def, with the commands and other definitions for luatex (e.g., \babelpatterns).

```

5437 (*luatex)
5438 \directlua{ Babel = Babel or {} } % DL2
5439 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5440 \bbl@trace{Read language.dat}
5441 \ifx\bbl@readstream\undefined
5442 \csname newread\endcsname\bbl@readstream
5443 \fi
5444 \begingroup
5445 \toks@{}
5446 \count@\z@ % 0=start, 1=0th, 2=normal
5447 \def\bbl@process@line#1#2 #3 #4 {%
5448 \ifx=#1%
5449 \bbl@process@synonym{#2}%
5450 \else
5451 \bbl@process@language{#1#2}{#3}{#4}%
5452 \fi
5453 \ignorespaces}
5454 \def\bbl@manylang{%
5455 \ifnum\bbl@last>\@ne
5456 \bbl@info{Non-standard hyphenation setup}%
5457 \fi
5458 \let\bbl@manylang\relax}
5459 \def\bbl@process@language#1#2#3{%
5460 \ifcase\count@
5461 \ifundefined{zth@#1}{\count@\tw@}{\count@\@ne}%
5462 \or
5463 \count@\tw@
5464 \fi
5465 \ifnum\count@=\tw@
5466 \expandafter\addlanguage\csname l@#1\endcsname
5467 \language\allocationnumber
5468 \chardef\bbl@last\allocationnumber
5469 \bbl@manylang
5470 \let\bbl@elt\relax
5471 \xdef\bbl@languages{%
5472 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5473 \fi
5474 \the\toks@
5475 \toks@{}}
5476 \def\bbl@process@synonym@aux#1#2{%
5477 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5478 \let\bbl@elt\relax
5479 \xdef\bbl@languages{%
5480 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5481 \def\bbl@process@synonym#1{%
5482 \ifcase\count@
5483 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5484 \or

```

```

5485 \ifundefined{zth#1}{\bbl@process@synonym@aux{#1}{0}}{}%
5486 \else
5487 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5488 \fi}
5489 \ifx\bbl@languages\@undefined % Just a (sensible?) guess
5490 \chardef\l@english\z@
5491 \chardef\l@USenglish\z@
5492 \chardef\bbl@last\z@
5493 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}{}
5494 \gdef\bbl@languages{%
5495 \bbl@elt{english}{0}{hyphen.tex}}{}%
5496 \bbl@elt{USenglish}{0}{}}{}
5497 \else
5498 \global\let\bbl@languages@format\bbl@languages
5499 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5500 \ifnum#2>\z@\else
5501 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5502 \fi}%
5503 \xdef\bbl@languages{\bbl@languages}%
5504 \fi
5505 \def\bbl@elt#1#2#3#4{\@namedef{zth#1}}{} % Define flags
5506 \bbl@languages
5507 \openin\bbl@readstream=language.dat
5508 \ifeof\bbl@readstream
5509 \bbl@warning{I couldn't find language.dat. No additional\\%
5510 patterns loaded. Reported}%
5511 \else
5512 \loop
5513 \endlinechar\m@ne
5514 \read\bbl@readstream to \bbl@line
5515 \endlinechar\^^M
5516 \if T\ifeof\bbl@readstream F\fi T\relax
5517 \ifx\bbl@line\@empty\else
5518 \edef\bbl@line{\bbl@line\space\space\space}%
5519 \expandafter\bbl@process@line\bbl@line\relax
5520 \fi
5521 \repeat
5522 \fi
5523 \closein\bbl@readstream
5524 \endgroup
5525 \bbl@trace{Macros for reading patterns files}
5526 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5527 \ifx\babelcatcodetablenum\@undefined
5528 \ifx\newcatcodetable\@undefined
5529 \def\babelcatcodetablenum{5211}
5530 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5531 \else
5532 \newcatcodetable\babelcatcodetablenum
5533 \newcatcodetable\bbl@pattcodes
5534 \fi
5535 \else
5536 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5537 \fi
5538 \def\bbl@luapatterns#1#2{%
5539 \bbl@get@enc#1:.\@@@
5540 \setbox\z@\hbox\bgroup
5541 \begingroup
5542 \savecatcodetable\babelcatcodetablenum\relax
5543 \initcatcodetable\bbl@pattcodes\relax
5544 \catcodetable\bbl@pattcodes\relax
5545 \catcode`\#=6 \catcode`\$=3 \catcode`\&=4 \catcode`\^=7
5546 \catcode`\_ =8 \catcode`\{=1 \catcode`\}=2 \catcode`\~=13
5547 \catcode`\@=11 \catcode`\^^I=10 \catcode`\^^J=12

```

```

5548 \catcode\<=12 \catcode\>=12 \catcode\*=12 \catcode\.=12
5549 \catcode\-=12 \catcode\/=12 \catcode\[=12 \catcode\]=12
5550 \catcode\`=12 \catcode\'=12 \catcode\"=12
5551 \input #1\relax
5552 \catcodetable\babelcatcodetablenum\relax
5553 \endgroup
5554 \def\bbl@tempa{#2}%
5555 \ifx\bbl@tempa\@empty\else
5556 \input #2\relax
5557 \fi
5558 \egroup}%
5559 \def\bbl@patterns@lua#1{%
5560 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5561 \csname l@#1\endcsname
5562 \edef\bbl@tempa{#1}%
5563 \else
5564 \csname l@#1:\f@encoding\endcsname
5565 \edef\bbl@tempa{#1:\f@encoding}%
5566 \fi\relax
5567 \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5568 \@ifundefined{bbl@hyphendata@the\language}%
5569 {\def\bbl@elt##1##2###4{%
5570 \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5571 \def\bbl@tempb{##3}%
5572 \ifx\bbl@tempb\@empty\else % if not a synonymous
5573 \def\bbl@tempc{{##3}{##4}}%
5574 \fi
5575 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5576 \fi}%
5577 \bbl@languages
5578 \@ifundefined{bbl@hyphendata@the\language}%
5579 {\bbl@info{No hyphenation patterns were set for\%
5580 language '\bbl@tempa'. Reported}}%
5581 {\expandafter\expandafter\expandafter\bbl@luapatterns
5582 \csname bbl@hyphendata@the\language\endcsname}}}%
5583 \endinput\fi

```

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

```

5584 \ifx\DisableBabelHook\@undefined
5585 \AddBabelHook{luatex}{everylanguage}{%
5586 \def\process@language##1##2##3{%
5587 \def\process@line####1####2 ####3 ####4 {}}%
5588 \AddBabelHook{luatex}{loadpatterns}{%
5589 \input #1\relax
5590 \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5591 {{#1}}}%
5592 \AddBabelHook{luatex}{loadexceptions}{%
5593 \input #1\relax
5594 \def\bbl@tempb##1##2{{##1}{##2}}%
5595 \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5596 {\expandafter\expandafter\expandafter\bbl@tempb
5597 \csname bbl@hyphendata@the\language\endcsname}}%
5598 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5599 \begingroup
5600 \catcode\%=12
5601 \catcode\'=12
5602 \catcode\"=12
5603 \catcode\:=12
5604 \directlua{
5605 Babel.locale_props = Babel.locale_props or {}
5606

```

```

5607 function Babel.lua_error(e, a)
5608     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5609         e .. '}' .. (a or '') .. '}{'}])
5610 end
5611
5612 function Babel.bytes(line)
5613     return line:gsub(".",
5614         function (chr) return unicode.utf8.char(string.byte(chr)) end)
5615 end
5616
5617 function Babel.priority_in_callback(name,description)
5618     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5619         if v == description then return i end
5620     end
5621     return false
5622 end
5623
5624 function Babel.begin_process_input()
5625     if luatexbase and luatexbase.add_to_callback then
5626         luatexbase.add_to_callback('process_input_buffer',
5627             Babel.bytes,'Babel.bytes')
5628     else
5629         Babel.callback = callback.find('process_input_buffer')
5630         callback.register('process_input_buffer',Babel.bytes)
5631     end
5632 end
5633 function Babel.end_process_input ()
5634     if luatexbase and luatexbase.remove_from_callback then
5635         luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5636     else
5637         callback.register('process_input_buffer',Babel.callback)
5638     end
5639 end
5640
5641 function Babel.str_to_nodes(fn, matches, base)
5642     local n, head, last
5643     if fn == nil then return nil end
5644     for s in string.utfvalues(fn(matches)) do
5645         if base.id == 7 then
5646             base = base.replace
5647         end
5648         n = node.copy(base)
5649         n.char = s
5650         if not head then
5651             head = n
5652         else
5653             last.next = n
5654         end
5655         last = n
5656     end
5657     return head
5658 end
5659
5660 Babel.linebreaking = Babel.linebreaking or {}
5661 Babel.linebreaking.before = {}
5662 Babel.linebreaking.after = {}
5663 Babel.locale = {}
5664 function Babel.linebreaking.add_before(func, pos)
5665     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5666     if pos == nil then
5667         table.insert(Babel.linebreaking.before, func)
5668     else
5669         table.insert(Babel.linebreaking.before, pos, func)

```

```

5670     end
5671 end
5672 function Babel.linebreaking.add_after(func)
5673     tex.print([[\\noexpand\\csname bbl@luahyphenate\\endcsname]])
5674     table.insert(Babel.linebreaking.after, func)
5675 end
5676
5677 function Babel.addpatterns(pp, lg)
5678     local lg = lang.new(lg)
5679     local pats = lang.patterns(lg) or ''
5680     lang.clear_patterns(lg)
5681     for p in pp:gmatch('[^%s]+') do
5682         ss = ''
5683         for i in string.utfcharacters(p:gsub('%d', '')) do
5684             ss = ss .. '%d?' .. i
5685         end
5686         ss = ss:gsub('^%d%?%.', '%%.') .. '%d?'
5687         ss = ss:gsub('%.%d%?$', '%%.')
5688         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5689         if n == 0 then
5690             tex.sprint(
5691                 [[\\string\\csname\\space bbl@info\\endcsname{New pattern: }
5692                 .. p .. [{}]])
5693             pats = pats .. ' ' .. p
5694         else
5695             tex.sprint(
5696                 [[\\string\\csname\\space bbl@info\\endcsname{Renew pattern: }
5697                 .. p .. [{}]])
5698         end
5699     end
5700     lang.patterns(lg, pats)
5701 end
5702
5703 Babel.characters = Babel.characters or {}
5704 Babel.ranges = Babel.ranges or {}
5705 function Babel.hlist_has_bidi(head)
5706     local has_bidi = false
5707     local ranges = Babel.ranges
5708     for item in node.traverse(head) do
5709         if item.id == node.id'glyph' then
5710             local itemchar = item.char
5711             local chardata = Babel.characters[itemchar]
5712             local dir = chardata and chardata.d or nil
5713             if not dir then
5714                 for nn, et in ipairs(ranges) do
5715                     if itemchar < et[1] then
5716                         break
5717                     elseif itemchar <= et[2] then
5718                         dir = et[3]
5719                         break
5720                     end
5721                 end
5722             end
5723             if dir and (dir == 'al' or dir == 'r') then
5724                 has_bidi = true
5725             end
5726         end
5727     end
5728     return has_bidi
5729 end
5730 function Babel.set_chranges_b (script, chrng)
5731     if chrng == '' then return end
5732     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')

```

```

5733     Babel.script_blocks[script] = {}
5734     for s, e in string.gmatch(chrng..' ', '(.-%.(-)%s') do
5735         table.insert(
5736             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5737     end
5738 end
5739
5740 function Babel.discard_sublr(str)
5741     if str:find( [[\string\indexentry]] ) and
5742         str:find( [[\string\babelsublr]] ) then
5743         str = str:gsub( [[\string\babelsublr%s*(%b{})]],
5744             function(m) return m:sub(2,-2) end )
5745     end
5746     return str
5747 end
5748 }
5749 \endgroup
5750 \ifx\newattribute\@undefined\else % Test for plain
5751     \newattribute\bbl@attr@locale % DL4
5752     \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5753     \AddBabelHook{luatex}{beforeextras}{%
5754         \setattribute\bbl@attr@locale\localeid}
5755 \fi
5756 %
5757 \def\BabelStringsDefault{unicode}
5758 \let\luabbl@stop\relax
5759 \AddBabelHook{luatex}{encodedcommands}{%
5760     \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5761     \ifx\bbl@tempa\bbl@tempb\else
5762         \directlua{Babel.begin_process_input()}%
5763         \def\luabbl@stop{%
5764             \directlua{Babel.end_process_input()}}%
5765     \fi}%
5766 \AddBabelHook{luatex}{stopcommands}{%
5767     \luabbl@stop
5768     \let\luabbl@stop\relax}
5769 %
5770 \AddBabelHook{luatex}{patterns}{%
5771     \@ifundefined{bbl@hyphendata@the\language}%
5772     {\def\bbl@elt##1##2##3##4{%
5773         \ifnum##2=\csname l@##2\endcsname % #2=spanish, dutch:OT1...
5774         \def\bbl@tempb{##3}%
5775         \ifx\bbl@tempb\empty\else % if not a synonymous
5776             \def\bbl@tempc{{##3}{##4}}%
5777         \fi
5778         \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5779     \fi}%
5780     \bbl@languages
5781     \@ifundefined{bbl@hyphendata@the\language}%
5782     {\bbl@info{No hyphenation patterns were set for\%
5783         language '#2'. Reported}}%
5784     {\expandafter\expandafter\expandafter\bbl@luapatterns
5785         \csname bbl@hyphendata@the\language\endcsname}}}%
5786 \@ifundefined{bbl@patterns@}{}%
5787 \begingroup
5788     \bbl@xin@{, \number\language,}{, \bbl@pttnlist}%
5789     \ifin@ \else
5790         \ifx\bbl@patterns@\empty\else
5791             \directlua{ Babel.addpatterns(
5792                 [[\bbl@patterns@]], \number\language) }%
5793         \fi
5794     \@ifundefined{bbl@patterns@#1}%
5795         \empty

```

```

5796      {\directlua{ Babel.addpatterns(
5797          [[\space\csname bbl@patterns@#1\endcsname]],
5798          \number\language) }}%
5799      \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5800      \fi
5801      \endgroup}%
5802      \bbl@exp{%
5803          \bbl@ifunset{\bbl@prehc@\languagename}{}%
5804          {\bbl@ifblank{\bbl@cs{\prehc@\languagename}}{}}%
5805          {\prehyphenchar=\bbl@cl{\prehc}\relax}}}}

```

\babelpatterns This macro adds patterns. Two macros are used to store them: `\bbl@patterns@` for the global ones and `\bbl@patterns@<language>` for language ones. We make sure there is a space between words when multiple commands are used.

```

5806 \@onlypreamble\babelpatterns
5807 \AtEndOfPackage{%
5808     \newcommand\babelpatterns[2][\@empty]{%
5809         \ifx\bbl@patterns@\relax
5810             \let\bbl@patterns@\@empty
5811         \fi
5812         \ifx\bbl@pttnlist@\@empty\else
5813             \bbl@warning{%
5814                 You must not intermingle \string\selectlanguage\space and\%
5815                 \string\babelpatterns\space or some patterns will not\%
5816                 be taken into account. Reported}%
5817             \fi
5818             \ifx\@empty#1%
5819                 \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5820             \else
5821                 \edef\bbl@tempb{\zap@space#1 \@empty}%
5822                 \bbl@for\bbl@tempa\bbl@tempb{%
5823                     \bbl@fixname\bbl@tempa
5824                     \bbl@iflanguage\bbl@tempa{%
5825                         \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5826                             \@ifundefined{\bbl@patterns@\bbl@tempa}%
5827                             \@empty
5828                             {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5829                             #2}}}%
5830             \fi}}

```

10.6. Southeast Asian scripts

First, some general code for line breaking, used by `\babelposthyphenation`.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5831 \def\bbl@intraspace#1 #2 #3\@{%
5832     \directlua{
5833         Babel.intraspaces = Babel.intraspaces or {}
5834         Babel.intraspaces['\csname bbl@sbc@p@\languagename\endcsname'] = %
5835             {b = #1, p = #2, m = #3}
5836         Babel.locale_props[\the\localeid].intraspace = %
5837             {b = #1, p = #2, m = #3}
5838     }}
5839 \def\bbl@intrapenalty#1\@{%
5840     \directlua{
5841         Babel.intrapenalties = Babel.intrapenalties or {}
5842         Babel.intrapenalties['\csname bbl@sbc@p@\languagename\endcsname'] = #1
5843         Babel.locale_props[\the\localeid].intrapenalty = #1
5844     }}
5845 \begingroup
5846 \catcode`\%=12

```

```

5847 \catcode`\&=14
5848 \catcode`\'=12
5849 \catcode`\~ =12
5850 \gdef\bbl@seaintraspace{&
5851 \let\bbl@seaintraspace\relax
5852 \directlua{
5853   Babel.sea_enabled = true
5854   Babel.sea_ranges = Babel.sea_ranges or {}
5855   function Babel.set_chranges (script, chrng)
5856     local c = 0
5857     for s, e in string.gmatch(chrng..' ', '(.-%.%.(.-%s)') do
5858       Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5859       c = c + 1
5860     end
5861   end
5862   function Babel.sea_disc_to_space (head)
5863     local sea_ranges = Babel.sea_ranges
5864     local last_char = nil
5865     local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5866     for item in node.traverse(head) do
5867       local i = item.id
5868       if i == node.id'glyph' then
5869         last_char = item
5870       elseif i == 7 and item.subtype == 3 and last_char
5871         and last_char.char > 0x0C99 then
5872         quad = font.getfont(last_char.font).size
5873         for lg, rg in pairs(sea_ranges) do
5874           if last_char.char > rg[1] and last_char.char < rg[2] then
5875             lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril1
5876             local intraspace = Babel.intraspaces[lg]
5877             local intrapenalty = Babel.intrapenalties[lg]
5878             local n
5879             if intrapenalty ~= 0 then
5880               n = node.new(14, 0)      &% penalty
5881               n.penalty = intrapenalty
5882               node.insert_before(head, item, n)
5883             end
5884             n = node.new(12, 13)      &% (glue, spaceskip)
5885             node.setglue(n, intraspace.b * quad,
5886               intraspace.p * quad,
5887               intraspace.m * quad)
5888             node.insert_before(head, item, n)
5889             node.remove(head, item)
5890           end
5891         end
5892       end
5893     end
5894   end
5895 }&
5896 \bbl@luahyphenate}

```

10.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5897 \catcode`\%=14
5898 \gdef\bbl@cjkkintraspace{%
5899 \let\bbl@cjkkintraspace\relax
5900 \directlua{

```

```

5901 require('babel-data-cjk.lua')
5902 Babel.cjk_enabled = true
5903 function Babel.cjk_linebreak(head)
5904     local GLYPH = node.id'glyph'
5905     local last_char = nil
5906     local quad = 655360      % 10 pt = 655360 = 10 * 65536
5907     local last_class = nil
5908     local last_lang = nil
5909     for item in node.traverse(head) do
5910         if item.id == GLYPH then
5911             local lang = item.lang
5912             local LOCALE = node.get_attribute(item,
5913                 Babel.attr_locale)
5914             local props = Babel.locale_props[LOCALE] or {}
5915             local class = Babel.cjk_class[item.char].c
5916             if props.cjk_quotes and props.cjk_quotes[item.char] then
5917                 class = props.cjk_quotes[item.char]
5918             end
5919             if class == 'cp' then class = 'cl' % ]) as CL
5920             elseif class == 'id' then class = 'I'
5921             elseif class == 'cj' then class = 'I' % loose
5922             end
5923             local br = 0
5924             if class and last_class and Babel.cjk_breaks[last_class][class] then
5925                 br = Babel.cjk_breaks[last_class][class]
5926             end
5927             if br == 1 and props.linebreak == 'c' and
5928                 lang ~= \the\l@nohyphenation\space and
5929                 last_lang ~= \the\l@nohyphenation then
5930                 local intrapenalty = props.intrapenalty
5931                 if intrapenalty ~= 0 then
5932                     local n = node.new(14, 0)      % penalty
5933                     n.penalty = intrapenalty
5934                     node.insert_before(head, item, n)
5935                 end
5936                 local intraspace = props.intraspace
5937                 local n = node.new(12, 13)      % (glue, spaceskip)
5938                 node.setglue(n, intraspace.b * quad,
5939                     intraspace.p * quad,
5940                     intraspace.m * quad)
5941                 node.insert_before(head, item, n)
5942             end
5943             if font.getfont(item.font) then
5944                 quad = font.getfont(item.font).size
5945             end
5946             last_class = class
5947             last_lang = lang
5948             else % if penalty, glue or anything else
5949                 last_class = nil
5950             end
5951         end
5952         lang.hyphenate(head)
5953     end
5954 }%
5955 \bbl@luahyphenate}
5956 \gdef\bbl@luahyphenate{%
5957 \let\bbl@luahyphenate\relax
5958 \directlua{
5959     luatexbase.add_to_callback('hyphenate',
5960     function (head, tail)
5961         if Babel.linebreaking.before then
5962             for k, func in ipairs(Babel.linebreaking.before) do
5963                 func(head)

```

```

5964     end
5965     end
5966     lang.hyphenate(head)
5967     if Babel.cjk_enabled then
5968         Babel.cjk_linebreak(head)
5969     end
5970     if Babel.linebreaking.after then
5971         for k, func in ipairs(Babel.linebreaking.after) do
5972             func(head)
5973         end
5974     end
5975     if Babel.set_hboxed then
5976         Babel.set_hboxed(head)
5977     end
5978     if Babel.sea_enabled then
5979         Babel.sea_disc_to_space(head)
5980     end
5981 end,
5982 'Babel.hyphenate')
5983 }}
5984 \endgroup
5985 %
5986 \def\bbl@provide@intraspace{%
5987   \bbl@ifunset{\bbl@intsp@{language}}{%
5988     {\expandafter\ifx\csname\bbl@intsp@{language}\endcsname\@empty\else
5989       \bbl@xin@{c}{\bbl@cl{lnbrk}}}%
5990     \ifin@           % cjk
5991       \bbl@cjk@intraspace
5992       \directlua{
5993         Babel.locale_props = Babel.locale_props or {}
5994         Babel.locale_props[\the\localeid].linebreak = 'c'
5995       }%
5996       \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%
5997       \ifx\bbl@KVP@intrapenalty\@nnil
5998         \bbl@intrapenalty0\@@
5999       \fi
6000     \else           % sea
6001       \bbl@sea@intraspace
6002       \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%
6003       \directlua{
6004         Babel.sea_ranges = Babel.sea_ranges or {}
6005         Babel.set_chranges('\bbl@cl{sbcpr}',
6006           '\bbl@cl{chrng}')
6007       }%
6008       \ifx\bbl@KVP@intrapenalty\@nnil
6009         \bbl@intrapenalty0\@@
6010       \fi
6011     \fi
6012   \fi
6013   \ifx\bbl@KVP@intrapenalty\@nnil\else
6014     \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
6015   \fi}}

```

10.8. Arabic justification

WIP. `\bbl@arabicjust` is executed with both elongated and kashida. This must be fine tuned. The attribute `kashida` is set by transforms with `kashida`.

```

6016 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
6017 \def\bblar@chars{%
6018   0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
6019   0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
6020   0640,0641,0642,0643,0644,0645,0646,0647,0649}
6021 \def\bblar@elongated{%

```

```

6022 0626,0628,062A,062B,0633,0634,0635,0636,063B,%
6023 063C,063D,063E,063F,0641,0642,0643,0644,0646,%
6024 0649,064A}
6025 \begingroup
6026 \catcode\_ =11 \catcode\:=11
6027 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6028 \endgroup
6029 \gdef\bbl@arabicjust{%
6030 \let\bbl@arabicjust\relax
6031 \newattribute\bblar@kashida
6032 \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6033 \bblar@kashida=\z@
6034 \bbl@patchfont{\bbl@parsejalt}}%
6035 \directlua{
6036 Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6037 Babel.arabic.elong_map[\the\localeid] = {}
6038 luatexbase.add_to_callback('post_linebreak_filter',
6039 Babel.arabic.justify, 'Babel.arabic.justify')
6040 luatexbase.add_to_callback('hpack_filter',
6041 Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6042 }}%

```

Save both node lists to make replacement.

```

6043 \def\bblar@fetchjalt#1#2#3#4{%
6044 \bbl@exp{\bbl@foreach{#1}}{%
6045 \bbl@ifunset{bblar@JE@##1}%
6046 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"##1#2}}%
6047 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"\@nameuse{bblar@JE@##1}#2}}%
6048 \directlua{%
6049 local last = nil
6050 for item in node.traverse(tex.box[0].head) do
6051 if item.id == node.id'glyph' and item.char > 0x600 and
6052 not (item.char == 0x200D) then
6053 last = item
6054 end
6055 end
6056 Babel.arabic.#3['##1#4'] = last.char
6057 }}}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswb?). What about kaf? And diacritic positioning?

```

6058 \gdef\bbl@parsejalt{%
6059 \ifx\addfontfeature\undefined\else
6060 \bbl@xin{/e}{/bbl@ccl{lnbrk}}%
6061 \ifin@
6062 \directlua{%
6063 if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
6064 Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6065 tex.print([[string\cswb\space bbl@parsejalti\endcswb]])
6066 end
6067 }%
6068 \fi
6069 \fi}
6070 \gdef\bbl@parsejalti{%
6071 \begingroup
6072 \let\bbl@parsejalt\relax % To avoid infinite loop
6073 \edef\bbl@tempb{\fontid\font}%
6074 \bblar@nofswarn
6075 \@nameuse{tracinglostchars}\z@
6076 \bblar@fetchjalt\bblar@elongated{}{from}}%
6077 \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6078 \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6079 \addfontfeature{RawFeature+=jalt}%
6080 % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf

```

```

6081 \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6082 \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6083 \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6084 \directlua{%
6085     for k, v in pairs(Babel.arabic.from) do
6086         if Babel.arabic.dest[k] and
6087             not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6088             Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6089             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6090         end
6091     end
6092 }%
6093 \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6094 \beginngroup
6095 \catcode`#=11
6096 \catcode`~=11
6097 \directlua{
6098
6099 Babel.arabic = Babel.arabic or {}
6100 Babel.arabic.from = {}
6101 Babel.arabic.dest = {}
6102 Babel.arabic.justify_factor = 0.95
6103 Babel.arabic.justify_enabled = true
6104 Babel.arabic.kashida_limit = -1
6105
6106 function Babel.arabic.justify(head)
6107     if not Babel.arabic.justify_enabled then return head end
6108     for line in node.traverse_id(node.id'hlist', head) do
6109         Babel.arabic.justify_hlist(head, line)
6110     end
6111     % In case the very first item is a line (eg, in \vbox):
6112     while head.prev do head = head.prev end
6113     return head
6114 end
6115
6116 function Babel.arabic.justify_hbox(head, gc, size, pack)
6117     local has_inf = false
6118     if Babel.arabic.justify_enabled and pack == 'exactly' then
6119         for n in node.traverse_id(12, head) do
6120             if n.stretch_order > 0 then has_inf = true end
6121         end
6122         if not has_inf then
6123             Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6124         end
6125     end
6126     return head
6127 end
6128
6129 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6130     local d, new
6131     local k_list, k_item, pos_inline
6132     local width, width_new, full, k_curr, wt_pos, goal, shift
6133     local subst_done = false
6134     local elong_map = Babel.arabic.elong_map
6135     local cnt
6136     local last_line
6137     local GLYPH = node.id'glyph'
6138     local KASHIDA = Babel.attr_kashida
6139     local LOCALE = Babel.attr_locale
6140
6141     if line == nil then

```

```

6142     line = {}
6143     line.glue_sign = 1
6144     line.glue_order = 0
6145     line.head = head
6146     line.shift = 0
6147     line.width = size
6148 end
6149
6150 % Exclude last line.
6151 if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6152     elongs = {}      % Stores elongated candidates of each line
6153     k_list = {}      % And all letters with kashida
6154     pos_inline = 0   % Not yet used
6155
6156     for n in node.traverse_id(GLYPH, line.head) do
6157         pos_inline = pos_inline + 1 % To find where it is. Not used.
6158
6159         % Elongated glyphs
6160         if elong_map then
6161             local locale = node.get_attribute(n, LOCALE)
6162             if elong_map[locale] and elong_map[locale][n.font] and
6163                 elong_map[locale][n.font][n.char] then
6164                 table.insert(elongs, {node = n, locale = locale} )
6165                 node.set_attribute(n.prev, KASHIDA, 0)
6166             end
6167         end
6168
6169         % Tatwil. First create a list of nodes marked with kashida. The
6170         % rest of nodes can be ignored. The list of used weights is build
6171         % when transforms with the key kashida= are declared.
6172         if Babel.kashida_wts then
6173             local k_wt = node.get_attribute(n, KASHIDA)
6174             if k_wt > 0 then % todo. parameter for multi inserts
6175                 table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6176             end
6177         end
6178
6179     end % of node.traverse_id
6180
6181     if #elongs == 0 and #k_list == 0 then goto next_line end
6182     full = line.width
6183     shift = line.shift
6184     goal = full * Babel.arabic.justify_factor % A bit crude
6185     width = node.dimensions(line.head)      % The 'natural' width
6186
6187     % == Elongated ==
6188     % Original idea taken from 'chickenize'
6189     while (#elongs > 0 and width < goal) do
6190         subst_done = true
6191         local x = #elongs
6192         local curr = elongs[x].node
6193         local oldchar = curr.char
6194         curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6195         width = node.dimensions(line.head) % Check if the line is too wide
6196         % Substitute back if the line would be too wide and break:
6197         if width > goal then
6198             curr.char = oldchar
6199             break
6200         end
6201         % If continue, pop the just substituted node from the list:
6202         table.remove(elongs, x)
6203     end
6204

```

```

6205 % == Tatwil ==
6206 % Traverse the kashida node list so many times as required, until
6207 % the line is filled. The first pass adds a tatweel after each
6208 % node with kashida in the line, the second pass adds another one,
6209 % and so on. In each pass, add first the kashida with the highest
6210 % weight, then with lower weight and so on.
6211 if #k_list == 0 then goto next_line end
6212
6213 width = node.dimensions(line.head) % The 'natural' width
6214 k_curr = #k_list % Traverse backwards, from the end
6215 wt_pos = 1
6216
6217 while width < goal do
6218     subst_done = true
6219     k_item = k_list[k_curr].node
6220     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6221         d = node.copy(k_item)
6222         d.char = 0x0640
6223         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6224         d.xoffset = 0
6225         line.head, new = node.insert_after(line.head, k_item, d)
6226         width_new = node.dimensions(line.head)
6227         if width > goal or width == width_new then
6228             node.remove(line.head, new) % Better compute before
6229             break
6230         end
6231         if Babel.fix_diacr then
6232             Babel.fix_diacr(k_item.next)
6233         end
6234         width = width_new
6235     end
6236     if k_curr == 1 then
6237         k_curr = #k_list
6238         wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6239     else
6240         k_curr = k_curr - 1
6241     end
6242 end
6243
6244 % Limit the number of tatweel by removing them. Not very efficient,
6245 % but it does the job in a quite predictable way.
6246 if Babel.arabic.kashida_limit > -1 then
6247     cnt = 0
6248     for n in node.traverse_id(GLYPH, line.head) do
6249         if n.char == 0x0640 then
6250             cnt = cnt + 1
6251             if cnt > Babel.arabic.kashida_limit then
6252                 node.remove(line.head, n)
6253             end
6254         else
6255             cnt = 0
6256         end
6257     end
6258 end
6259
6260 ::next_line::
6261
6262 % Must take into account marks and ins, see luatex manual.
6263 % Have to be executed only if there are changes. Investigate
6264 % what's going on exactly.
6265 if subst_done and not gc then
6266     d = node.hpack(line.head, full, 'exactly')
6267     d.shift = shift

```

```

6268      node.insert_before(head, line, d)
6269      node.remove(head, line)
6270    end
6271  end % if process line
6272 end
6273 }
6274 \endgroup
6275 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6276 \def\bbl@scr@node@list{%
6277   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6278   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6279 \ifnum\bbl@bidimode=102 % bidi-r
6280   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6281 \fi
6282 \def\bbl@set@renderer{%
6283   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6284   \ifin@
6285     \let\bbl@unset@renderer\relax
6286   \else
6287     \bbl@exp{%
6288       \def\\bbl@unset@renderer{%
6289         \def<g__fontspec_default_fontopts_clist>{%
6290           [g__fontspec_default_fontopts_clist]}%
6291         \def<g__fontspec_default_fontopts_clist>{%
6292           Renderer=Harfbuzz,[g__fontspec_default_fontopts_clist]}%
6293       \fi}
6294 <@Font selection@>

```

10.10. Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6295 \directlua{% DL6
6296 Babel.script_blocks = {
6297   ['dflt'] = {},
6298   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6299             {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6300   ['Armn'] = {{0x0530, 0x058F}},
6301   ['Beng'] = {{0x0980, 0x09FF}},
6302   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6303   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6304   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6305             {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6306   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6307   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF}},

```

```

6308         {0xAB00, 0xAB2F}},
6309 ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6310 % Don't follow strictly Unicode, which places some Coptic letters in
6311 % the 'Greek and Coptic' block
6312 ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6313 ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6314             {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6315             {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6316             {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6317             {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6318             {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6319 ['Hebr'] = {{0x0590, 0x05FF},
6320             {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6321 ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6322             {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6323 ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6324 ['Knda'] = {{0x0C80, 0x0CFF}},
6325 ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6326             {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6327             {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6328 ['Lao'] = {{0x0E80, 0x0EFF}},
6329 ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6330             {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6331             {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6332 ['Mahj'] = {{0x11150, 0x1117F}},
6333 ['Mlym'] = {{0x0D00, 0x0D7F}},
6334 ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6335 ['Orya'] = {{0x0B00, 0x0B7F}},
6336 ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6337 ['Syr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6338 ['Taml'] = {{0x0B80, 0x0BFF}},
6339 ['Telu'] = {{0x0C00, 0x0C7F}},
6340 ['Tfng'] = {{0x2D30, 0x2D7F}},
6341 ['Thai'] = {{0x0E00, 0x0E7F}},
6342 ['Tibt'] = {{0x0F00, 0x0FFF}},
6343 ['Vaii'] = {{0xA500, 0xA63F}},
6344 ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6345 }
6346
6347 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6348 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6349 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6350
6351 function Babel.locale_map(head)
6352   if not Babel.locale_mapped then return head end
6353
6354   local LOCALE = Babel.attr_locale
6355   local GLYPH = node.id('glyph')
6356   local inmath = false
6357   local toloc_save
6358   for item in node.traverse(head) do
6359     local toloc
6360     if not inmath and item.id == GLYPH then
6361       % Optimization: build a table with the chars found
6362       if Babel.chr_to_loc[item.char] then
6363         toloc = Babel.chr_to_loc[item.char]
6364       else
6365         for lc, maps in pairs(Babel.loc_to_scr) do
6366           for _, rg in pairs(maps) do
6367             if item.char >= rg[1] and item.char <= rg[2] then
6368               Babel.chr_to_loc[item.char] = lc
6369             toloc = lc
6370             break

```

```

6371         end
6372     end
6373 end
6374 % Treat composite chars in a different fashion, because they
6375 % 'inherit' the previous locale.
6376 if (item.char >= 0x0300 and item.char <= 0x036F) or
6377    (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6378    (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6379     Babel.chr_to_loc[item.char] = -2000
6380     toloc = -2000
6381 end
6382 if not toloc then
6383     Babel.chr_to_loc[item.char] = -1000
6384 end
6385 end
6386 if toloc == -2000 then
6387     toloc = toloc_save
6388 elseif toloc == -1000 then
6389     toloc = nil
6390 end
6391 if toloc and Babel.locale_props[toloc] and
6392    Babel.locale_props[toloc].letters and
6393    tex.getcatcode(item.char) \string~= 11 then
6394     toloc = nil
6395 end
6396 if toloc and Babel.locale_props[toloc].script
6397    and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6398    and Babel.locale_props[toloc].script ==
6399    Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6400     toloc = nil
6401 end
6402 if toloc then
6403     if Babel.locale_props[toloc].lg then
6404         item.lang = Babel.locale_props[toloc].lg
6405         node.set_attribute(item, LOCALE, toloc)
6406     end
6407     if Babel.locale_props[toloc]['/'..item.font] then
6408         item.font = Babel.locale_props[toloc]['/'..item.font]
6409     end
6410 end
6411 toloc_save = toloc
6412 elseif not inmath and item.id == 7 then % Apply recursively
6413     item.replace = item.replace and Babel.locale_map(item.replace)
6414     item.pre      = item.pre and Babel.locale_map(item.pre)
6415     item.post     = item.post and Babel.locale_map(item.post)
6416 elseif item.id == node.id'math' then
6417     inmath = (item.subtype == 0)
6418 end
6419 end
6420 return head
6421 end
6422 }

```

The code for \babelcharproperty is straightforward. Just note the modified lua table can be different.

```

6423 \newcommand\babelcharproperty[1]{%
6424   \count@=#1\relax
6425   \ifvmode
6426     \expandafter\bbl@chprop
6427   \else
6428     \bbl@error{charproperty-only-vertical}{}{}{}%
6429   \fi}
6430 \newcommand\bbl@chprop[3][\the\count@]{%

```

```

6431 \@tempcnta=#1\relax
6432 \bbl@ifunset{bbl@chprop@#2}% {unknown-char-property}
6433   {\bbl@error{unknown-char-property}{\#2}}}%
6434   }%
6435 \loop
6436   \bbl@cs{chprop@#2}{#3}%
6437   \ifnum\count@<\@tempcnta
6438     \advance\count@\@ne
6439   \repeat}
6440 %
6441 \def\bbl@chprop@direction#1{%
6442   \directlua{
6443     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6444     Babel.characters[\the\count@]['d'] = '#1'
6445   }}
6446 \let\bbl@chprop@bc\bbl@chprop@direction
6447 %
6448 \def\bbl@chprop@mirror#1{%
6449   \directlua{
6450     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6451     Babel.characters[\the\count@]['m'] = '\number#1'
6452   }}
6453 \let\bbl@chprop@bmg\bbl@chprop@mirror
6454 %
6455 \def\bbl@chprop@linebreak#1{%
6456   \directlua{
6457     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6458     Babel.cjk_characters[\the\count@]['c'] = '#1'
6459   }}
6460 \let\bbl@chprop@lb\bbl@chprop@linebreak
6461 %
6462 \def\bbl@chprop@locale#1{%
6463   \directlua{
6464     Babel.chr_to_loc = Babel.chr_to_loc or {}
6465     Babel.chr_to_loc[\the\count@] =
6466       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6467   }}

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6468 \directlua{% DL7
6469   Babel.nohyphenation = \the\l@nohyphenation
6470 }

```

Now the $\TeX$ high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ - becomes `function(m) return m[1]..m[1]..'-' end`, where `m` are the matches returned after applying the pattern. With a mapped capture the functions are similar to `function(m) return Babel.capt_map(m[1],1) end`, where the last argument identifies the mapping to be applied to `m[1]`. The way it is carried out is somewhat tricky, but the effect is not dissimilar to `lua load` – save the code as string in a $\TeX$ macro, and expand this macro at the appropriate place. As `\directlua` does not take into account the current catcode of `@`, we just avoid this character in macro names (which explains the internal group, too).

```

6471 \begingroup
6472 \catcode`\~ =12
6473 \catcode`\% =12
6474 \catcode`\& =14
6475 \catcode`\| =12
6476 \gdef\babelprehyphenation{%&
6477   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}}{]}
6478 \gdef\babelposthyphenation{%&
6479   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}}{]}
6480 %
6481 \gdef\bbl@settransform#1[#2]#3#4#5{%&

```

```

6482 \ifcase#1
6483   \bbl@activateprehyphen
6484 \or
6485   \bbl@activateposthyphen
6486 \fi
6487 \beginingroup
6488   \def\babeltempa{\bbl@add@list\babeltempb}&%
6489   \let\babeltempb\@empty
6490   \def\bbl@tempa{#5}&%
6491   \bbl@replace\bbl@tempa{,}{,}&% TODO. Ugly trick to preserve {}
6492   \expandafter\bbl@foreach\expandafter{\bbl@tempa}{&%
6493     \bbl@ifsamestring{##1}{remove}&%
6494     {\bbl@add@list\babeltempb{nil}}&%
6495     {\directlua{
6496       local rep = [=##1]=]
6497       local three_args = '%s*=%s*([%-%d%.%a{}|]+)%s+([%-%d%.%a{}|]+)%s+([%-%d%.%a{}|]+)'
6498       &% Numeric passes directly: kern, penalty...
6499       rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6500       rep = rep:gsub('^%s*(insert)%s*', 'insert = true, ')
6501       rep = rep:gsub('^%s*(after)%s*', 'after = true, ')
6502       rep = rep:gsub('(string)%s*=%s*([^\s,]*)', Babel.capture_func)
6503       rep = rep:gsub('node%s*=%s*(%a+)%s*(%a*)', Babel.capture_node)
6504       rep = rep:gsub(' (norule)' .. three_args,
6505         'norule = {' .. '%2, %3, %4' .. '})')
6506       if #1 == 0 or #1 == 2 then
6507         rep = rep:gsub(' (space)' .. three_args,
6508           'space = {' .. '%2, %3, %4' .. '})')
6509         rep = rep:gsub(' (spacefactor)' .. three_args,
6510           'spacefactor = {' .. '%2, %3, %4' .. '})')
6511         rep = rep:gsub('(kashida)%s*=%s*([^\s,]*)', Babel.capture_kashida)
6512         &% Transform values
6513         rep, n = rep:gsub(' {[([a%-%.]+)|([a%-_.]+)}',
6514           function(v,d)
6515             return string.format (
6516               '{\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6517               v,
6518               load( 'return Babel.locale_props'..
6519                 '[\the\csname bbl@id@@#3\endcsname].' .. d)() )
6520             end )
6521         rep, n = rep:gsub(' {[([a%-%.]+)|([%-%d%.]+)}',
6522           '{\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6523       end
6524       if #1 == 1 then
6525         rep = rep:gsub(' (no)%s*=%s*([^\s,]*)', Babel.capture_func)
6526         rep = rep:gsub(' (pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6527         rep = rep:gsub(' (post)%s*=%s*([^\s,]*)', Babel.capture_func)
6528       end
6529       tex.print([[ \string\babeltempa{[]] .. rep .. [[]]])
6530     }]}&%
6531 \bbl@foreach\babeltempb{&%
6532   \bbl@forkv{##1}{&%
6533     \in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6534     post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6535     \ifin@else
6536       \bbl@error{bad-transform-option}{###1}{,}{&%
6537     \fi}}&%
6538 \let\bbl@kv@attribute\relax
6539 \let\bbl@kv@label\relax
6540 \let\bbl@kv@fonts\@empty
6541 \let\bbl@kv@prepend\relax
6542 \bbl@forkv{#2}{\bbl@csarg\edef{kv###1}{##2}}&%
6543 \ifx\bbl@kv@fonts\@empty\else\bbl@settransfont\fi
6544 \ifx\bbl@kv@attribute\relax

```

```

6545 \ifx\bbk@kv@label\relax\else
6546 \bbk@exp{\bbk@trim@def\bbk@kv@font{\bbk@kv@font}}&%
6547 \bbk@replace\bbk@kv@font{ }{,}&%
6548 \edef\bbk@kv@attribute{\bbk@ATR@bbk@kv@label @#3@bbk@kv@font}&%
6549 \count@z@
6550 \def\bbk@elt##1##2##3{&%
6551 \bbk@ifsamestring{#3,\bbk@kv@label}{##1,##2}&%
6552 {\bbk@ifsamestring{\bbk@kv@font}{##3}&%
6553 {\count@z@ne}&%
6554 {\bbk@error{font-conflict-transforms}{}}}&%
6555 }&%
6556 \bbk@transfont@list
6557 \ifnum\count@z@
6558 \bbk@exp{\global\bbk@add\bbk@transfont@list
6559 {\bbk@elt{#3}{\bbk@kv@label}{\bbk@kv@font}}}&%
6560 \fi
6561 \bbk@ifunset{\bbk@kv@attribute}&%
6562 {\global\bbk@carg\newattribute{\bbk@kv@attribute}}&%
6563 }&%
6564 \global\bbk@carg\setattribute{\bbk@kv@attribute}\@ne
6565 \fi
6566 \else
6567 \edef\bbk@kv@attribute{\expandafter\bbk@stripslash\bbk@kv@attribute}&%
6568 \fi
6569 \directlua{
6570 local lbrk = Babel.linebreaking.replacements[#1]
6571 local u = unicode.utf8
6572 local id, attr, label
6573 if #1 == 0 then
6574 id = \the\csname bbl@id@#3\endcsname\space
6575 else
6576 id = \the\csname l@#3\endcsname\space
6577 end
6578 \ifx\bbk@kv@attribute\relax
6579 attr = -1
6580 \else
6581 attr = luatexbase.registernumber'\bbk@kv@attribute'
6582 \fi
6583 \ifx\bbk@kv@label\relax\else &% Same refs:
6584 label = [==[\bbk@kv@label]==]
6585 \fi
6586 &% Convert pattern:
6587 local patt = string.gsub([==[#4]==], '%s', '')
6588 if #1 == 0 then
6589 patt = string.gsub(patt, '|', ' ')
6590 end
6591 if not u.find(patt, '()', nil, true) then
6592 patt = '()' .. patt .. '()'
6593 end
6594 patt = string.gsub(patt, '%(%)^', '^()')
6595 patt = string.gsub(patt, '%$(%)', '()$')
6596 patt = u.gsub(patt, '{(.)}',
6597 function (n)
6598 return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6599 end)
6600 patt = u.gsub(patt, '{(%X%X%X%X+)}',
6601 function (n)
6602 return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6603 end)
6604 lbrk[id] = lbrk[id] or {}
6605 table.insert(lbrk[id], \ifx\bbk@kv@prepend\relax\else 1,\fi
6606 { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6607 }&%

```

```

6608 \endgroup}
6609 \endgroup
6610 %
6611 \let\bbl@transfont@list\@empty
6612 \def\bbl@settransfont{%
6613   \global\let\bbl@settransfont\relax % Execute only once
6614   \gdef\bbl@transfont{%
6615     \def\bbl@elt####1####2####3{%
6616       \bbl@ifblank{####3}%
6617       {\count@tw@}% Do nothing if no fonts
6618       {\count@z@
6619         \bbl@vforeach{####3}{%
6620           \def\bbl@tempd{#####1}%
6621           \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6622           \ifx\bbl@tempd\bbl@tempe
6623             \count@@ne
6624             \else\ifx\bbl@tempd\bbl@transfam
6625               \count@@ne
6626             \fi\fi}%
6627           \ifcase\count@
6628             \bbl@csarg\unsetattribute{ATR@###2@###1@###3}%
6629           \or
6630             \bbl@csarg\setattribute{ATR@###2@###1@###3}\@ne
6631           \fi}}%
6632     \bbl@transfont@list}%
6633 \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6634 \gdef\bbl@transfam{-unknown-}%
6635 \bbl@foreach\bbl@font@fams{%
6636   \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6637   \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6638     {\xdef\bbl@transfam{##1}}%
6639   {}}
6640 %
6641 \DeclareRobustCommand\enablelocaletransform[1]{%
6642   \bbl@ifunset{\bbl@ATR@#1@\language @}%
6643   {\bbl@error{transform-not-available}{#1}{}}}%
6644   {\bbl@csarg\setattribute{ATR@#1@\language @}\@ne}}
6645 \DeclareRobustCommand\disablelocaletransform[1]{%
6646   \bbl@ifunset{\bbl@ATR@#1@\language @}%
6647   {\bbl@error{transform-not-available-b}{#1}{}}}%
6648   {\bbl@csarg\unsetattribute{ATR@#1@\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6649 \def\bbl@activateposthyphen{%
6650   \let\bbl@activateposthyphen\relax
6651   \ifx\bbl@attr@hboxed\@undefined
6652     \newattribute\bbl@attr@hboxed
6653   \fi
6654   \directlua{
6655     require('babel-transforms.lua')
6656     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6657   }}
6658 \def\bbl@activateprehyphen{%
6659   \let\bbl@activateprehyphen\relax
6660   \ifx\bbl@attr@hboxed\@undefined
6661     \newattribute\bbl@attr@hboxed
6662   \fi
6663   \directlua{
6664     require('babel-transforms.lua')
6665     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6666   }}
6667 \newcommand\SetTransformValue[3]{%

```

```

6668 \directlua{
6669   Babel.locale_props[\the\csname bbl@id@@#1\endcsname].vars["#2"] = #3
6670 }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6671 \newcommand\ShowBabelTransforms[1]{%
6672   \bbl@activateprehyphen
6673   \bbl@activateposthyphen
6674   \begingroup
6675   \directlua{ Babel.show_transforms = true }%
6676   \setbox\z@\vbox{#1}%
6677   \directlua{ Babel.show_transforms = false }%
6678   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]==]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6679 \newcommand\localeprehyphenation[1]{%
6680   \directlua{ Babel.string_prehyphenation([==#1]==], \the\localeid) }}

```

10.11.Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```

6681 \AtBeginDocument{\pagedir\bodydir}

```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by `TeX`. Just in case, consider the possibility it has not been loaded.

```

6682 \def\bbl@activate@preotf{%
6683   \let\bbl@activate@preotf\relax % only once
6684   \directlua{
6685     function Babel.pre_otfload_v(head)
6686       if Babel.numbers and Babel.digits_mapped then
6687         head = Babel.numbers(head)
6688       end
6689       if Babel.bidi_enabled then
6690         head = Babel.bidi(head, false, dir)
6691       end
6692       return head
6693     end
6694     %
6695     function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6696       if Babel.numbers and Babel.digits_mapped then
6697         head = Babel.numbers(head)
6698       end
6699       if Babel.bidi_enabled then
6700         head = Babel.bidi(head, false, dir)
6701       end
6702       return head
6703     end
6704     %
6705     luatexbase.add_to_callback('pre_linebreak_filter',
6706       Babel.pre_otfload_v,
6707       'Babel.pre_otfload_v',
6708       Babel.priority_in_callback('pre_linebreak_filter',
6709         'luaotfload.node_processor') or nil)
6710     %
6711     luatexbase.add_to_callback('hpack_filter',

```

```

6712     Babel.pre_otfload_h,
6713     'Babel.pre_otfload_h',
6714     Babel.priority_in_callback('hpack_filter',
6715     'luaotfload.node_processor') or nil)
6716 }

```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in basic-r.

```

6717 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6718   \let\bbl@beforeforeign\leavevmode
6719   \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6720   \ifnum\bbl@bidimode>300
6721     \RequirePackage{unibidi-lua}
6722   \else
6723     \RequirePackage{luatexbase}
6724     \bbl@activate@preotf
6725     \directlua{
6726       require('babel-data-bidi.lua')
6727       \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6728         require('babel-bidi-basic.lua')
6729       \or
6730         require('babel-bidi-basic-r.lua')
6731       table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6732       table.insert(Babel.ranges, {0xF0000, 0xFFFFFD, 'on'})
6733       table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6734     }
6735   \fi
6736   \newattribute\bbl@attr@dir
6737   \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6738   \bbl@expf{output{\bodydir\pagedir\the\output}}
6739 \fi
6740 %
6741 \chardef\bbl@thetextdir\z@
6742 \chardef\bbl@thepardir\z@
6743 \def\bbl@setluadir#1#2{% 1=\text/pardirection 2= 0:l 1:r 2:a}
6744   \ifcase#2\relax
6745     \ifcase#1\else#1=\z@\fi
6746   \else
6747     \ifcase#1#1=\@ne\fi
6748   \fi

```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (PP is the par dir) and `0x3` (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

```

6749 \def\bbl@thedir{0}
6750 \def\bbl@textdir#1{%
6751   \bbl@setluadir\textdirection{#1}%
6752   \chardef\bbl@thetextdir#1\relax
6753   \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6754   \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6755 \def\bbl@pardir#1{% Used twice
6756   \bbl@setluadir\pardirection{#1}%
6757   \chardef\bbl@thepardir#1\relax}
6758 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6759 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

```

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to ‘tabular’, which is based on a fake math.

```

6760 \ifnum\bbl@bidimode>\z@ % Any bidi=
6761   \def\bbl@insidemath{0}%
6762   \def\bbl@everymath{\def\bbl@insidemath{1}}
6763   \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6764   \frozen@everymath\expandafter{%

```

```

6765 \expandafter\bb1@everymath\the\ frozen@everymath}
6766 \frozen@everydisplay\expandafter{%
6767 \expandafter\bb1@everydisplay\the\ frozen@everydisplay}
6768 \AtBeginDocument{
6769 \directlua{
6770   function Babel.math_box_dir(head)
6771     if not (token.get_macro('bb1@insidemath') == '0') then
6772       if Babel.hlist_has_bidi(head) then
6773         local d = node.new(node.id'dir')
6774         d.dir = '+TRT'
6775         for item in node.traverse(head) do
6776           if item.id == 11 or item.id == node.id'glyph' then
6777             head = node.insert_before(head, item, d)
6778             break
6779           end
6780         end
6781         local inmath = false
6782         for item in node.traverse(head) do
6783           if item.id == 11 then
6784             inmath = (item.subtype == 0)
6785           elseif not inmath then
6786             node.set_attribute(item,
6787               Babel.attr_dir, token.get_macro('bb1@thedir'))
6788           end
6789         end
6790       end
6791     end
6792     return head
6793   end
6794   luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6795     "Babel.math_box_dir", 0)
6796   if Babel.unset_atdir then
6797     luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6798       "Babel.unset_atdir")
6799     luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6800       "Babel.unset_atdir")
6801   end
6802   luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6803     local dir = tex.mathdirection
6804     local n = node.new("dir")
6805     n.direction = dir
6806     head = node.insert_before(head, head, n)
6807     n = node.new("dir", 1)
6808     n.direction = dir
6809     head = node.insert_after(head, node.tail(head), n)
6810     return head
6811   end, "Babel.ensure_math_dir")
6812 } }%
6813 \fi

Experimental. Tentative name.

6814 \DeclareRobustCommand\localebox[1]{%
6815   {\def\bb1@insidemath{0}%
6816     \mbox{\foreignlanguage{\language name}{#1}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with bidi=basic, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I've made some progress in graphics, but

they're essentially hacks; I've also made some progress in 'tabular', but when I decided to tackle math (both standard math and 'amsmath') the nightmare began. I'm still not sure how 'amsmath' should be modified, but the main problem is that, boxes are "generic" containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with 'math' (11) nodes too).

\@hangfrom is useful in many contexts and it is redefined always with the layout option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by \bodydir), and when \parbox and \hangindent are involved. Fortunately, latest releases of luatex simplify a lot the solution with \shapemode.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, tabular seems to work (at least in simple cases) with array, tabularx, hline, colortbl, longtable, booktabs, etc. However, dcolumn still fails.

```
6817 \bbl@trace{Redefinitions for bidi layout}
6818 %
6819 ((*More package options)) ≡
6820 \chardef\bbl@eqnpos\z@
6821 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6822 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6823 (/More package options))
```

Fixes in luatex are sometimes done with flags. We first activate all all them.

```
6824 \ifnum\bbl@bidimode>\z@ % Any bidi=
6825 \matheqdirmode\@ne
6826 \mathemptydisplaymode\@ne
6827 \breakafterdirmode\@ne
6828 \fixupboxesmode\@ne
6829 \let\bbl@eqnodir\relax
6830 \def\bbl@eqdel{()}
6831 \def\bbl@eqnum{%
6832   {\normalfont\normalcolor
6833     \expandafter\@firstoftwo\bbl@eqdel
6834     \theequation
6835     \expandafter\@secondoftwo\bbl@eqdel}}
6836 \def\bbl@puteqno#1{\eqno\hbox{#1}}
6837 \def\bbl@putleqno#1{\leqno\hbox{#1}}
6838 \def\bbl@eqno@flip#1{% So
6839   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6840     \eqno
6841     \hb@xt@.01pt{%
6842       \hb@xt@\displaywidth{\hss#1\glet\bbl@upset\@currentlabel}}\hss}%
6843   \else
6844     \leqno\hbox{#1\glet\bbl@upset\@currentlabel}%
6845   \fi
6846   \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}}
6847 \def\bbl@leqno@flip#1{%
6848   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6849     \leqno
6850     \hb@xt@.01pt{%
6851       \hss\hb@xt@\displaywidth{\#1\glet\bbl@upset\@currentlabel}\hss}}%
6852   \else
6853     \eqno\hbox{#1\glet\bbl@upset\@currentlabel}%
6854   \fi
6855   \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}}
6856 %
6857 \AtBeginDocument{%
6858   \ifx\bbl@noamsmath\relax\else
6859   \ifx\maketag@@@\undefined % Normal equation, eqnarray
6860     \ifnum\bbl@eqnpos=\tw@
6861       \bbl@replace\equation{\hb@xt@\linewidth}%
6862       {\hbox bdir\mathdirection to\linewidth}%
6863       \bbl@carg\bbl@sreplace[ ]{\hb@xt@\linewidth}%
6864       {\hbox bdir\mathdirection to\linewidth}%
```

```

6865 \fi
6866 \AddToHook{env/equation/begin}{%
6867 \ifnum\bb@thetextdir>\z@
6868 \let\@eqnnum\bb@eqnum
6869 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6870 \chardef\bb@thetextdir\z@
6871 \bb@add\normalfont{\bb@eqnodir}%
6872 \ifcase\bb@eqnpos
6873 \let\bb@puteqno\bb@eqno@flip
6874 \or
6875 \let\bb@puteqno\bb@leqno@flip
6876 \fi
6877 \fi}%
6878 \ifnum\bb@eqnpos=\tw@%else
6879 \def\endequation{\bb@puteqno{\@eqnnum}$$\@ignoretrue}%
6880 \fi
6881 \AddToHook{env/eqnarray/begin}{%
6882 \ifnum\bb@thetextdir>\z@
6883 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6884 \chardef\bb@thetextdir\z@
6885 \bb@add\normalfont{\bb@eqnodir}%
6886 \ifnum\bb@eqnpos=\@ne
6887 \def\@eqnnum{%
6888 \setbox\z@\hbox{\bb@eqnum}%
6889 \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%
6890 \else
6891 \let\@eqnnum\bb@eqnum
6892 \fi
6893 \fi}
6894 \else % amstex
6895 \bb@exp{% Hack to hide maybe undefined conditionals:
6896 \chardef\bb@eqnpos=0<\iftagsleft>1<\else>\if@fleqn>2<\fi>\relax}%
6897 \ifnum\bb@eqnpos=\@ne
6898 \let\bb@ams@lap\hbox
6899 \else
6900 \let\bb@ams@lap\llap
6901 \fi
6902 \ExplSyntaxOn % Required by \bb@sreplace with \intertext@
6903 \bb@sreplace\intertext@{\normalbaselines}
6904 {\normalbaselines
6905 \ifx\bb@eqnodir\relax\else\bb@paddir\@ne\bb@eqnodir\fi}%
6906 \ExplSyntaxOff
6907 \def\bb@ams@tagbox#1#2{\bb@eqnodir#2}% #1=hbox|@lap|flip
6908 \ifx\bb@ams@lap\hbox % leqno
6909 \def\bb@ams@flip#1{%
6910 \hbox to 0.01pt{\hss\hbox to\displaywidth{\#1\hss}}}%
6911 \else % eqno
6912 \def\bb@ams@flip#1{%
6913 \hbox to 0.01pt{\hbox to\displaywidth{\hss\#1\hss}}}%
6914 \fi
6915 \def\bb@ams@preset#1{%
6916 \ifnum\bb@thetextdir>\z@
6917 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6918 \bb@sreplace\textdef@{\hbox}{\bb@ams@tagbox\hbox}%
6919 \bb@sreplace\maketag@@@{\hbox}{\bb@ams@tagbox#1}%
6920 \fi}%
6921 \def\bb@ams@equation{%
6922 \ifnum\bb@thetextdir>\z@
6923 \bb@ams@preset\hbox
6924 \chardef\bb@thetextdir\z@
6925 \bb@add\normalfont{\bb@eqnodir}%
6926 \ifcase\bb@eqnpos
6927 \def\veqno##1##2{\bb@eqno@flip{##1##2}}%

```

```

6928         \or
6929         \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6930         \fi
6931     \fi}%
6932     \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6933     \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6934     \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6935     \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6936     \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6937     \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6938     \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6939     \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6940     \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6941     \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6942     \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\hbox}%
6943     % Hackish, for proper alignment. Don't ask me why it works!:
6944     \bbl@exp{% Avoid a 'visible' conditional
6945         \\\AddToHook{env/align*/end}{\<iftag@>\<else>\\tag*{\<fi>}}%
6946         \\\AddToHook{env/alignat*/end}{\<iftag@>\<else>\\tag*{\<fi>}}%
6947     \AddToHook{env/flalign/begin}{\bbl@ams@preset\hbox}%
6948     \AddToHook{env/split/before}{%
6949         \ifnum\bbl@thetextdir>\z@
6950             \bbl@ifsamestring\@currentvir{equation}%
6951             {\ifx\bbl@ams@lap\hbox % leqno
6952                 \def\bbl@ams@flip#1{%
6953                     \hbox to 0.01pt{\hbox to\displaywidth{{#1}\hss}\hss}}%
6954                 \else
6955                     \def\bbl@ams@flip#1{%
6956                         \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}%
6957                     \fi}%
6958             }%
6959         \fi}%
6960     \fi\fi}
6961 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6962 \def\bbl@provide@extra#1{%
6963     % == onchar ==
6964     \ifx\bbl@KVP@onchar\@nnil\else
6965         \bbl@luahyphenate
6966         \bbl@exp{%
6967             \\\AddToHook{env/document/before}{%
6968                 {\let\\bbl@ifrestoring\\@firstoftwo
6969                     \\\select@language{#1}}}%
6970         \directlua{
6971             if Babel.locale_mapped == nil then
6972                 Babel.locale_mapped = true
6973                 Babel.linebreaking.add_before(Babel.locale_map, 1)
6974                 Babel.loc_to_scr = {}
6975                 Babel.chr_to_loc = Babel.chr_to_loc or {}
6976             end
6977             Babel.locale_props[\the\localeid].letters = false
6978         }%
6979         \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
6980     \ifin@
6981         \directlua{
6982             Babel.locale_props[\the\localeid].letters = true
6983         }%
6984     \fi
6985     \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
6986     \ifin@
6987         \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
6988             \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}%

```

```

6989 \fi
6990 \bbl@exp{\bbl@add{\bbl@starthyphens
6991 {\bbl@patterns@lua{\language}}}%
6992 \directlua{
6993   if Babel.script_blocks['\bbl@cl{sbc}'] then
6994     Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
6995     Babel.locale_props[\the\localeid].lg = \the\@nameuse{l@language}\space
6996   end
6997 }%
6998 \fi
6999 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
7000 \ifin@
7001 \bbl@ifunset{bbl@lsys@language}{\bbl@provide@lsys@language}}}%
7002 \bbl@ifunset{bbl@wdir@language}{\bbl@provide@dirs@language}}}%
7003 \directlua{
7004   if Babel.script_blocks['\bbl@cl{sbc}'] then
7005     Babel.loc_to_scr[\the\localeid] =
7006     Babel.script_blocks['\bbl@cl{sbc}']
7007   end}%
7008 \ifx\bbl@mapselect\undefined
7009 \AtBeginDocument{%
7010   \bbl@patchfont{\bbl@mapselect}}%
7011   {\selectfont}}%
7012 \def\bbl@mapselect{%
7013   \let\bbl@mapselect\relax
7014   \edef\bbl@prefontid{\fontid\font}}%
7015 \def\bbl@mapdir##1{%
7016   \begingroup
7017     \setbox\z@\hbox{% Force text mode
7018       \def\language{##1}%
7019       \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
7020       \bbl@switchfont
7021       \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
7022         \directlua{
7023           Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
7024             [\bbl@prefontid] = \fontid\font\space}%
7025         \fi}%
7026       \endgroup}%
7027 \fi
7028 \bbl@exp{\bbl@add{\bbl@mapselect{\bbl@mapdir@language}}}%
7029 \fi
7030 \fi
7031 % == mapfont ==
7032 % For bidi texts, to switch the font based on direction. Deprecated
7033 \ifx\bbl@KVP@mapfont\@nnil\else
7034   \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}}{%
7035     {\bbl@error{unknown-mapfont}}}%
7036   \bbl@ifunset{bbl@lsys@language}{\bbl@provide@lsys@language}}}%
7037   \bbl@ifunset{bbl@wdir@language}{\bbl@provide@dirs@language}}}%
7038 \ifx\bbl@mapselect\undefined
7039 \AtBeginDocument{%
7040   \bbl@patchfont{\bbl@mapselect}}%
7041   {\selectfont}}%
7042 \def\bbl@mapselect{%
7043   \let\bbl@mapselect\relax
7044   \edef\bbl@prefontid{\fontid\font}}%
7045 \def\bbl@mapdir##1{%
7046   {\def\language{##1}%
7047     \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7048     \bbl@switchfont
7049     \directlua{Babel.fontmap
7050       [\the\csname bbl@wdir@##1\endcsname]%
7051       [\bbl@prefontid]=\fontid\font}}}%

```

```

7052 \fi
7053 \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\language}}}%
7054 \fi
7055 % == Line breaking: CJK quotes ==
7056 \ifcase\bbl@engine\or
7057 \bbl@xin{/c}{/\bbl@ccl{lnbrk}}%
7058 \ifin@
7059 \bbl@ifunset{\bbl@quote@\language}%
7060 {\directlua{
7061     Babel.locale_props[\the\localeid].cjk_quotes = {}
7062     local cs = 'op'
7063     for c in string.utfvalues(
7064         [[\csname \bbl@quote@\language\endcsname]]) do
7065         if Babel.cjk_characters[c].c == 'qu' then
7066             Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7067         end
7068         cs = ( cs == 'op') and 'cl' or 'op'
7069     end
7070 }}%
7071 \fi
7072 \fi
7073 % == Counters: mapdigits ==
7074 % Native digits
7075 \ifx\bbl@KVP@mapdigits\@nnil\else
7076 \bbl@ifunset{\bbl@dgnat@\language}%
7077 {\bbl@activate@preotf
7078 \directlua{
7079     Babel.digits_mapped = true
7080     Babel.digits = Babel.digits or {}
7081     Babel.digits[\the\localeid] =
7082         table.pack(string.utfvalue('\bbl@ccl{dgnat}'))
7083     if not Babel.numbers then
7084         function Babel.numbers(head)
7085             local LOCALE = Babel.attr_locale
7086             local GLYPH = node.id'glyph'
7087             local inmath = false
7088             for item in node.traverse(head) do
7089                 if not inmath and item.id == GLYPH then
7090                     local temp = node.get_attribute(item, LOCALE)
7091                     if Babel.digits[temp] then
7092                         local chr = item.char
7093                         if chr > 47 and chr < 58 then
7094                             item.char = Babel.digits[temp][chr-47]
7095                         end
7096                     end
7097                 elseif item.id == node.id'math' then
7098                     inmath = (item.subtype == 0)
7099                 end
7100             end
7101             return head
7102         end
7103     end
7104 }}%
7105 \fi
7106 % == transforms ==
7107 \ifx\bbl@KVP@transforms\@nnil\else
7108 \def\bbl@elt##1##2##3{%
7109     \in@{$transforms.}{$##1}%
7110     \ifin@
7111     \def\bbl@tempa{##1}%
7112     \bbl@replace\bbl@tempa{transforms.}%
7113     \bbl@carg\bbl@transforms{babel\bbl@tempa}{##2}{##3}%
7114 \fi}%

```

```

7115 \bbl@exp{%
7116   \\\bbl@ifblank{\bbl@cl{dgnat}}}%
7117   {\let\\bbl@tempa\relax}%
7118   {\def\\bbl@tempa{%
7119     \\\bbl@elt{transforms.prehyphenation}%
7120     {digits.native.1.0}{([0-9])}%
7121     \\\bbl@elt{transforms.prehyphenation}%
7122     {digits.native.1.1}{string={1\string|0123456789\string|\bbl@cl{dgnat}}}}}%
7123 \ifx\bbl@tempa\relax\else
7124   \toks@{\expandafter\expandafter\expandafter{%
7125     \csname bbl@inidata@\language\endcsname}%
7126     \bbl@csarg\edef{inidata@\language}{%
7127       \unexpanded\expandafter{\bbl@tempa}%
7128       \the\toks@}%
7129   \fi
7130   \csname bbl@inidata@\language\endcsname
7131   \bbl@release@transforms\relax % \relax closes the last item.
7132 \fi}

```

Start tabular here:

```

7133 \def\localerestoredirs{%
7134   \ifcase\bbl@thetextdir
7135     \ifnum\textdirection=\z@ \else \textdirection=\z@ \fi
7136   \else
7137     \ifnum\textdirection=\@ne \else \textdirection=\@ne \fi
7138   \fi
7139   \ifcase\bbl@thepardir
7140     \ifnum\pardirection=\z@ \else \pardirection=\z@ \bodydirection=\z@ \fi
7141   \else
7142     \ifnum\pardirection=\@ne \else \pardirection=\@ne \bodydirection=\@ne \fi
7143   \fi}
7144 %
7145 \IfBabelLayout{tabular}%
7146   {\chardef\bbl@tabular@mode\z@}% All RTL
7147   {\IfBabelLayout{lrtabular}%
7148     {\chardef\bbl@tabular@mode\@ne}%
7149     {\chardef\bbl@tabular@mode\z@}}% Mixed, with LTR cols
7150 %
7151 \ifnum\bbl@bidimode>\@ne % Any lua bidi= except default=1
7152 % Redefine: vrules mess up dirs (why?).
7153 \AtBeginDocument{\def\arstrut{\relax\copy\arstrutbox}}%
7154 \ifcase\bbl@tabular@mode \else % 1 = Mixed
7155   \let\bbl@parabefore\relax
7156   \AddToHook{para/before}{\bbl@parabefore}
7157   \AtBeginDocument{%
7158     \bbl@replace@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7159     \ifnum\bbl@tabular@mode=\@ne
7160       \bbl@ifunset{tabclassz}{}%
7161       \bbl@exp{% Hide conditionals
7162         \\\bbl@sreplace\\\@tabclassz
7163         {\<ifcase>\\\@chnum}%
7164         {\\\localerestoredirs\<ifcase>\\\@chnum}}}%
7165       \@ifpackageloaded{colortbl}%
7166       {\bbl@sreplace\@classz
7167         {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7168       {\@ifpackageloaded{array}%
7169         {\bbl@exp{% Hide conditionals
7170           \\\bbl@sreplace\\\@classz
7171           {\<ifcase>\\\@chnum}%
7172           {\bgroup\\\localerestoredirs\<ifcase>\\\@chnum}%
7173           \\\bbl@sreplace\\\@classz
7174           {\\\do@row@strut\<fi>}{\\do@row@strut\<fi>\egroup}}}%
7175         {}}}%

```

```

7176 \fi}%
7177 \fi

```

Very likely the `\output` routine must be patched in a quite general way to make sure the `\bodydir` is set to `\pagedir`. Note outside `\output` they can be different (and often are). For the moment, two *ad hoc* changes.

```

7178 \AtBeginDocument{%
7179 \ifpackageloaded{multicol}%
7180 {\toks@%expandafter{\multi@column@out}%
7181 \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7182 {}%
7183 \ifpackageloaded{paracol}%
7184 {\edef\pcol@output{%
7185 \bodydir\pagedir\unexpanded%expandafter{\pcol@output}}}%
7186 {}}%
7187 \fi

```

Finish here if there in no layout.

```

7188 \ifx\bbl@opt@layout\@nnil\endinput\fi

```

`\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7189 \chardef\bbl@trace@vboxdir\z@
7190 \ifnum\bbl@bidimode>\z@ % Any bidi=
7191 \IfBabelLayout{nopars}{}
7192 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7193 \IfBabelLayout{pars}
7194 {\chardef\bbl@trace@vboxdir\@ne
7195 \def\@hangfrom#1{%
7196 \setbox\@tempboxa\hbox{{#1}}%
7197 \hangindent\wd\@tempboxa
7198 \ifnum\bbl@vbox@bodydir=\pardirection\else
7199 \shapemode\@ne
7200 \fi
7201 \noindent\box\@tempboxa}}
7202 {}
7203 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7204 \IfBabelLayout{lists}
7205 {\chardef\bbl@trace@vboxdir\@ne
7206 \let\bbl@OL@list\list
7207 \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7208 \let\bbl@NL@list\list
7209 \def\bbl@listparshape#1#2#3{%
7210 \parshape #1 #2 #3 %
7211 \ifnum\bbl@vbox@bodydir=\pardirection\else
7212 \shapemode\tw@
7213 \fi}}
7214 {}
7215 %
7216 \ifcase\bbl@trace@vboxdir\else
7217 \AtBeginDocument{\chardef\bbl@vbox@bodydir\pagedirection}%
7218 \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7219 \let\bbl@bidi@vbox\everyvbox
7220 \@nameuse{newtoks}\everyvbox % \outer in Plain
7221 \everyvbox%expandafter{\the\bbl@bidi@vbox}%
7222 \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7223 \fi
7224 %
7225 \IfBabelLayout{graphics}
7226 {\let\bbl@pictresetdir\relax
7227 \def\bbl@pictsetdir#1{%
7228 \ifcase\bbl@thetextdir

```

```

7229     \let\bbl@pictresetdir\relax
7230 \else
7231     \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7232     \or\textdir TLT
7233     \else\bodydir TLT \textdir TLT
7234     \fi
7235     % \(\text|par)dir required in pgf:
7236     \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7237 \fi}%
7238 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7239 \directlua{
7240     Babel.get_picture_dir = true
7241     Babel.picture_has_bidi = 0
7242     %
7243     function Babel.picture_dir (head)
7244         if not Babel.get_picture_dir then return head end
7245         if Babel.hlist_has_bidi(head) then
7246             Babel.picture_has_bidi = 1
7247         end
7248         return head
7249     end
7250     luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7251         "Babel.picture_dir")
7252 }%
7253 \AddToHook{package/graphics/after}{%
7254     \bbl@exp{%
7255         \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7256         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}}}%
7257 \AddToHook{package/graphicsx/after}{%
7258     \bbl@exp{%
7259         \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7260         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}}%
7261         \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7262         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#3}}}}}%
7263         \\bbl@sreplace\\Grot@box{\hbox{\hbox bdir\z@}}}%
7264 \AtBeginDocument{%
7265     \long\def\put(#1,#2)#3{%
7266         \@killglue
7267         % Try:
7268         \ifx\bbl@pictresetdir\relax
7269             \def\bbl@tempc{0}%
7270         \else
7271             \directlua{
7272                 Babel.get_picture_dir = true
7273                 Babel.picture_has_bidi = 0
7274             }%
7275             \setbox\z@\hb@xt@{z@}%
7276             \@defaultunitsset\@tempdimc{#1}\unitlength
7277             \kern\@tempdimc
7278             #3\hss}%
7279             \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}}%
7280         \fi
7281         % Do:
7282         \@defaultunitsset\@tempdimc{#2}\unitlength
7283         \raise\@tempdimc\hb@xt@{z@}%
7284         \@defaultunitsset\@tempdimc{#1}\unitlength
7285         \kern\@tempdimc
7286         {\ifnum\bbl@tempc>z@\bbl@pictresetdir\fi#3}\hss}%
7287     \ignorespaces}%
7288 \MakeRobust\put}%
7289 \AtBeginDocument
7290 {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7291 \ifx\pgfpicture\@undefined\else

```

```

7292      \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7293      \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7294      \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7295      \fi
7296      \ifx\tikzpicture\undefined\else
7297      \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw@}%
7298      \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7299      \bbl@sreplace\tikz{\begingroup}\begingroup\bbl@pictsetdir\tw@}%
7300      \bbl@sreplace\tikzpicture{\begingroup}\begingroup\bbl@pictsetdir\tw@}%
7301      \fi
7302  }}
7303  {}

```

Implicitly reverses sectioning labels in `bidi=basic-r`, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes `bidi=basic`, but there are some additional readjustments for `bidi=default`.

```

7304 \IfBabelLayout{counters*}%
7305   {\bbl@add\bbl@opt@layout{.counters.}%
7306     \directlua{
7307       luatexbase.add_to_callback("process_output_buffer",
7308         Babel.discard_sublr , "Babel.discard_sublr") }%
7309   }{}
7310 \IfBabelLayout{counters}%
7311   {\let\bbl@latinarabic=\@arabic
7312     \let\bbl@OL@@arabic=\@arabic
7313     \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7314     \ifpackagewith{babel}{bidi=default}%
7315       {\let\bbl@asciroman=\@roman
7316         \let\bbl@OL@@roman=\@roman
7317         \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7318         \let\bbl@asciiRoman=\@Roman
7319         \let\bbl@OL@@roman=\@Roman
7320         \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7321         \let\bbl@OL@labelenumii\labelenumii
7322         \def\labelenumii{}\theenumii}%
7323         \let\bbl@OL@p@enumiii\p@enumiii
7324         \def\p@enumiii{\p@enumii}\theenumii{}\}\}\}\}

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7325 \IfBabelLayout{extras}%
7326   {\bbl@ncarg\let\bbl@OL@underline{underline }%
7327     \bbl@carg\bbl@sreplace{underline }{\hbox}%
7328     {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7329     \let\bbl@OL@LaTeXe\LaTeXe
7330     \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7331       \if b\expandafter\@car\@f@series\@nil\boldmath\fi
7332       \babelsublr{\LaTeX\kern.15em2$_{\textstyle\varepsilon}$}}}%
7333   {}
7334 (/luatex)

```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionaries, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex manual`), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into

account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7335 (*transforms)
7336 Babel.linebreaking.replacements = {}
7337 Babel.linebreaking.replacements[0] = {} -- pre
7338 Babel.linebreaking.replacements[1] = {} -- post
7339
7340 function Babel.tovalue(v)
7341   if type(v) == 'table' then
7342     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7343   else
7344     return v
7345   end
7346 end
7347
7348 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7349
7350 function Babel.set_hboxed(head, gc)
7351   for item in node.traverse(head) do
7352     node.set_attribute(item, Babel.attr_hboxed, 1)
7353   end
7354   return head
7355 end
7356
7357 Babel.fetch_subtext = {}
7358
7359 Babel.ignore_pre_char = function(node)
7360   return (node.lang == Babel.nohyphenation)
7361 end
7362
7363 Babel.show_transforms = false
7364
7365 -- Merging both functions doesn't seem feasible, because there are too
7366 -- many differences.
7367 Babel.fetch_subtext[0] = function(head)
7368   local word_string = ''
7369   local word_nodes = {}
7370   local lang
7371   local item = head
7372   local inmath = false
7373
7374   while item do
7375     if item.id == 11 then
7376       inmath = (item.subtype == 0)
7377     end
7378
7379     if inmath then
7380       -- pass
7381     elseif item.id == 29 then
7382       local locale = node.get_attribute(item, Babel.attr_locale)
7383
7384       if lang == locale or lang == nil then
7385         lang = lang or locale
7386         if Babel.ignore_pre_char(item) then
7387           word_string = word_string .. Babel.us_char
7388         else
7389           if node.has_attribute(item, Babel.attr_hboxed) then
7390             word_string = word_string .. Babel.us_char
7391           else
7392             word_string = word_string .. unicode.utf8.char(item.char)
7393           end
7394         end
7395       end

```

```

7396         end
7397         word_nodes[#word_nodes+1] = item
7398     else
7399         break
7400     end
7401
7402     elseif item.id == 12 and item.subtype == 13 then
7403         if node.has_attribute(item, Babel.attr_hboxed) then
7404             word_string = word_string .. Babel.us_char
7405         else
7406             word_string = word_string .. ' '
7407         end
7408         word_nodes[#word_nodes+1] = item
7409
7410         -- Ignore leading unrecognized nodes, too.
7411         elseif word_string ~= '' then
7412             word_string = word_string .. Babel.us_char
7413             word_nodes[#word_nodes+1] = item -- Will be ignored
7414         end
7415
7416         item = item.next
7417     end
7418
7419     -- Here and above we remove some trailing chars but not the
7420     -- corresponding nodes. But they aren't accessed.
7421     if word_string:sub(-1) == ' ' then
7422         word_string = word_string:sub(1,-2)
7423     end
7424     if Babel.show_transforms then texio.write_nl(word_string) end
7425     word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7426     return word_string, word_nodes, item, lang
7427 end
7428
7429 Babel.fetch_subtext[1] = function(head)
7430     local word_string = ''
7431     local word_nodes = {}
7432     local lang
7433     local item = head
7434     local inmath = false
7435
7436     while item do
7437
7438         if item.id == 11 then
7439             inmath = (item.subtype == 0)
7440         end
7441
7442         if inmath then
7443             -- pass
7444         end
7445
7446         elseif item.id == 29 then
7447             if item.lang == lang or lang == nil then
7448                 lang = lang or item.lang
7449                 if node.has_attribute(item, Babel.attr_hboxed) then
7450                     word_string = word_string .. Babel.us_char
7451                 elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7452                     word_string = word_string .. Babel.us_char
7453                 else
7454                     word_string = word_string .. unicode.utf8.char(item.char)
7455                 end
7456                 word_nodes[#word_nodes+1] = item
7457             else
7458                 break
7459             end
7460         end
7461     end

```

```

7459
7460 elseif item.id == 7 and item.subtype == 2 then
7461     if node.has_attribute(item, Babel.attr_hboxed) then
7462         word_string = word_string .. Babel.us_char
7463     else
7464         word_string = word_string .. '='
7465     end
7466     word_nodes[#word_nodes+1] = item
7467
7468 elseif item.id == 7 and item.subtype == 3 then
7469     if node.has_attribute(item, Babel.attr_hboxed) then
7470         word_string = word_string .. Babel.us_char
7471     else
7472         word_string = word_string .. '|'
7473     end
7474     word_nodes[#word_nodes+1] = item
7475
7476 -- (1) Go to next word if nothing was found, and (2) implicitly
7477 -- remove leading USs.
7478 elseif word_string == '' then
7479     -- pass
7480
7481 -- This is the responsible for splitting by words.
7482 elseif (item.id == 12 and item.subtype == 13) then
7483     break
7484
7485 else
7486     word_string = word_string .. Babel.us_char
7487     word_nodes[#word_nodes+1] = item -- Will be ignored
7488 end
7489
7490 item = item.next
7491 end
7492 if Babel.show_transforms then texio.write_nl(word_string) end
7493 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7494 return word_string, word_nodes, item, lang
7495 end
7496
7497 function Babel.pre_hyphenate_replace(head)
7498     Babel.hyphenate_replace(head, 0)
7499 end
7500
7501 function Babel.post_hyphenate_replace(head)
7502     Babel.hyphenate_replace(head, 1)
7503 end
7504
7505 Babel.us_char = string.char(31)
7506
7507 function Babel.hyphenate_replace(head, mode)
7508     local u = unicode.utf8
7509     local lbkr = Babel.linebreaking.replacements[mode]
7510     local tovalue = Babel.tovalue
7511
7512     local word_head = head
7513
7514     if Babel.show_transforms then
7515         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7516     end
7517
7518     while true do -- for each subtext block
7519
7520         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7521

```

```

7522 if Babel.debug then
7523     print()
7524     print((mode == 0) and '@@@<' or '@@@>', w)
7525 end
7526
7527 if nw == nil and w == '' then break end
7528
7529 if not lang then goto next end
7530 if not lbkr[lang] then goto next end
7531
7532 -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7533 -- loops are nested.
7534 for k=1, #lbkr[lang] do
7535     local p = lbkr[lang][k].pattern
7536     local r = lbkr[lang][k].replace
7537     local attr = lbkr[lang][k].attr or -1
7538
7539     if Babel.debug then
7540         print('*****', p, mode)
7541     end
7542
7543     -- This variable is set in some cases below to the first *byte*
7544     -- after the match, either as found by u.match (faster) or the
7545     -- computed position based on sc if w has changed.
7546     local last_match = 0
7547     local step = 0
7548
7549     -- For every match.
7550     while true do
7551         if Babel.debug then
7552             print('====')
7553         end
7554         local new -- used when inserting and removing nodes
7555         local dummy_node -- used by after
7556
7557         local matches = { u.match(w, p, last_match) }
7558
7559         if #matches < 2 then break end
7560
7561         -- Get and remove empty captures (with ())'s, which return a
7562         -- number with the position), and keep actual captures
7563         -- (from (...)), if any, in matches.
7564         local first = table.remove(matches, 1)
7565         local last = table.remove(matches, #matches)
7566         -- Non re-fetched substrings may contain \31, which separates
7567         -- subsubstrings.
7568         if string.find(w:sub(first, last-1), Babel.us_char) then break end
7569
7570         local save_last = last -- with A()BC()D, points to D
7571
7572         -- Fix offsets, from bytes to unicode. Explained above.
7573         first = u.len(w:sub(1, first-1)) + 1
7574         last = u.len(w:sub(1, last-1)) -- now last points to C
7575
7576         -- This loop stores in a small table the nodes
7577         -- corresponding to the pattern. Used by 'data' to provide a
7578         -- predictable behavior with 'insert' (w_nodes is modified on
7579         -- the fly), and also access to 'remove'd nodes.
7580         local sc = first-1 -- Used below, too
7581         local data_nodes = {}
7582
7583         local enabled = true
7584         for q = 1, last-first+1 do

```

```

7585         data_nodes[q] = w_nodes[sc+q]
7586     if enabled
7587         and attr > -1
7588         and not node.has_attribute(data_nodes[q], attr)
7589     then
7590         enabled = false
7591     end
7592 end
7593
7594 -- This loop traverses the matched substring and takes the
7595 -- corresponding action stored in the replacement list.
7596 -- sc = the position in substr nodes / string
7597 -- rc = the replacement table index
7598 local rc = 0
7599
7600 ----- TODO. dummy_node?
7601 while rc < last-first+1 or dummy_node do -- for each replacement
7602     if Babel.debug then
7603         print('.....', rc + 1)
7604     end
7605     sc = sc + 1
7606     rc = rc + 1
7607
7608     if Babel.debug then
7609         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7610         local ss = ''
7611         for itt in node.traverse(head) do
7612             if itt.id == 29 then
7613                 ss = ss .. unicode.utf8.char(itt.char)
7614             else
7615                 ss = ss .. '{' .. itt.id .. '}'
7616             end
7617         end
7618         print('*****', ss)
7619     end
7620
7621     local crep = r[rc]
7622     local item = w_nodes[sc]
7623     local item_base = item
7624     local placeholder = Babel.us_char
7625     local d
7626
7627     if crep and crep.data then
7628         item_base = data_nodes[crep.data]
7629     end
7630
7631     if crep then
7632         step = crep.step or step
7633     end
7634
7635     if crep and crep.after then
7636         crep.insert = true
7637         if dummy_node then
7638             item = dummy_node
7639         else -- TODO. if there is a node after?
7640             d = node.copy(item_base)
7641             head, item = node.insert_after(head, item, d)
7642             dummy_node = item
7643         end
7644     end
7645
7646     if crep and not crep.after and dummy_node then

```

```

7648         node.remove(head, dummy_node)
7649         dummy_node = nil
7650     end
7651
7652     if not enabled then
7653         last_match = save_last
7654         goto next
7655
7656     elseif crep and next(crep) == nil then -- = {}
7657         if step == 0 then
7658             last_match = save_last    -- Optimization
7659         else
7660             last_match = utf8.offset(w, sc+step)
7661         end
7662         goto next
7663
7664     elseif crep == nil or crep.remove then
7665         node.remove(head, item)
7666         table.remove(w_nodes, sc)
7667         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7668         sc = sc - 1 -- Nothing has been inserted.
7669         last_match = utf8.offset(w, sc+1+step)
7670         goto next
7671
7672     elseif crep and crep.kashida then
7673         node.set_attribute(item,
7674             Babel.attr_kashida,
7675             crep.kashida)
7676         last_match = utf8.offset(w, sc+1+step)
7677         goto next
7678
7679     elseif crep and crep.string then
7680         local str = crep.string(matches)
7681         if str == '' then -- Gather with nil
7682             node.remove(head, item)
7683             table.remove(w_nodes, sc)
7684             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7685             sc = sc - 1 -- Nothing has been inserted.
7686         else
7687             local loop_first = true
7688             for s in string.utfvalues(str) do
7689                 d = node.copy(item_base)
7690                 d.char = s
7691                 if loop_first then
7692                     loop_first = false
7693                     head, new = node.insert_before(head, item, d)
7694                     if sc == 1 then
7695                         word_head = head
7696                     end
7697                     w_nodes[sc] = d
7698                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7699                 else
7700                     sc = sc + 1
7701                     head, new = node.insert_before(head, item, d)
7702                     table.insert(w_nodes, sc, new)
7703                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7704                 end
7705                 if Babel.debug then
7706                     print('.....', 'str')
7707                     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7708                 end
7709             end -- for
7710             node.remove(head, item)

```

```

7711         end -- if ''
7712         last_match = utf8.offset(w, sc+1+step)
7713         goto next
7714
7715     elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7716         d = node.new(7, 3) -- (disc, regular)
7717         d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7718         d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7719         d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7720         d.attr = item_base.attr
7721         if crep.pre == nil then -- TeXbook p96
7722             d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7723         else
7724             d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7725         end
7726         placeholder = '|'
7727         head, new = node.insert_before(head, item, d)
7728
7729     elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7730         -- ERROR
7731
7732     elseif crep and crep.penalty then
7733         d = node.new(14, 0) -- (penalty, userpenalty)
7734         d.attr = item_base.attr
7735         d.penalty = tovalue(crep.penalty)
7736         head, new = node.insert_before(head, item, d)
7737
7738     elseif crep and crep.space then
7739         -- 655360 = 10 pt = 10 * 65536 sp
7740         d = node.new(12, 13) -- (glue, spaceskip)
7741         local quad = font.getfont(item_base.font).size or 655360
7742         node.setglue(d, tovalue(crep.space[1]) * quad,
7743             tovalue(crep.space[2]) * quad,
7744             tovalue(crep.space[3]) * quad)
7745         if mode == 0 then
7746             placeholder = ' '
7747         end
7748         head, new = node.insert_before(head, item, d)
7749
7750     elseif crep and crep.norule then
7751         -- 655360 = 10 pt = 10 * 65536 sp
7752         d = node.new(2, 3) -- (rule, empty) = \no*rule
7753         local quad = font.getfont(item_base.font).size or 655360
7754         d.width = tovalue(crep.norule[1]) * quad
7755         d.height = tovalue(crep.norule[2]) * quad
7756         d.depth = tovalue(crep.norule[3]) * quad
7757         head, new = node.insert_before(head, item, d)
7758
7759     elseif crep and crep.spacefactor then
7760         d = node.new(12, 13) -- (glue, spaceskip)
7761         local base_font = font.getfont(item_base.font)
7762         node.setglue(d,
7763             tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7764             tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7765             tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7766         if mode == 0 then
7767             placeholder = ' '
7768         end
7769         head, new = node.insert_before(head, item, d)
7770
7771     elseif mode == 0 and crep and crep.space then
7772         -- ERROR
7773

```

```

7774     elseif crep and crep.kern then
7775         d = node.new(13, 1)      -- (kern, user)
7776         local quad = font.getfont(item_base.font).size or 655360
7777         d.attr = item_base.attr
7778         d.kern = tovalue(crep.kern) * quad
7779         head, new = node.insert_before(head, item, d)
7780
7781     elseif crep and crep.node then
7782         d = node.new(crep.node[1], crep.node[2])
7783         d.attr = item_base.attr
7784         head, new = node.insert_before(head, item, d)
7785
7786     end -- i.e., replacement cases
7787
7788     -- Shared by disc, space(factor), kern, node and penalty.
7789     if sc == 1 then
7790         word_head = head
7791     end
7792     if crep.insert then
7793         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7794         table.insert(w_nodes, sc, new)
7795         last = last + 1
7796     else
7797         w_nodes[sc] = d
7798         node.remove(head, item)
7799         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7800     end
7801
7802     last_match = utf8.offset(w, sc+1+step)
7803
7804     ::next::
7805
7806     end -- for each replacement
7807
7808     if Babel.show_transforms then texio.write_nl('> ' .. w) end
7809     if Babel.debug then
7810         print('.....', '/')
7811         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7812     end
7813
7814     if dummy_node then
7815         node.remove(head, dummy_node)
7816         dummy_node = nil
7817     end
7818
7819     end -- for match
7820
7821     end -- for patterns
7822
7823     ::next::
7824     word_head = nw
7825 end -- for substring
7826
7827 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7828 return head
7829 end
7830
7831 -- This table stores capture maps, numbered consecutively
7832 Babel.capture_maps = {}
7833
7834 function Babel.esc_hex_to_char(h)
7835     if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7836         tex.getcatcode(tonumber(h, 16)) ~= 12 then

```

```

7837     return string.format([[Uchar"%X ]], tonumber(h,16))
7838 else
7839     return unicode.utf8.char(tonumber(h, 16))
7840 end
7841 end
7842
7843 -- The following functions belong to the next macro
7844 function Babel.capture_func(key, cap)
7845     local ret = "[" .. cap:gsub('{{([0-9])}}', "]]..m[%1]..[" .. "]"
7846     local cnt
7847     local u = unicode.utf8
7848     ret = u.gsub(ret, '{{(%%x%%x%%x+}}', '\x01%1\x04')
7849     ret, cnt = ret:gsub('{{([0-9])|(^|+)|(.-)}}', Babel.capture_func_map)
7850     ret = u.gsub(ret, '\x01(%%x%%x%%x+)\x04', Babel.esc_hex_to_char)
7851     ret = ret:gsub("%[%[%]%]%.%", '')
7852     ret = ret:gsub("%.%.%[%[%]%]", '')
7853     return key .. [[=function(m) return ]] .. ret .. [[ end]]
7854 end
7855
7856 function Babel.capt_map(from, mapno)
7857     return Babel.capture_maps[mapno][from] or from
7858 end
7859
7860 -- Handle the {n|abc|ABC} syntax in captures
7861 function Babel.capture_func_map(capno, from, to)
7862     local u = unicode.utf8
7863     from = u.gsub(from, '\x01(%%x%%x%%x+)\x04',
7864         function (n)
7865             return u.char(tonumber(n, 16))
7866         end)
7867     to = u.gsub(to, '\x01(%%x%%x%%x+)\x04',
7868         function (n)
7869             return u.char(tonumber(n, 16))
7870         end)
7871     local froms = {}
7872     for s in string.utfcharacters(from) do
7873         table.insert(froms, s)
7874     end
7875     local cnt = 1
7876     table.insert(Babel.capture_maps, {})
7877     local mlen = table.getn(Babel.capture_maps)
7878     for s in string.utfcharacters(to) do
7879         Babel.capture_maps[mlen][froms[cnt]] = s
7880         cnt = cnt + 1
7881     end
7882     return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7883         (mlen) .. ")]" .. " ["
7884 end
7885
7886 -- Create/Extend reversed sorted list of kashida weights:
7887 function Babel.capture_kashida(key, wt)
7888     wt = tonumber(wt)
7889     if Babel.kashida_wts then
7890         for p, q in ipairs(Babel.kashida_wts) do
7891             if wt == q then
7892                 break
7893             elseif wt > q then
7894                 table.insert(Babel.kashida_wts, p, wt)
7895                 break
7896             elseif table.getn(Babel.kashida_wts) == p then
7897                 table.insert(Babel.kashida_wts, wt)
7898             end
7899         end
7900     end

```

```

7900 else
7901     Babel.kashida_wts = { wt }
7902 end
7903 return 'kashida = ' .. wt
7904 end
7905
7906 function Babel.capture_node(id, subtype)
7907     local sbt = 0
7908     for k, v in pairs(node.subtypes(id)) do
7909         if v == subtype then sbt = k end
7910     end
7911     return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7912 end
7913
7914 -- Experimental: applies prehyphenation transforms to a string (letters
7915 -- and spaces).
7916 function Babel.string_prehyphenation(str, locale)
7917     local n, head, last, res
7918     head = node.new(8, 0) -- dummy (hack just to start)
7919     last = head
7920     for s in string.utfvalues(str) do
7921         if s == 20 then
7922             n = node.new(12, 0)
7923         else
7924             n = node.new(29, 0)
7925             n.char = s
7926         end
7927         node.set_attribute(n, Babel.attr_locale, locale)
7928         last.next = n
7929         last = n
7930     end
7931     head = Babel.hyphenate_replace(head, 0)
7932     res = ''
7933     for n in node.traverse(head) do
7934         if n.id == 12 then
7935             res = res .. ' '
7936         elseif n.id == 29 then
7937             res = res .. unicode.utf8.char(n.char)
7938         end
7939     end
7940     tex.print(res)
7941 end
7942 (/transforms)

```

10.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%

```

For the meaning of these codes, see the Unicode standard.

Now the basic-r bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following

text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In babel the dir is set by a higher protocol based on the language/script, which in turn sets the correct dir (<l>, <r> or <al>).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don't think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where luatex excels, because everything related to bidi writing is under our control.

```
7943 (*basic-r)
7944 Babel.bidi_enabled = true
7945
7946 require('babel-data-bidi.lua')
7947
7948 local characters = Babel.characters
7949 local ranges = Babel.ranges
7950
7951 local DIR = node.id("dir")
7952
7953 local function dir_mark(head, from, to, outer)
7954   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7955   local d = node.new(DIR)
7956   d.dir = '+' .. dir
7957   node.insert_before(head, from, d)
7958   d = node.new(DIR)
7959   d.dir = '-' .. dir
7960   node.insert_after(head, to, d)
7961 end
7962
7963 function Babel.bidi(head, ispar)
7964   local first_n, last_n          -- first and last char with nums
7965   local last_es                  -- an auxiliary 'last' used with nums
7966   local first_d, last_d          -- first and last char in L/R block
7967   local dir, dir_real
```

Next also depends on script/lang (<al>/<r>). To be set by babel. `tex.pardir` is dangerous, could be (re)set but it should be changed only in vmode. There are two strong's – `strong = l/al/r` and `strong_lr = l/r` (there must be a better way):

```
7968   local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7969   local strong_lr = (strong == 'l') and 'l' or 'r'
7970   local outer = strong
7971
7972   local new_dir = false
7973   local first_dir = false
7974   local inmath = false
7975
7976   local last_lr
7977
7978   local type_n = ''
7979
7980   for item in node.traverse(head) do
7981     -- three cases: glyph, dir, otherwise
7982
```

```

7983   if item.id == node.id'glyph'
7984     or (item.id == 7 and item.subtype == 2) then
7985
7986     local itemchar
7987     if item.id == 7 and item.subtype == 2 then
7988       itemchar = item.replace.char
7989     else
7990       itemchar = item.char
7991     end
7992     local chardata = characters[itemchar]
7993     dir = chardata and chardata.d or nil
7994     if not dir then
7995       for nn, et in ipairs(ranges) do
7996         if itemchar < et[1] then
7997           break
7998         elseif itemchar <= et[2] then
7999           dir = et[3]
8000           break
8001         end
8002       end
8003     end
8004     dir = dir or 'l'
8005     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

8006   if new_dir then
8007     attr_dir = 0
8008     for at in node.traverse(item.attr) do
8009       if at.number == Babel.attr_dir then
8010         attr_dir = at.value & 0x3
8011       end
8012     end
8013     if attr_dir == 1 then
8014       strong = 'r'
8015     elseif attr_dir == 2 then
8016       strong = 'al'
8017     else
8018       strong = 'l'
8019     end
8020     strong_lr = (strong == 'l') and 'l' or 'r'
8021     outer = strong_lr
8022     new_dir = false
8023   end
8024
8025   if dir == 'nsm' then dir = strong end          -- W1

```

Numbers. The dual <al>/<r> system for R is somewhat cumbersome.

```

8026   dir_real = dir          -- We need dir_real to set strong below
8027   if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no <en> <et> <es> if strong == <al>, only <an>. Therefore, there are not <et en> nor <en et>, W5 can be ignored, and W6 applied:

```

8028   if strong == 'al' then
8029     if dir == 'en' then dir = 'an' end          -- W2
8030     if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8031     strong_lr = 'r'                             -- W3
8032   end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8033   elseif item.id == node.id'dir' and not inmath then

```

```

8034     new_dir = true
8035     dir = nil
8036   elseif item.id == node.id'math' then
8037     inmath = (item.subtype == 0)
8038   else
8039     dir = nil          -- Not a char
8040   end

```

Numbers in R mode. A sequence of <en>, <et>, <an>, <es> and <cs> is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only <an> is relevant if <al>.

```

8041   if dir == 'en' or dir == 'an' or dir == 'et' then
8042     if dir ~= 'et' then
8043       type_n = dir
8044     end
8045     first_n = first_n or item
8046     last_n = last_es or item
8047     last_es = nil
8048   elseif dir == 'es' and last_n then -- W3+W6
8049     last_es = item
8050   elseif dir == 'cs' then          -- it's right - do nothing
8051   elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8052     if strong_lr == 'r' and type_n ~= '' then
8053       dir_mark(head, first_n, last_n, 'r')
8054     elseif strong_lr == 'l' and first_d and type_n == 'an' then
8055       dir_mark(head, first_n, last_n, 'r')
8056       dir_mark(head, first_d, last_d, outer)
8057       first_d, last_d = nil, nil
8058     elseif strong_lr == 'l' and type_n ~= '' then
8059       last_d = last_n
8060     end
8061     type_n = ''
8062     first_n, last_n = nil, nil
8063   end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8064   if dir == 'l' or dir == 'r' then
8065     if dir ~= outer then
8066       first_d = first_d or item
8067       last_d = item
8068     elseif first_d and dir ~= strong_lr then
8069       dir_mark(head, first_d, last_d, outer)
8070       first_d, last_d = nil, nil
8071     end
8072   end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it's clearly <r> and <l>, resptly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn't hurt.

```

8073   if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8074     item.char = characters[item.char] and
8075       characters[item.char].m or item.char
8076   elseif (dir or new_dir) and last_lr ~= item then
8077     local mir = outer .. strong_lr .. (dir or outer)
8078     if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8079       for ch in node.traverse(node.next(last_lr)) do
8080         if ch == item then break end

```

```

8081         if ch.id == node.id'glyph' and characters[ch.char] then
8082             ch.char = characters[ch.char].m or ch.char
8083         end
8084     end
8085 end
8086 end

```

Save some values for the next iteration. If the current node is 'dir', open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8087     if dir == 'l' or dir == 'r' then
8088         last_lr = item
8089         strong = dir_real          -- Don't search back - best save now
8090         strong_lr = (strong == 'l') and 'l' or 'r'
8091     elseif new_dir then
8092         last_lr = nil
8093     end
8094 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8095     if last_lr and outer == 'r' then
8096         for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8097             if characters[ch.char] then
8098                 ch.char = characters[ch.char].m or ch.char
8099             end
8100         end
8101     end
8102     if first_n then
8103         dir_mark(head, first_n, last_n, outer)
8104     end
8105     if first_d then
8106         dir_mark(head, first_d, last_d, outer)
8107     end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8108     return node.prev(head) or head
8109 end
8110 (/basic-r)

```

And here the Lua code for bidi=basic:

```

8111 (*basic)
8112 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8113
8114 Babel.fontmap = Babel.fontmap or {}
8115 Babel.fontmap[0] = {}          -- l
8116 Babel.fontmap[1] = {}          -- r
8117 Babel.fontmap[2] = {}          -- al/an
8118
8119 -- To cancel mirroring. Also OML, OMS, U?
8120 Babel.symbol_fonts = Babel.symbol_fonts or {}
8121 Babel.symbol_fonts[font.id('tenln')] = true
8122 Babel.symbol_fonts[font.id('tenlnw')] = true
8123 Babel.symbol_fonts[font.id('tencirc')] = true
8124 Babel.symbol_fonts[font.id('tencircw')] = true
8125
8126 Babel.bidi_enabled = true
8127 Babel.mirroring_enabled = true
8128
8129 require('babel-data-bidi.lua')
8130
8131 local characters = Babel.characters
8132 local ranges = Babel.ranges
8133
8134 local DIR = node.id('dir')

```

```

8135 local GLYPH = node.id('glyph')
8136
8137 local function insert_implicit(head, state, outer)
8138   local new_state = state
8139   if state.sim and state.eim and state.sim ~= state.eim then
8140     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8141     local d = node.new(DIR)
8142     d.dir = '+' .. dir
8143     node.insert_before(head, state.sim, d)
8144     local d = node.new(DIR)
8145     d.dir = '-' .. dir
8146     node.insert_after(head, state.eim, d)
8147   end
8148   new_state.sim, new_state.eim = nil, nil
8149   return head, new_state
8150 end
8151
8152 local function insert_numeric(head, state)
8153   local new
8154   local new_state = state
8155   if state.san and state.ean and state.san ~= state.ean then
8156     local d = node.new(DIR)
8157     d.dir = '+TLT'
8158     _, new = node.insert_before(head, state.san, d)
8159     if state.san == state.sim then state.sim = new end
8160     local d = node.new(DIR)
8161     d.dir = '-TLT'
8162     _, new = node.insert_after(head, state.ean, d)
8163     if state.ean == state.eim then state.eim = new end
8164   end
8165   new_state.san, new_state.ean = nil, nil
8166   return head, new_state
8167 end
8168
8169 local function glyph_not_symbol_font(node)
8170   if node.id == GLYPH then
8171     return not Babel.symbol_fonts[node.font]
8172   else
8173     return false
8174   end
8175 end
8176
8177 -- TODO - \hbox with an explicit dir can lead to wrong results
8178 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8179 -- was made to improve the situation, but the problem is the 3-dir
8180 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8181 -- well.
8182
8183 function Babel.bidi(head, ispar, hdir)
8184   local d -- d is used mainly for computations in a loop
8185   local prev_d = ''
8186   local new_d = false
8187
8188   local nodes = {}
8189   local outer_first = nil
8190   local inmath = false
8191
8192   local glue_d = nil
8193   local glue_i = nil
8194
8195   local has_en = false
8196   local first_et = nil
8197

```

```

8198 local has_hyperlink = false
8199
8200 local ATDIR = Babel.attr_dir
8201 local attr_d, temp
8202 local locale_d
8203
8204 local save_outer
8205 local locale_d = node.get_attribute(head, ATDIR)
8206 if locale_d then
8207     locale_d = locale_d & 0x3
8208     save_outer = (locale_d == 0 and 'l') or
8209                 (locale_d == 1 and 'r') or
8210                 (locale_d == 2 and 'al')
8211 elseif ispar then -- Or error? Shouldn't happen
8212     -- when the callback is called, we are just _after_ the box,
8213     -- and the textdir is that of the surrounding text
8214     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8215 else -- Empty box
8216     save_outer = ('TRT' == hdir) and 'r' or 'l'
8217 end
8218 local outer = save_outer
8219 local last = outer
8220 -- 'al' is only taken into account in the first, current loop
8221 if save_outer == 'al' then save_outer = 'r' end
8222
8223 local fontmap = Babel.fontmap
8224
8225 for item in node.traverse(head) do
8226
8227     -- Mask: DxxxPPTT (Done, Pardir [0-2], Textdir [0-2])
8228     locale_d = node.get_attribute(item, ATDIR)
8229     node.set_attribute(item, ATDIR, 0x80)
8230
8231     -- In what follows, #node is the last (previous) node, because the
8232     -- current one is not added until we start processing the neutrals.
8233     -- three cases: glyph, dir, otherwise
8234     if glyph_not_symbol_font(item)
8235         or (item.id == 7 and item.subtype == 2) then
8236
8237         if inmath then goto nextnode end
8238
8239         if locale_d == 0x80 then goto nextnode end
8240         locale_d = locale_d or ((save_outer == 'l') and 0 or 1)
8241
8242         local d_font = nil
8243         local item_r
8244         if item.id == 7 and item.subtype == 2 then
8245             item_r = item.replace -- automatic discs have just 1 glyph
8246         else
8247             item_r = item
8248         end
8249
8250         local chardata = characters[item_r.char]
8251         d = chardata and chardata.d or nil
8252         if not d or d == 'nsm' then
8253             for nn, et in ipairs(ranges) do
8254                 if item_r.char < et[1] then
8255                     break
8256                 elseif item_r.char <= et[2] then
8257                     if not d then d = et[3]
8258                     elseif d == 'nsm' then d_font = et[3]
8259                     end
8260                     break

```

```

8261         end
8262     end
8263 end
8264 d = d or 'l'
8265
8266 -- A short 'pause' in bidi for mapfont
8267 -- %%% TODO. move if fontmap here
8268 d_font = d_font or d
8269 d_font = (d_font == 'l' and 0) or
8270           (d_font == 'nsm' and 0) or
8271           (d_font == 'r' and 1) or
8272           (d_font == 'al' and 2) or
8273           (d_font == 'an' and 2) or nil
8274 if d_font and fontmap and fontmap[d_font][item_r.font] then
8275     item_r.font = fontmap[d_font][item_r.font]
8276 end
8277
8278 if new_d then
8279     table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8280     if inmath then
8281         attr_d = 0
8282     else
8283         attr_d = locale_d & 0x3
8284     end
8285     if attr_d == 1 then
8286         outer_first = 'r'
8287         last = 'r'
8288     elseif attr_d == 2 then
8289         outer_first = 'r'
8290         last = 'al'
8291     else
8292         outer_first = 'l'
8293         last = 'l'
8294     end
8295     outer = last
8296     has_en = false
8297     first_et = nil
8298     new_d = false
8299 end
8300
8301 if glue_d then
8302     if (d == 'l' and 'l' or 'r') ~= glue_d then
8303         table.insert(nodes, {glue_i, 'on', nil})
8304     end
8305     glue_d = nil
8306     glue_i = nil
8307 end
8308
8309 elseif item.id == DIR then
8310     if inmath then goto nextnode end
8311     d = nil
8312     new_d = true
8313
8314 elseif item.id == node.id'glue' and item.subtype == 13 then
8315     if inmath then goto nextnode end
8316     glue_d = d
8317     glue_i = item
8318     d = nil
8319
8320 elseif item.id == node.id'math' then
8321     if item.subtype == 0 then -- mathon
8322         inmath = true
8323         d = 'on'

```

```

8324         table.insert(nodes, {item, 'on', outer_first})
8325         outer_first = nil
8326         goto nextnode
8327     else -- mathoff
8328         inmath = false
8329     end
8330
8331     elseif item.id == 8 and item.subtype == 19 then
8332         has_hyperlink = true
8333
8334     else
8335         d = nil
8336     end
8337
8338     -- AL <= EN/ET/ES      -- W2 + W3 + W6
8339     if last == 'al' and d == 'en' then
8340         d = 'an'          -- W3
8341     elseif last == 'al' and (d == 'et' or d == 'es') then
8342         d = 'on'          -- W6
8343     end
8344
8345     -- EN + CS/ES + EN      -- W4
8346     if d == 'en' and #nodes >= 2 then
8347         if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8348             and nodes[#nodes-1][2] == 'en' then
8349             nodes[#nodes][2] = 'en'
8350         end
8351     end
8352
8353     -- AN + CS + AN        -- W4 too, because uax9 mixes both cases
8354     if d == 'an' and #nodes >= 2 then
8355         if (nodes[#nodes][2] == 'cs')
8356             and nodes[#nodes-1][2] == 'an' then
8357             nodes[#nodes][2] = 'an'
8358         end
8359     end
8360
8361     -- ET/EN              -- W5 + W7->l / W6->on
8362     if d == 'et' then
8363         first_et = first_et or (#nodes + 1)
8364     elseif d == 'en' then
8365         has_en = true
8366         first_et = first_et or (#nodes + 1)
8367     elseif first_et then -- d may be nil here !
8368         if has_en then
8369             if last == 'l' then
8370                 temp = 'l' -- W7
8371             else
8372                 temp = 'en' -- W5
8373             end
8374         else
8375             temp = 'on' -- W6
8376         end
8377         for e = first_et, #nodes do
8378             if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8379         end
8380         first_et = nil
8381         has_en = false
8382     end
8383
8384     -- Force mathdir in math if ON (currently works as expected only
8385     -- with 'l')
8386

```

```

8387     if inmath and d == 'on' then
8388         d = ('TRT' == tex.mathdir) and 'r' or 'l'
8389     end
8390
8391     if d then
8392         if d == 'al' then
8393             d = 'r'
8394             last = 'al'
8395         elseif d == 'l' or d == 'r' then
8396             last = d
8397         end
8398         prev_d = d
8399         table.insert(nodes, {item, d, outer_first})
8400     end
8401
8402     outer_first = nil
8403
8404     ::nextnode::
8405
8406 end -- for each node
8407
8408 -- TODO -- repeated here in case EN/ET is the last node. Find a
8409 -- better way of doing things:
8410 if first_et then -- dir may be nil here !
8411     if has_en then
8412         if last == 'l' then
8413             temp = 'l' -- W7
8414         else
8415             temp = 'en' -- W5
8416         end
8417     else
8418         temp = 'on' -- W6
8419     end
8420     for e = first_et, #nodes do
8421         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8422     end
8423 end
8424
8425 -- dummy node, to close things
8426 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8427
8428 ----- NEUTRAL -----
8429
8430 outer = save_outer
8431 last = outer
8432
8433 local first_on = nil
8434
8435 for q = 1, #nodes do
8436     local item
8437
8438     local outer_first = nodes[q][3]
8439     outer = outer_first or outer
8440     last = outer_first or last
8441
8442     local d = nodes[q][2]
8443     if d == 'an' or d == 'en' then d = 'r' end
8444     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8445
8446     if d == 'on' then
8447         first_on = first_on or q
8448     elseif first_on then
8449         if last == d then

```

```

8450     temp = d
8451 else
8452     temp = outer
8453 end
8454 for r = first_on, q - 1 do
8455     nodes[r][2] = temp
8456     item = nodes[r][1]    -- MIRRORING
8457     if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8458         and temp == 'r' and characters[item.char] then
8459         local font_mode = ''
8460         if item.font > 0 and font.fonts[item.font].properties then
8461             font_mode = font.fonts[item.font].properties.mode
8462         end
8463         if font_mode ~= 'harf' and font_mode ~= 'plug' then
8464             item.char = characters[item.char].m or item.char
8465         end
8466     end
8467 end
8468     first_on = nil
8469 end
8470
8471 if d == 'r' or d == 'l' then last = d end
8472 end
8473
8474 ----- IMPLICIT, REORDER -----
8475
8476 outer = save_outer
8477 last = outer
8478
8479 local state = {}
8480 state.has_r = false
8481
8482 for q = 1, #nodes do
8483
8484     local item = nodes[q][1]
8485
8486     outer = nodes[q][3] or outer
8487
8488     local d = nodes[q][2]
8489
8490     if d == 'nsm' then d = last end          -- W1
8491     if d == 'en' then d = 'an' end
8492     local isdir = (d == 'r' or d == 'l')
8493
8494     if outer == 'l' and d == 'an' then
8495         state.san = state.san or item
8496         state.ean = item
8497     elseif state.san then
8498         head, state = insert_numeric(head, state)
8499     end
8500
8501     if outer == 'l' then
8502         if d == 'an' or d == 'r' then      -- im -> implicit
8503             if d == 'r' then state.has_r = true end
8504             state.sim = state.sim or item
8505             state.eim = item
8506         elseif d == 'l' and state.sim and state.has_r then
8507             head, state = insert_implicit(head, state, outer)
8508         elseif d == 'l' then
8509             state.sim, state.eim, state.has_r = nil, nil, false
8510         end
8511     else
8512         if d == 'an' or d == 'l' then

```

```

8513         if nodes[q][3] then -- nil except after an explicit dir
8514             state.sim = item -- so we move sim 'inside' the group
8515         else
8516             state.sim = state.sim or item
8517         end
8518         state.eim = item
8519     elseif d == 'r' and state.sim then
8520         head, state = insert_implicit(head, state, outer)
8521     elseif d == 'r' then
8522         state.sim, state.eim = nil, nil
8523     end
8524 end
8525
8526 if isdir then
8527     last = d -- Don't search back - best save now
8528 elseif d == 'on' and state.san then
8529     state.san = state.san or item
8530     state.ean = item
8531 end
8532
8533 end
8534
8535 head = node.prev(head) or head
8536 % \end{macrocode}
8537 %
8538 % Now direction nodes has been distributed with relation to characters
8539 % and spaces, we need to take into account \TeX-specific elements in
8540 % the node list, to move them at an appropriate place. Firstly, with
8541 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8542 % that the latter are still discardable.
8543 %
8544 % \begin{macrocode}
8545 --- FIXES ---
8546 if has_hyperlink then
8547     local flag, linking = 0, 0
8548     for item in node.traverse(head) do
8549         if item.id == DIR then
8550             if item.dir == '+TRT' or item.dir == '+TLT' then
8551                 flag = flag + 1
8552             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8553                 flag = flag - 1
8554             end
8555             elseif item.id == 8 and item.subtype == 19 then
8556                 linking = flag
8557             elseif item.id == 8 and item.subtype == 20 then
8558                 if linking > 0 then
8559                     if item.prev.id == DIR and
8560                         (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8561                         d = node.new(DIR)
8562                         d.dir = item.prev.dir
8563                         node.remove(head, item.prev)
8564                         node.insert_after(head, item, d)
8565                     end
8566                 end
8567                 linking = 0
8568             end
8569         end
8570     end
8571
8572 for item in node.traverse_id(10, head) do
8573     local p = item
8574     local flag = false
8575     while p.prev and p.prev.id == 14 do

```

```

8576     flag = true
8577     p = p.prev
8578 end
8579 if flag then
8580     node.insert_before(head, p, node.copy(item))
8581     node.remove(head,item)
8582 end
8583 end
8584
8585 return head
8586 end

8587 function Babel.unset_atdir(head)
8588     local ATDIR = Babel.attr_dir
8589     for item in node.traverse(head) do
8590         node.set_attribute(item, ATDIR, 0x80)
8591     end
8592     return head
8593 end

```

The following code has been contributed by Udi Fogiel. It fixes misplaced glues when combined with penalties at line breaks.

```

8594 local GLUE      = node.id'glue'
8595 local PENALTY    = node.id'penalty'
8596 local HLIST      = node.id'hlist'
8597 local LINE       = table.swapped(node.subtypes("hlist")).line
8598
8599 local characters = Babel.characters
8600 local ranges     = Babel.ranges
8601
8602 -- L1's whitespace set: WS + isolate formats + (per the UBA reference) the
8603 -- explicit embedding/override formats + BN.
8604 -- Babel does not use embedding/override/isolates, but well, I copy pasted it...
8605 local ll_whitespace = {
8606     ws = true, bn = true,
8607     lre = true, rle = true, lro = true, rlo = true, pdf = true,
8608     lri = true, rli = true, fsi = true, pdi = true,
8609 }
8610
8611 local function bidi_class(c)
8612     local cd = characters[c]
8613     local d = cd and cd.d
8614     if not d then
8615         for _, et in ipairs(ranges) do
8616             if c < et[1] then
8617                 break
8618             elseif c <= et[2] then
8619                 d = et[3]
8620                 break
8621             end
8622         end
8623     end
8624     return d or 'l'
8625 end
8626
8627 -- Nodes we step over as part of a trailing-whitespace run: glue,
8628 -- penalties, and a whitespace glyph.
8629 local function skippable(n)
8630     local id = n.id
8631     if id == GLUE or id == PENALTY then
8632         return true
8633     elseif id == GLYPH then
8634         return ll_whitespace[bidi_class(n.char)] == true
8635     end

```

```

8636 return false
8637 end
8638
8639 local function is_enddir(n)
8640   return n.id == DIR and n.subtype == 1
8641 end
8642
8643 -- Rule L1, clause 4, for one line: pull its trailing whitespace out past the
8644 -- closing dir run so it renders in the box (base) direction.
8645 local function reset_line(head)
8646   local last = node.slide(head)          -- tail, fixes prev links
8647   -- 1. skip the tail that follows the closing dir run (rightskip/parfillskip/...)
8648   local q = last
8649   while q and skippable(q) do q = q.prev end
8650   if not (q and is_enddir(q)) then return head end
8651   local d_last = q
8652   -- 2. step back over the whole closing dir run
8653   while q and q.id == DIR do q = q.prev end
8654   -- 3. what precedes the dirs must be the trailing whitespace run
8655   if not (q and skippable(q)) then return head end
8656   local w_last = q
8657   while q and skippable(q) do q = q.prev end
8658   local before_w = q
8659   if not before_w then return head end      -- whole line is whitespace; leave it
8660   local w_first = before_w.next
8661   local d_first = w_last.next
8662   local after_d = d_last.next
8663   -- detach W = [w_first .. w_last]
8664   before_w.next = d_first
8665   if d_first then d_first.prev = before_w end
8666   -- reinsert W after the dir run (outside every dir => box/base direction)
8667   d_last.next = w_first
8668   w_first.prev = d_last
8669   w_last.next = after_d
8670   if after_d then after_d.prev = w_last end
8671   return head
8672 end
8673
8674 function Babel.reset_lines(head)
8675   for line, subtype in node.traverse_id(HLIST, head) do
8676     if subtype == LINE then
8677       line.list = reset_line(line.list)
8678     end
8679   end
8680   return head
8681 end
8682
8683 luatexbase.add_to_callback('post_linebreak_filter',
8684   function(head, gc)
8685     if Babel.bidi_enabled then head = Babel.reset_lines(head) end
8686     return head
8687   end,
8688   'Babel.reset_lines')
8689 (/basic)

```

11. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},

```

```
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%
```

For the meaning of these codes, see the Unicode standard.

12. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```
8690 (*nil)
8691 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8692 \LdfInit{nil}{datenil}
```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```
8693 \ifx\l@nil\undefined
8694 \newlanguage\l@nil
8695 \namedef{bbl@hyphendata@the\l@nil}{}}}% Remove warning
8696 \let\bbl@elt\relax
8697 \edef\bbl@languages{% Add it to the list of languages
8698 \bbl@languages\bbl@elt{nil}{the\l@nil}{}}
8699 \fi
```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```
8700 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}
```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

`\captionnil`

`\datenil`

```
8701 \let\captionnil\@empty
8702 \let\datenil\@empty
```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```
8703 \def\bbl@inidata@nil{%
8704 \bbl@elt{identification}{tag.ini}{und}%
8705 \bbl@elt{identification}{load.level}{0}%
8706 \bbl@elt{identification}{charset}{utf8}%
8707 \bbl@elt{identification}{version}{1.0}%
8708 \bbl@elt{identification}{date}{2022-05-16}%
8709 \bbl@elt{identification}{name.local}{nil}%
8710 \bbl@elt{identification}{name.english}{nil}%
8711 \bbl@elt{identification}{name.babel}{nil}%
8712 \bbl@elt{identification}{tag.bcp47}{und}%
8713 \bbl@elt{identification}{language.tag.bcp47}{und}%
8714 \bbl@elt{identification}{tag.opentype}{dflt}%
8715 \bbl@elt{identification}{script.name}{Latin}%
8716 \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8717 \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8718 \bbl@elt{identification}{level}{1}%
8719 \bbl@elt{identification}{encodings}{}%
8720 \bbl@elt{identification}{derivate}{no}}
8721 \@namedef{bbl@tbc@nil}{und}
8722 \@namedef{bbl@lbc@nil}{und}
8723 \@namedef{bbl@casing@nil}{und}
8724 \@namedef{bbl@lotf@nil}{dflt}
```

```

8725 \@namedef{bbl@elname@nil}{nil}
8726 \@namedef{bbl@lname@nil}{nil}
8727 \@namedef{bbl@esname@nil}{Latin}
8728 \@namedef{bbl@sname@nil}{Latin}
8729 \@namedef{bbl@sbcpl@nil}{Latn}
8730 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8731 \ldf@finish{nil}
8732 (/nil)

```

13. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8733 ((*Compute Julian day)) ≡
8734 \def\bbl@fpmo#1#2{(#1-#2*floor((#1)/(#2)))}
8735 \def\bbl@cs@gregleap#1{%
8736   (\bbl@fpmo{#1}{4} == 0) &&
8737   (!((\bbl@fpmo{#1}{100} == 0) && (\bbl@fpmo{#1}{400} != 0)))}
8738 \def\bbl@cs@jd#1#2#3{% year, month, day
8739   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8740     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8741     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8742     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3} }%
8743 (/*Compute Julian day)

```

13.1. Islamic

The code for the Civil calendar is based on it, too.

```

8744 (*ca-islamic)
8745 <@Compute Julian day@>
8746 % == islamic (default)
8747 % Not yet implemented
8748 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{}

```

The Civil calendar.

```

8749 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8750   ((#3 + ceil(29.5 * (#2 - 1)) +
8751     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8752     1948439.5) - 1) }
8753 \@namedef{bbl@ca@islamic-civil++}{\bbl@ca@islamicvl{x}{+2}}
8754 \@namedef{bbl@ca@islamic-civil+}{\bbl@ca@islamicvl{x}{+1}}
8755 \@namedef{bbl@ca@islamic-civil}{\bbl@ca@islamicvl{x}{}}
8756 \@namedef{bbl@ca@islamic-civil-}{\bbl@ca@islamicvl{x}{-1}}
8757 \@namedef{bbl@ca@islamic-civil--}{\bbl@ca@islamicvl{x}{-2}}
8758 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8759   \edef\bbl@tempa{%
8760     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1}}%
8761     \edef#5{%
8762       \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8763       \edef#6{\fpeval{
8764         min(12,ceil((\bbl@tempa-(29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8765       \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable `\today`, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```

8766 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8767 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8768 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8769 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8770 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8771 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8772 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8773 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8774 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8775 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8776 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8777 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8778 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8779 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8780 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8781 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8782 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8783 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8784 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8785 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8786 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8787 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8788 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8789 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8790 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8791 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8792 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8793 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8794 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8795 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8796 65401,65431,65460,65490,65520}
8797 \@namedef\bbl@ca@islamic-umalqura+{\bbl@ca@islamcuqr@x{+1}}
8798 \@namedef\bbl@ca@islamic-umalqura-{\bbl@ca@islamcuqr@x{-1}}
8799 \@namedef\bbl@ca@islamic-umalqura-{\bbl@ca@islamcuqr@x{-1}}
8800 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8801 \ifnum#2>2014 \ifnum#2<2038
8802 \bbl@afterfi\expandafter@gobble
8803 \fi\fi
8804 {\bbl@error{year-out-range}{2014-2038}}}%
8805 \edef\bbl@tempd{\fpeval{ % (Julian) day
8806 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8807 \count@\@ne
8808 \bbl@foreach\bbl@cs@umalqura@data{%
8809 \advance\count@\@ne
8810 \ifnum##1>\bbl@tempd\else
8811 \edef\bbl@tempe{\the\count@}%
8812 \edef\bbl@tempb{##1}%
8813 \fi}%
8814 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8815 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1 ) / 12) }}% annus
8816 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8817 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8818 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8819 \bbl@add\bbl@precalendar{%
8820 \bbl@replace\bbl@ld@calendar{-civil}}}%
8821 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8822 \bbl@replace\bbl@ld@calendar{+}}}%
8823 \bbl@replace\bbl@ld@calendar{-}}}%
8824 (/ca-islamic)

```

13.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaption by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```
8825 (*ca-hebrew)
8826 \newcount\bbl@cntcommon
8827 \def\bbl@remainder#1#2#3{%
8828   #3=#1\relax
8829   \divide #3 by #2\relax
8830   \multiply #3 by -#2\relax
8831   \advance #3 by #1\relax}%
8832 \newif\ifbbl@divisible
8833 \def\bbl@checkifdivisible#1#2{%
8834   {\countdef\tmp=0
8835    \bbl@remainder{#1}{#2}{\tmp}%
8836    \ifnum \tmp=0
8837      \global\bbl@divisibletrue
8838    \else
8839      \global\bbl@divisiblefalse
8840    \fi}}
8841 \newif\ifbbl@gregleap
8842 \def\bbl@ifgregleap#1{%
8843   \bbl@checkifdivisible{#1}{4}%
8844   \ifbbl@divisible
8845     \bbl@checkifdivisible{#1}{100}%
8846     \ifbbl@divisible
8847       \bbl@checkifdivisible{#1}{400}%
8848       \ifbbl@divisible
8849         \bbl@gregleaptrue
8850       \else
8851         \bbl@gregleapfalse
8852       \fi
8853     \else
8854       \bbl@gregleaptrue
8855     \fi
8856   \else
8857     \bbl@gregleapfalse
8858   \fi
8859   \ifbbl@gregleap}
8860 \def\bbl@gregdayspriormonths#1#2#3{%
8861   {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8862     181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8863   \bbl@ifgregleap{#2}%
8864   \ifnum #1 > 2
8865     \advance #3 by 1
8866   \fi
8867   \fi
8868   \global\bbl@cntcommon=#3}%
8869   #3=\bbl@cntcommon}
8870 \def\bbl@gregdaysprioryears#1#2{%
8871   {\countdef\tmpc=4
8872    \countdef\tmpb=2
8873    \tmpb=#1\relax
8874    \advance \tmpb by -1
8875    \tmpc=\tmpb
8876    \multiply \tmpc by 365
8877    #2=\tmpc
8878    \tmpc=\tmpb
8879    \divide \tmpc by 4
8880    \advance #2 by \tmpc
8881    \tmpc=\tmpb
8882    \divide \tmpc by 100
```

```

8883 \advance #2 by -\tmpc
8884 \tmpc=\tmpb
8885 \divide \tmpc by 400
8886 \advance #2 by \tmpc
8887 \global\bbl@cntcommon=#2\relax}%
8888 #2=\bbl@cntcommon}
8889 \def\bbl@absfromgreg#1#2#3#4{%
8890 {\countdef\tmpd=0
8891 #4=#1\relax
8892 \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8893 \advance #4 by \tmpd
8894 \bbl@gregdaysprioryears{#3}{\tmpd}%
8895 \advance #4 by \tmpd
8896 \global\bbl@cntcommon=#4\relax}%
8897 #4=\bbl@cntcommon}
8898 \newif\ifbbl@hebrleap
8899 \def\bbl@checkleaphebryear#1{%
8900 {\countdef\tmpa=0
8901 \countdef\tmpb=1
8902 \tmpa=#1\relax
8903 \multiply \tmpa by 7
8904 \advance \tmpa by 1
8905 \bbl@remainder{\tmpa}{19}{\tmpb}%
8906 \ifnum \tmpb < 7
8907 \global\bbl@hebrleaptrue
8908 \else
8909 \global\bbl@hebrleapfalse
8910 \fi}}
8911 \def\bbl@hebrleapsedmonths#1#2{%
8912 {\countdef\tmpa=0
8913 \countdef\tmpb=1
8914 \countdef\tmpc=2
8915 \tmpa=#1\relax
8916 \advance \tmpa by -1
8917 #2=\tmpa
8918 \divide #2 by 19
8919 \multiply #2 by 235
8920 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8921 \tmpc=\tmpb
8922 \multiply \tmpb by 12
8923 \advance #2 by \tmpb
8924 \multiply \tmpc by 7
8925 \advance \tmpc by 1
8926 \divide \tmpc by 19
8927 \advance #2 by \tmpc
8928 \global\bbl@cntcommon=#2}%
8929 #2=\bbl@cntcommon}
8930 \def\bbl@hebrleapseddays#1#2{%
8931 {\countdef\tmpa=0
8932 \countdef\tmpb=1
8933 \countdef\tmpc=2
8934 \bbl@hebrleapsedmonths{#1}{#2}%
8935 \tmpa=#2\relax
8936 \multiply \tmpa by 13753
8937 \advance \tmpa by 5604
8938 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8939 \divide \tmpa by 25920
8940 \multiply #2 by 29
8941 \advance #2 by 1
8942 \advance #2 by \tmpa
8943 \bbl@remainder{#2}{7}{\tmpa}%
8944 \ifnum \tmpc < 19440
8945 \ifnum \tmpc < 9924

```

```

8946     \else
8947         \ifnum \tmpa=2
8948             \bbl@checkleaphebyear{#1}% of a common year
8949             \ifbbl@hebrleap
8950                 \else
8951                     \advance #2 by 1
8952                 \fi
8953             \fi
8954         \fi
8955     \ifnum \tmpc < 16789
8956     \else
8957         \ifnum \tmpa=1
8958             \advance #1 by -1
8959             \bbl@checkleaphebyear{#1}% at the end of leap year
8960             \ifbbl@hebrleap
8961                 \advance #2 by 1
8962             \fi
8963         \fi
8964     \fi
8965 \else
8966     \advance #2 by 1
8967 \fi
8968 \bbl@remainder{#2}{7}{\tmpa}%
8969 \ifnum \tmpa=0
8970     \advance #2 by 1
8971 \else
8972     \ifnum \tmpa=3
8973         \advance #2 by 1
8974     \else
8975         \ifnum \tmpa=5
8976             \advance #2 by 1
8977         \fi
8978     \fi
8979 \fi
8980 \global\bbl@cntcommon=#2\relax}%
8981 #2=\bbl@cntcommon}
8982 \def\bbl@daysinhebyear#1#2{%
8983 {\countdef\tmpe=12
8984  \bbl@hebreleaseddays{#1}{\tmpe}%
8985  \advance #1 by 1
8986  \bbl@hebreleaseddays{#1}{#2}%
8987  \advance #2 by -\tmpe
8988  \global\bbl@cntcommon=#2}%
8989 #2=\bbl@cntcommon}
8990 \def\bbl@hebrdayspriormonths#1#2#3{%
8991 {\countdef\tmpf= 14
8992  #3=\ifcase #1
8993      0 \or
8994      0 \or
8995      30 \or
8996      59 \or
8997      89 \or
8998      118 \or
8999      148 \or
9000      148 \or
9001      177 \or
9002      207 \or
9003      236 \or
9004      266 \or
9005      295 \or
9006      325 \or
9007      400
9008  \fi

```

```

9009 \bbl@checkleaphebryear{#2}%
9010 \ifbbl@hebrleap
9011     \ifnum #1 > 6
9012         \advance #3 by 30
9013     \fi
9014 \fi
9015 \bbl@daysinhebryear{#2}{\tmpf}%
9016 \ifnum #1 > 3
9017     \ifnum \tmpf=353
9018         \advance #3 by -1
9019     \fi
9020     \ifnum \tmpf=383
9021         \advance #3 by -1
9022     \fi
9023 \fi
9024 \ifnum #1 > 2
9025     \ifnum \tmpf=355
9026         \advance #3 by 1
9027     \fi
9028     \ifnum \tmpf=385
9029         \advance #3 by 1
9030     \fi
9031 \fi
9032 \global\bbl@cntcommon=#3\relax}%
9033 #3=\bbl@cntcommon}
9034 \def\bbl@absfromhebr#1#2#3#4{%
9035 {#4=#1\relax
9036 \bbl@hebrdayspriormonths{#2}{#3}{#1}%
9037 \advance #4 by #1\relax
9038 \bbl@hebrrelapseddays{#3}{#1}%
9039 \advance #4 by #1\relax
9040 \advance #4 by -1373429
9041 \global\bbl@cntcommon=#4\relax}%
9042 #4=\bbl@cntcommon}
9043 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
9044 {\countdef\tmpx= 17
9045 \countdef\tmpy= 18
9046 \countdef\tmpz= 19
9047 #6=#3\relax
9048 \global\advance #6 by 3761
9049 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
9050 \tmpz=1 \tmpy=1
9051 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9052 \ifnum \tmpx > #4\relax
9053     \global\advance #6 by -1
9054     \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9055 \fi
9056 \advance #4 by -\tmpx
9057 \advance #4 by 1
9058 #5=#4\relax
9059 \divide #5 by 30
9060 \loop
9061     \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
9062     \ifnum \tmpx < #4\relax
9063         \advance #5 by 1
9064         \tmpy=\tmpx
9065     \repeat
9066 \global\advance #5 by -1
9067 \global\advance #4 by -\tmpy}}
9068 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebryear
9069 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
9070 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
9071 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax

```

```

9072 \bbl@hebrfromgreg
9073 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
9074 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebyear}%
9075 \edef#4{\the\bbl@hebyear}%
9076 \edef#5{\the\bbl@hebrmonth}%
9077 \edef#6{\the\bbl@hebrday}%
9078 (/ca-hebrew)

```

13.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

9079 (*ca-persian)
9080 <@Compute Julian day@>
9081 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
9082 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
9083 \def\bbl@ca@persian#1-#2-#3\@@#4#5#6{%
9084 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
9085 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
9086 \bbl@afterfi\expandafter\@gobble
9087 \fi\fi
9088 {\bbl@error{year-out-range}{2013-2050}{}}}%
9089 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9090 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9091 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
9092 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
9093 \ifnum\bbl@tempc<\bbl@tempb
9094 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
9095 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9096 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9097 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
9098 \fi
9099 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
9100 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
9101 \edef#5{\fpeval{% set Jalali month
9102 (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
9103 \edef#6{\fpeval{% set Jalali day
9104 (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : ((#5 - 1) * 30) + 6))}}
9105 (/ca-persian)

```

13.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

9106 (*ca-coptic)
9107 <@Compute Julian day@>
9108 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
9109 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9110 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
9111 \edef#4{\fpeval{%
9112 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9113 \edef\bbl@tempc{\fpeval{%
9114 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
9115 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9116 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}
9117 (/ca-coptic)
9118 (*ca-ethiopic)
9119 <@Compute Julian day@>
9120 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%

```

```

9121 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9122 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
9123 \edef#4{\fpeval{%
9124     floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9125 \edef\bbl@tempc{\fpeval{%
9126     \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
9127 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9128 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9129 (/ca-ethiopic)

```

13.5. Julian

Based on [ReinDersh].

```

9130 (*ca-julian)
9131 <@Compute Julian day@>
9132 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
9133     \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9134     \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9135     \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9136     \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9137     \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9138     \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9139     \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9140     \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}%
9141 (/ca-julian)

```

The Indian National Calendar, Saka era.

```

9142 (*ca-indian)
9143 <@Compute Julian day@>
9144 \def\bbl@sakachaitra#1{%
9145     \bbl@cs@jd{#1}{3}%
9146     \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100}!=0)
9147     || \bbl@fpmmod{#1}{400}==0) ? 21 : 22 }}%
9148 \def\bbl@ca@indian#1-#2-#3\@@#4#5#6{%
9149     \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%
9150     \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9151     \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9152     \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9153     \edef\bbl@tempe{\fpeval{ ((\bbl@fpmmod{\bbl@tempc}{4})==0 &&
9154     \bbl@fpmmod{\bbl@tempc}{100}!=0) || \bbl@fpmmod{\bbl@tempc}{400}==0) ? 31 : 30 }}%
9155     \edef#4{\fpeval{\bbl@tempc-78}}%
9156     \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9157     (\bbl@tempd-\bbl@tempe < 155 ?
9158     2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9159     7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0)}}}%
9160     \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9161     (\bbl@tempd-\bbl@tempe < 155 ?
9162     \bbl@fpmmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9163     \bbl@fpmmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}%
9164 (/ca-indian)
9165 %

```

13.6. Buddhist

That's very simple.

```

9166 (*ca-buddhist)
9167 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9168     \edef#4{\number\numexpr#1+543\relax}%
9169     \edef#5{#2}%
9170     \edef#6{#3}%
9171 (/ca-buddhist)
9172 %

```

```

9173 % \subsection{Chinese}
9174 %
9175 % Brute force, with the Julian day of first day of each month. The
9176 % table has been computed with the help of \textsf{python-lunardate} by
9177 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9178 % is 2015-2044.
9179 %
9180 % \begin{macrocode}
9181 (*ca-chinese)
9182 \ExplSyntaxOn
9183 <@Compute Julian day@>
9184 \def\bbl@ca@chinese#1-#2-#3\@#4#5#6{%
9185 \edef\bbl@tempd{\fpeval{%
9186 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9187 \count@ \z@
9188 \@tempcnta=2015
9189 \bbl@foreach\bbl@cs@chinese@data{%
9190 \ifnum##1>\bbl@tempd\else
9191 \advance\count@\@ne
9192 \ifnum\count@>12
9193 \count@\@ne
9194 \advance\@tempcnta\@ne\fi
9195 \bbl@xin@{,##1,}{,\bbl@cs@chinese@leap,}%
9196 \ifin@
9197 \advance\count@\m@ne
9198 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9199 \else
9200 \edef\bbl@tempe{\the\count@}%
9201 \fi
9202 \edef\bbl@tempb{##1}%
9203 \fi}%
9204 \edef#4{\the\@tempcnta}%
9205 \edef#5{\bbl@tempe}%
9206 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9207 \def\bbl@cs@chinese@leap{%
9208 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9209 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9210 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9211 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9212 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9213 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9214 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9215 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9216 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9217 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9218 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9219 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9220 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9221 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9222 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9223 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9224 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9225 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9226 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9227 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9228 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9229 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9230 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9231 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9232 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9233 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9234 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9235 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%

```

```

9236 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9237 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9238 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9239 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9240 10896,10926,10956,10986,11015,11045,11074,11103}
9241 \ExplSyntaxOff
9242 (/ca-chinese)

```

14. Support for Plain T_EX (plain.def)

14.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `localhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `lplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `lplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9243 (*bplain|lplain)
9244 \catcode`\{=1 % left brace is begin-group character
9245 \catcode`\}=2 % right brace is end-group character
9246 \catcode`\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```

9247 \openin 0 hyphen.cfg
9248 \ifeof0
9249 \else
9250   \let\input

```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that’s done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```

9251 \def\input #1 {%
9252   \let\input\input
9253   \a hyphen.cfg
9254   \let\input\input
9255 }
9256 \fi
9257 (/bplain|lplain)

```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```

9258 (bplain)\a plain.tex
9259 (lplain)\a lplain.tex

```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```

9260 (bplain)\def\fmtname{babel-plain}
9261 (lplain)\def\fmtname{babel-lplain}

```

When you are using a different format, based on `plain.tex` you can make a copy of `lplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2. Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX} 2_{\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9262 <(<*Emulate LaTeX)> ≡
9263 \def\@empty{}
9264 \def\loadlocalcfg#1{%
9265   \openin0#1.cfg
9266   \ifeof0
9267     \closein0
9268   \else
9269     \closein0
9270     {\immediate\write16{*****}%
9271      \immediate\write16{* Local config file #1.cfg used}%
9272      \immediate\write16{*}%
9273     }
9274     \input #1.cfg\relax
9275   \fi
9276   \@endoflfd}
```

14.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```
9277 \long\def\@firstofone#1{#1}
9278 \long\def\@firstoftwo#1#2{#1}
9279 \long\def\@secondoftwo#1#2{#2}
9280 \def\@nnil{\@nil}
9281 \def\@gobbletwo#1#2{}
9282 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9283 \def\@staror@long#1{%
9284   \@ifstar
9285   {\let\l@ngrel@x\relax#1}%
9286   {\let\l@ngrel@x\long#1}}
9287 \let\l@ngrel@x\relax
9288 \def\@car#1#2\@nil{#1}
9289 \def\@cdr#1#2\@nil{#2}
9290 \let\@typeset@protect\relax
9291 \let\protected@edef\edef
9292 \long\def\@gobble#1{}
9293 \edef\@backslashchar{\expandafter\@gobble\string\}
9294 \def\strip@prefix#1>{}
9295 \def\g@addto@macro#1#2{{%
9296   \toks@\expandafter{#1#2}%
9297   \xdef#1{\the\toks@}}}
9298 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9299 \def\@nameuse#1{\csname #1\endcsname}
9300 \def\@ifundefined#1{%
9301   \expandafter\ifx\csname#1\endcsname\relax
9302     \expandafter\@firstoftwo
9303   \else
9304     \expandafter\@secondoftwo
9305   \fi}
9306 \def\@expandtwoargs#1#2#3{%
9307   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9308 \def\zap@space#1 #2{%
9309   #1%
9310   \ifx#2\@empty\else\expandafter\zap@space\fi
9311   #2}
9312 \let\bbl@trace\@gobble
9313 \def\bbl@error#1{% Implicit #2#3#4}
```

```

9314 \begingroup
9315 \catcode\`=0 \catcode\==12 \catcode\`=12
9316 \catcode\^^M=5 \catcode\%=14
9317 \input errbabel.def
9318 \endgroup
9319 \bbl@error{#1}}
9320 \def\bbl@warning#1{%
9321 \begingroup
9322 \newlinechar=\`^^J
9323 \def\{^^J(babel) }%
9324 \message{\`#1}%
9325 \endgroup}
9326 \let\bbl@infowarn\bbl@warning
9327 \def\bbl@info#1{%
9328 \begingroup
9329 \newlinechar=\`^^J
9330 \def\{^^J}%
9331 \wlog{#1}%
9332 \endgroup}

```

$\LaTeX 2\epsilon$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9333 \ifx\@preamblecmds\undefined
9334 \def\@preamblecmds{}
9335 \fi
9336 \def\@onlypreamble#1{%
9337 \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9338 \@preamblecmds\do#1}}
9339 \@onlypreamble\@onlypreamble

```

Mimic $\LaTeX$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9340 \def\begindocument{%
9341 \@begindocumenthook
9342 \global\let\@begindocumenthook\undefined
9343 \def\do##1{\global\let##1\undefined}%
9344 \@preamblecmds
9345 \global\let\do\noexpand}
9346 \ifx\@begindocumenthook\undefined
9347 \def\@begindocumenthook{}
9348 \fi
9349 \@onlypreamble\@begindocumenthook
9350 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\LaTeX$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endofldf`.

```

9351 \def\AtEndOfPackage#1{\g@addto@macro\@endofldf{#1}}
9352 \@onlypreamble\AtEndOfPackage
9353 \def\@endofldf{}
9354 \@onlypreamble\@endofldf
9355 \let\bbl@afterlang\@empty
9356 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9357 \catcode\&=\z@
9358 \ifx&\if@files\undefined
9359 \expandafter\let\csname if@files\expandafter\endcsname
9360 \csname iffalse\endcsname
9361 \fi
9362 \catcode\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9363 \def\newcommand{\@star@or@long\new@command}
9364 \def\new@command#1{%
9365   \@testopt{\@newcommand#1}0}
9366 \def\@newcommand#1[#2]{%
9367   \@ifnextchar [{\@xargdef#1[#2]}%
9368     {\@argdef#1[#2]}}
9369 \long\def\@argdef#1[#2]#3{%
9370   \@yargdef#1\@ne{#2}{#3}}
9371 \long\def\@xargdef#1[#2][#3]#4{%
9372   \expandafter\def\expandafter#1\expandafter{%
9373     \expandafter\@protected@testopt\expandafter #1%
9374     \csname\string#1\expandafter\endcsname{#3}}}%
9375   \expandafter\@yargdef \csname\string#1\endcsname
9376   \tw@{#2}{#4}}
9377 \long\def\@yargdef#1#2#3{%
9378   \@tempcnta#3\relax
9379   \advance \@tempcnta \@ne
9380   \let\@hash@\relax
9381   \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9382   \@tempcntb #2%
9383   \@whilenum\@tempcntb <\@tempcnta
9384   \do{%
9385     \edef\reserved@a{\reserved@a\@hash@the\@tempcntb}%
9386     \advance\@tempcntb \@ne}%
9387   \let\@hash@###%
9388   \l@ngrelx\expandafter\def\expandafter#1\reserved@a}
9389 \def\providecommand{\@star@or@long\provide@command}
9390 \def\provide@command#1{%
9391   \begingroup
9392     \escapechar\m@ne\xdef\@gtempa{\string#1}%
9393   \endgroup
9394   \expandafter\ifundefined\@gtempa
9395     {\def\reserved@a{\new@command#1}}%
9396     {\let\reserved@a\relax
9397       \def\reserved@a{\new@command\reserved@a}}%
9398   \reserved@a}%

9399 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9400 \def\declare@robustcommand#1{%
9401   \edef\reserved@a{\string#1}%
9402   \def\reserved@b{#1}%
9403   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9404   \edef#1{%
9405     \ifx\reserved@a\reserved@b
9406       \noexpand\x@protect
9407       \noexpand#1%
9408     \fi
9409     \noexpand\protect
9410     \expandafter\noexpand\csname
9411       \expandafter\@gobble\string#1 \endcsname
9412   }%
9413   \expandafter\new@command\csname
9414     \expandafter\@gobble\string#1 \endcsname
9415 }
9416 \def\x@protect#1{%
9417   \ifx\protect\@typeset@protect\else
9418     \@x@protect#1%
9419   \fi
9420 }
9421 \catcode`\&=\z@ % Trick to hide conditionals
9422 \def\@x@protect#1&\fi#2#3{&\fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`, allocating a new boolean inside conditionally

executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9423 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9424 \catcode`\&=4
9425 \ifx\in@\@undefined
9426 \def\in@#1#2{%
9427 \def\in@@##1#1##2##3\in@@{%
9428 \ifx\in@@##2\in@false\else\in@true\fi}%
9429 \in@@##2#1\in@\in@@}
9430 \else
9431 \let\bbl@tempa\@empty
9432 \fi
9433 \bbl@tempa

```

$\TeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (activegrave and activeacute). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9434 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\TeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9435 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\TeX 2_{\epsilon}$ versions; just enough to make things work in plain $\TeX$ environments.

```

9436 \ifx\@tempcnta\@undefined
9437 \csname newcount\endcsname\@tempcnta\relax
9438 \fi
9439 \ifx\@tempcntb\@undefined
9440 \csname newcount\endcsname\@tempcntb\relax
9441 \fi

```

To prevent wasting two counters in $\TeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9442 \ifx\bye\@undefined
9443 \advance\count10 by -2\relax
9444 \fi
9445 \ifx\@ifnextchar\@undefined
9446 \def\@ifnextchar#1#2#3{%
9447 \let\reserved@d=#1%
9448 \def\reserved@a{#2}\def\reserved@b{#3}%
9449 \futurelet\@let@token\@ifnch}
9450 \def\@ifnch{%
9451 \ifx\@let@token\@sptoken
9452 \let\reserved@c\@xifnch
9453 \else
9454 \ifx\@let@token\reserved@d
9455 \let\reserved@c\reserved@a
9456 \else
9457 \let\reserved@c\reserved@b
9458 \fi
9459 \fi
9460 \reserved@c}
9461 \def\:{\let\@sptoken= }\: % this makes \@sptoken a space token
9462 \def\:{\@xifnch} \expandafter\def\:{\futurelet\@let@token\@ifnch}
9463 \fi
9464 \def\@testopt#1#2{%
9465 \@ifnextchar[#{1}{#1[#{2}]}}
9466 \def\@protected@testopt#1{%
9467 \ifx\protect\@typeset@protect
9468 \expandafter\@testopt

```

```

9469 \else
9470 \@@protect#1%
9471 \fi}
9472 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9473 #2\relax}\fi}
9474 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9475 \else\expandafter\@gobble\fi{#1}}

```

14.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9476 \def\DeclareTextCommand{%
9477 \@@dec@text@cmd\providecommand
9478 }
9479 \def\ProvideTextCommand{%
9480 \@@dec@text@cmd\providecommand
9481 }
9482 \def\DeclareTextSymbol#1#2#3{%
9483 \@@dec@text@cmd\chardef#1{#2}#3\relax
9484 }
9485 \def\@dec@text@cmd#1#2#3{%
9486 \expandafter\def\expandafter#2%
9487 \expandafter{%
9488 \csname#3-cmd\expandafter\endcsname
9489 \expandafter#2%
9490 \csname#3\string#2\endcsname
9491 }%
9492 % \let\@ifdefinable\@rc@ifdefinable
9493 \expandafter#1\csname#3\string#2\endcsname
9494 }
9495 \def\@current@cmd#1{%
9496 \ifx\protect\@typeset@protect\else
9497 \noexpand#1\expandafter\@gobble
9498 \fi
9499 }
9500 \def\@changed@cmd#1#2{%
9501 \ifx\protect\@typeset@protect
9502 \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9503 \expandafter\ifx\csname ?\string#1\endcsname\relax
9504 \expandafter\def\csname ?\string#1\endcsname{%
9505 \@changed@x@err{#1}%
9506 }%
9507 \fi
9508 \global\expandafter\let
9509 \csname\cf@encoding\string#1\expandafter\endcsname
9510 \csname ?\string#1\endcsname
9511 \fi
9512 \csname\cf@encoding\string#1%
9513 \expandafter\endcsname
9514 \else
9515 \noexpand#1%
9516 \fi
9517 }
9518 \def\@changed@x@err#1{%
9519 \errhelp{Your command will be ignored, type <return> to proceed}%
9520 \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9521 \def\DeclareTextCommandDefault#1{%
9522 \DeclareTextCommand#1?%
9523 }
9524 \def\ProvideTextCommandDefault#1{%
9525 \ProvideTextCommand#1?%
9526 }
9527 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd

```

```

9528 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9529 \def\DeclareTextAccent#1#2#3{%
9530   \DeclareTextCommand#1{#2}[1]{\accent#3 #1}
9531 }
9532 \def\DeclareTextCompositeCommand#1#2#3#4{%
9533   \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9534   \edef\reserved@b{\string##1}%
9535   \edef\reserved@c{%
9536     \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9537   \ifx\reserved@b\reserved@c
9538     \expandafter\expandafter\expandafter\ifx
9539       \expandafter\@car\reserved@a\relax\relax\@nil
9540       \@text@composite
9541   \else
9542     \edef\reserved@b##1{%
9543       \def\expandafter\noexpand
9544         \csname#2\string#1\endcsname###1{%
9545           \noexpand\@text@composite
9546             \expandafter\noexpand\csname#2\string#1\endcsname
9547             ###1\noexpand\@empty\noexpand\@text@composite
9548             {##1}%
9549       }%
9550     }%
9551     \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9552   \fi
9553   \expandafter\def\csname\expandafter\string\csname
9554     #2\endcsname\string#1-\string#3\endcsname{#4}
9555 \else
9556   \errhelp{Your command will be ignored, type <return> to proceed}%
9557   \errmessage{\string\DeclareTextCompositeCommand\space used on
9558     inappropriate command \protect#1}
9559 \fi
9560 }
9561 \def\@text@composite#1#2#3\@text@composite{%
9562   \expandafter\@text@composite@x
9563     \csname\string#1-\string#2\endcsname
9564 }
9565 \def\@text@composite@x#1#2{%
9566   \ifx#1\relax
9567     #2%
9568   \else
9569     #1%
9570   \fi
9571 }
9572 %
9573 \def\@strip@args#1:#2-#3\@strip@args{#2}
9574 \def\DeclareTextComposite#1#2#3#4{%
9575   \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9576   \bgroup
9577     \lccode`\@=#4%
9578     \lowercase{%
9579   \egroup
9580     \reserved@a @%
9581   }%
9582 }
9583 %
9584 \def\UseTextSymbol#1#2{#2}
9585 \def\UseTextAccent#1#2#3{}
9586 \def\@use@text@encoding#1{}
9587 \def\DeclareTextSymbolDefault#1#2{%
9588   \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9589 }
9590 \def\DeclareTextAccentDefault#1#2{%

```

```

9591 \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9592 }
9593 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX 2}_\epsilon$ method for accents for those that are known to be made active in *some* language definition file.

```

9594 \DeclareTextAccent{"}{OT1}{127}
9595 \DeclareTextAccent{'}{OT1}{19}
9596 \DeclareTextAccent{^}{OT1}{94}
9597 \DeclareTextAccent{\`}{OT1}{18}
9598 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for PLAIN $\text{\TeX}$.

```

9599 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9600 \DeclareTextSymbol{\textquotedblright}{OT1}{`\'}
9601 \DeclareTextSymbol{\textquoteleft}{OT1}{`\'}
9602 \DeclareTextSymbol{\textquoteright}{OT1}{`\'}
9603 \DeclareTextSymbol{\i}{OT1}{16}
9604 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because plain $\text{\TeX}$ doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9605 \ifx\scriptsize\undefined
9606 \let\scriptsize\sevenrm
9607 \fi

```

And a few more “dummy” definitions.

```

9608 \def\language{english}%
9609 \let\bbl@opt@shorthands\@nnil
9610 \def\bbl@ifshorthand#1#2#3{#2}%
9611 \let\bbl@language@opts\@empty
9612 \let\bbl@provide@locale\relax
9613 \ifx\babeloptionstrings\undefined
9614 \let\bbl@opt@strings\@nnil
9615 \else
9616 \let\bbl@opt@strings\babeloptionstrings
9617 \fi
9618 \def\BabelStringsDefault{generic}
9619 \def\bbl@tempa{normal}
9620 \ifx\babeloptionmath\bbl@tempa
9621 \def\bbl@mathnormal{\noexpand\textormath}
9622 \fi
9623 \def\AfterBabelLanguage#1#2{}
9624 \ifx\BabelModifiers\undefined\let\BabelModifiers\relax\fi
9625 \let\bbl@afterlang\relax
9626 \def\bbl@opt@safe{BR}
9627 \ifx\@uclclist\undefined\let\@uclclist\@empty\fi
9628 \ifx\bbl@trace\undefined\def\bbl@trace#1{}\fi
9629 \expandafter\newif\csname ifbbl@single\endcsname
9630 \chardef\bbl@bidimode\z@
9631 {/Emulate LaTeX}

```

A proxy file:

```

9632 {*plain}
9633 \input babel.def
9634 {/plain}

```

15. Acknowledgements

In the initial stages of the development of `babel`, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\LaTeX$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\TeX$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\LaTeX$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\TeX$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018
- [10] Hubert Partl, *German $\TeX$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\LaTeX$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\LaTeX$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer; een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).